



Desarrollo de un prototipo de aplicación móvil para android utilizando redes neuronales profundas CNN-LSTM para la detección de clonación de voz mediante deepfakes como medida de prevención en ciberseguridad

Angelo Josue Yanacallo Monta y Diego Alexander Páez Zapata

Departamento de Ciencias de la Computación

Carrera de Software

Trabajo de integración curricular, previo a la obtención del título de Ingeniero en Software

Gualotuña Álvarez Tatiana Marisol, Ph.D

4 de marzo de 2026



Ing. Gualotuña Álvarez Tatiana Marisol, Ph.D

Director/a



Departamento de Ciencias de la Computación

Carrera de Software

Certificación

Certifico que el trabajo de titulación: **“Desarrollo de un prototipo de aplicación móvil para android utilizando redes neuronales profundas CNN-LSTM para la detección de clonación de voz mediante deepfakes como medida de prevención en ciberseguridad”** fue realizado por los señores **Yanacallo Monta Angelo Josue y Páez Zapata Diego Alexander**, el mismo que cumple con los requisitos legales, teóricos, científicos, técnicos y metodológicos establecidos por la Universidad de las Fuerzas Armadas ESPE, además fue revisado y analizada en su totalidad por la herramienta de prevención y/o verificación de similitud de contenidos; razón por la cual me permito acreditar y autorizar para que se lo sustente públicamente.

Sangolquí, 4 de marzo de 2026

Ing. Gualotuña Álvarez Tatiana Marisol, Ph.D

Director/a

C.C.: 1711498418



Departamento de Ciencias de la Computación

Carrera de Software

Responsabilidad de Autoría

Nosotros, **Yanacallo Monta Angelo Josue y Páez Zapata Diego Alexander**, con cédula/cédulas de ciudadanía n°1751411628 y n°1750343210, declaramos que el contenido, ideas y criterios del trabajo de titulación: **Desarrollo de un prototipo de aplicación móvil para android utilizando redes neuronales profundas CNN-LSTM para la detección de clonación de voz mediante deepfakes como medida de prevención en ciberseguridad** es de nuestra autoría y responsabilidad, cumpliendo con los requisitos legales, teóricos, científicos, técnicos, y metodológicos establecidos por la Universidad de las Fuerzas Armadas ESPE, respetando los derechos intelectuales de terceros y referenciando las citas bibliográficas.

Sangolquí, 4 de marzo de 2026

Yanacallo Monta Angelo Josue

C.C.: 1751411628

Páez Zapata Diego Alexander

C.C: 1750343210



Departamento de Ciencias de la Computación

Carrera de Software

Autorización de Publicación

Nosotros **Yanacallo Monta Angelo Josue y Páez Zapata Diego Alexander**, con cédula/cédulas de ciudadanía n°1751411628 y n°1750343210 , autorizamos a la Universidad de las Fuerzas Armadas ESPE publicar el trabajo de titulación: **Desarrollo de un prototipo de aplicación móvil para android utilizando redes neuronales profundas CNN-LSTM para la detección de clonación de voz mediante deepfakes como medida de prevención en ciberseguridad** en el Repositorio Institucional, cuyo contenido, ideas y criterios son de mi/nuestra responsabilidad.

Sangolquí, 4 de marzo de 2026

Yanacallo Monta Angelo Josue

C.C.: 1751411628

Páez Zapata Diego Alexander

C.C.: 1750343210

Dedicatoria

Dedico este trabajo a Dios, quien ha sido el pilar fundamental en toda mi vida. A mis padres, Trinidad Monta Y Victor Haro, por apoyarme siempre incondicionalmente, durante mi niñez, mi adolescencia y ahora en una de las etapas más importantes de mi vida, por brindarme muchos consejos, por enseñarme a ser una persona con valores, por llenarme de amor, seguridad y paz todo el tiempo. A mi hermano, Maximiliano Haro, por ser el niño que alegra mi vida, quien día a día me llena de fuerzas para seguir adelante. Dedico este triunfo a mi hermana, Celeste Haro, quien partió de este mundo, pero que fue, es y seguirá siendo mi pequeña princesa y que desde el cielo está orgullosa de mí. Y, para terminar, quiero dedicar este logro a mi persona, ya que hubo momentos donde intenté tirar la toalla, pero decidí ser fuerte y continuar sin importar que tan difícil fuera.

-Yanacallo Monta Angelo Josue

Dedico este trabajo, en primer lugar, a mis padres, Edgar Páez y Miryan Zapata, quienes con su constante esfuerzo han sido pilares fundamentales a lo largo de mi vida académica y personal. Gracias por creer en mí, por sus consejos y por enseñarme, con su ejemplo, el valor del sacrificio, la responsabilidad y la perseverancia.

A mi hermana, Alejandra Páez por su cariño, compañía y apoyo incondicional, que han sido una fuente constante de motivación para seguir adelante.

De manera especial, dedico este trabajo a todos los ingenieros y profesores que compartieron sus conocimientos, experiencias y enseñanzas las cuales han sido fundamentales para mi crecimiento tanto profesional como personal.

Finalmente, me dedico este logro a mí mismo, por no rendirme ante las dificultades y mantenerme firme hasta alcanzar este objetivo.

-Páez Zapata Diego Alexander

Agradecimiento

Quisiera expresar nuevamente de todo corazón el mayor agradecimiento a mis padres y a toda mi familia, quienes me apoyaron desde el inicio hasta el final para conseguir este logro, me brindaron sus consejos, experiencias, fallos para que yo no me diera por vencido y por creer en mi incluso aun cuando yo lo dudaba.

A mis amigos, con los cuales viví muchas experiencias sensacionales e inolvidables, que compartimos momentos buenos y malos, anécdotas que hicieron que se conviertan en parte de mi vida.

Agradezco a todos y cada uno de los ingenieros/ingenieras que tuve durante esta etapa, los cuales antes de formarme académicamente, me formaron como persona, como ser humano compartiéndome valores que llevare siempre conmigo y que estoy seguro me servirán en cada paso que dé.

A mi tutora de tesis, por su paciencia, su comprensión, sus enseñanzas. Por brindar el tiempo para concluir este trabajo satisfactoriamente, por ser siempre atenta ante todas nuestras inquietudes ayudándonos a resolver cada problema y desafíos que se nos presentó

Para finalizar, quiero agradecer a mi compañero de tesis, por haber sido constante y dedicado en este trabajo.

-Yanacallo Monta Angelo Josue

Quiero expresar mi sincero agradecimiento a las personas que me acompañaron en este proceso y en la elaboración de esta tesis.

En primer lugar, agradezco a mi familia, por su apoyo incondicional, comprensión y motivación constante, los cuales fueron fundamentales para culminar esta etapa de mi vida.

A mis amigos, Elian Llorente, Juan Zapata, Daniel Fonseca y Carlos Romero, por su apoyo, compañía y palabras de ánimo durante este proceso académico. De manera especial, agradezco a Kevin Salazar, por su apoyo constante, por su disposición para ayudarme en momentos complejos y por compartir sus conocimientos tanto académicos como personales, los cuales fueron fundamentales durante este proceso.

A mi tutora de tesis, Ing. Gualotuña Álvarez Tatiana Marisol, por brindarme su guía, paciencia y conocimientos, los cuales fueron clave para el desarrollo y culminación de este trabajo.

A todos los ingenieros e ingenieras que formaron parte de mi proceso de formación académica, por las enseñanzas impartidas y por contribuir a mi crecimiento profesional.

Finalmente, agradezco a mi compañero de tesis, por el trabajo en equipo, el compromiso y la colaboración durante el desarrollo de este proyecto.

-Páez Zapata Diego Alexander

Índice de contenidos

Dedicatoria.....	6
Agradecimiento.....	8
Resumen	18
Capítulo I	20
Antecedentes	20
Planteamiento del problema.....	21
Descripción del problema	21
Diagnóstico del problema	21
Análisis de causas.....	22
Formulación del problema	24
Justificación.....	24
Objetivos	25
Alcance	25
Capítulo II	27
Marco Teórico	27
Inteligencia Artificial.....	28
Aprendizaje Automático.....	28
Aprendizaje Profundo	29
Procesamiento Digital de Señales de Audio	30
Representaciones espectrales de audio	31
Redes Neuronales Convolucionales (CNN).....	33
Redes Neuronales Recurrentes (RNN/LSTM)	34
Clonación de voz y deepfake de audio	35

Detección de clonación de voz	36
Aplicaciones móviles con inferencia de IA	37
Arquitectura n-capas	39
Diagrama de componentes.....	39
Revisión sistemática de literatura.....	39
Pregunta de investigación (RQ).....	40
Planteamiento del objetivo de búsqueda	40
Criterios de inclusión y exclusión.....	40
Conformación de grupo de control (GC).....	41
Construcción de cadena de búsqueda.....	43
Selección de los estudios primarios.....	44
Síntesis de los Estudios Primarios.....	45
Métricas seleccionadas para responder las preguntas de investigación	48
Síntesis de resultados de la revisión	49
Capítulo III.....	52
Proceso metodológico general de los modelos	53
Recolección de dataset.....	54
Datasets utilizados.....	54
HISPASpoof	54
Latin America Spanish Anti-Spoofing Dataset (FoR)	55
Dataset HABLA	57
Construcción del dataset maestro	58
Análisis	60
Preprocesamiento y extracción de características	62
Preparación del entorno y carga de librerías.....	63
Normalización y segmentación de audios	64

Extracción de características acústicas.....	66
Espectrogramas log-Mel.....	66
Coeficientes MFCC (Mel Frequency Cepstral Coefficients)	66
Aumento de datos.....	66
Diseño e implementación de los modelos híbridos CNN–LSTM.....	67
Enfoque arquitectónico general	67
Modelo 1: Detección de voces reales o falsas	68
Modelo 2: Clasificación de género.....	69
Implementación en TensorFlow/Keras.....	71
Entrenamiento y validación de los modelos	72
Configuración del entorno.....	72
Estrategia de división de datos	73
Configuración de entrenamiento.....	73
Entrenamiento del modelo Real/Fake.....	74
Entrenamiento del modelo de género.....	75
Validación del modelo	76
Evaluación de resultados	76
Métricas utilizadas.....	77
Resultados del modelo Real/Fake	78
Resultados del modelo de clasificación de género	79
Validación en formato TensorFlow Lite.....	79
Capítulo IV	81
Requerimientos del Sistema	81
Casos de uso (UML).....	82
Requerimientos no funcionales	88
Diseño del Sistema	89

Arquitectura de la aplicación móvil	90
Arquitectura del sistema general	91
Diagrama de componentes.....	93
Diagrama de clases (UML)	94
Diagrama entidad-relación (ER)	96
Diagrama de secuencia	98
Diseño de la interfaz de usuario	99
Implementación y Despliegue	101
Dependencias y librerías utilizadas	102
Integración del modelo de clasificación de género	104
Integración del modelo de detección real/fake.....	105
Gestión offline/online con Firebase.....	106
Ajustes de precisión y optimización de los modelos	107
Diagrama de despliegue.....	109
Pruebas y Validación	110
Estrategia de pruebas	110
Pruebas funcionales	111
Pruebas de integración.....	117
Evaluación de rendimiento	119
Usabilidad y experiencia de usuario	121
Análisis de resultados.....	123
Capítulo V	124
Conclusiones	124
Recomendaciones	125
Trabajos Futuros.....	126
Referencias.....	127

Índice de tablas

Tabla 1 Comparación de algoritmos clásicos de aprendizaje automático aplicados al análisis de voz	29
Tabla 2 Comparación entre MFCC y espectrograma en el análisis de voz	33
Tabla 3 Estudios del grupo de control (GC).....	42
Tabla 4 Elaboración de la cadena de búsqueda óptima	43
Tabla 5 Estudios Primarios	44
Tabla 6 Métricas seleccionadas y relación con las preguntas de investigación	48
Tabla 7 Métricas de evaluación y estudios primarios donde se reportan.....	50
Tabla 8 Distribución general del dataset HISPASpoof.....	54
Tabla 9 Distribución general del dataset FoR.....	55
Tabla 10 Distribución por tipo de falsificación en el dataset FoR.....	56
Tabla 11 Distribución de voces reales por país (FoR)	57
Tabla 12 Distribución general del dataset HABLA	57
Tabla 13 Columnas del CSV	59
Tabla 14 Librerías empleadas	63
Tabla 15 Argumentos de sobreajuste.....	67
Tabla 16 Estructura del modelo.....	68
Tabla 17 Parámetros de entrenamiento	69
Tabla 18 Estructura del modelo de clasificación de género.....	70
Tabla 19 Parámetros de entrenamiento en el modelo de género	71
Tabla 20 Archivos generados en el entrenamiento.....	71
Tabla 21 Resultados del entrenamiento del modelo Real/Fake.....	74
Tabla 22 Resultados del entrenamiento del modelo Real/Fake.....	75
Tabla 23 Métricas obtenidas en el modelo Real/fake	78

Tabla 24	Resultados finales del modelo de clasificación de género	79
Tabla 25	Caso de uso CU1 – Grabar audio.....	82
Tabla 26	Caso de uso CU2 – Clasificar género de la voz.....	83
Tabla 27	Caso de uso CU3 – Detectar voz real o clonada	84
Tabla 28	Caso de uso CU4 – Almacenar resultados offline.....	84
Tabla 29	Caso de uso CU5 – Sincronizar resultados con Firebase.....	85
Tabla 30	Caso de uso CU6 – Consultar historial de resultados.....	85
Tabla 31	Caso de uso CU7 - Visualizar resultados	86
Tabla 32	Requerimientos No Funcionales.....	88
Tabla 33	Condiciones de ejecución de las pruebas funcionales.....	113

Índice de figuras

Figura 1 Diagrama de Ishikawa sobre factores que inciden en la clonación de voz	23
Figura 2 Relación conceptual entre inteligencia artificial y ciberseguridad en la detección de clonación de voz mediante modelos CNN–LSTM	27
Figura 3 Diferencia entre aprendizaje automático tradicional y aprendizaje profundo ...	30
Figura 4 Espectrograma de la señal de voz Fake1.wav	32
Figura 5 Esquema de una red neuronal convolucional aplicada al análisis de espectrogramas de voz.	34
Figura 6 Diagrama general del proceso metodológico para el diseño y entrenamiento de los modelos híbridos CNN–LSTM con transformadores.....	53
Figura 7 Cantidad de datos de cada dataset.....	58
Figura 8 Flujo de consolidación del dataset maestro para entrenamiento de modelos..	60
Figura 9 Distribución global de audios reales (bonafide) y falsificados (spoof).....	61
Figura 10 Proceso de preprocesamiento acústico aplicado a los audios de los datasets HISPASpoof, FoR y HABLA	65
Figura 11 Diagrama de casos de uso de la aplicación AntiFakeVoice	87
Figura 12 Arquitectura general del sistema AntiFakeVoice.	91
Figura 13 Arquitectura general del sistema AntiFakeVoice: end–to–end	92
Figura 14 Diagrama de componentes del sistema AntiFakeVoice	93
Figura 15 Diagrama de clases del sistema AntiFakeVoice.....	95
Figura 16 Modelo entidad-relación del sistema AntiFakeVoice.	96
Figura 17 Diagrama de secuencia del flujo de análisis de voz en la aplicación AntiFakeVoice	98
Figura 18 Pantalla de bienvenida de AntiFakeVoice	99
Figura 19 Pantalla principal de análisis de audio	100
Figura 20 Pantalla de historial de resultados	101

Figura 21	Diagrama de despliegue de la aplicación AntiFakeVoice	109
Figura 22	Análisis global del sistema	114
Figura 23	Distribución de clasificaciones	115
Figura 24	Tendencia de probabilidades promedio	117
Figura 25	Flujo de integración de los módulos del sistema AntiFakeVoice	118
Figura 26	Monitoreo de CPU y memoria durante el análisis de audio con duración mayor a 10 segundos mediante View Live Telemetry.	120
Figura 27	Indica el monitoreo del consumo de la CPU y memoria durante el análisis de audios con menos de 10 segundos, realizado a través de la herramienta View Live Telemetry.	121

Resumen

Este trabajo de titulación analiza y explora el avance de tecnologías que tienen la capacidad de clonar la voz humana, lo cual incrementa la posibilidad de suplantación de identidad y pone en riesgo la seguridad de los sistemas que utilizan la autenticación de voz. Frente a este problema, se elaboró un prototipo de aplicación móvil para Android que sea capaz de identificar casos de clonación de voz, mediante el análisis comparativo de audios reales y audios generados con técnicas de deepfake. El propósito es contribuir al ámbito de la ciberseguridad, al permitir una detección temprana de audios con voces clonadas. El prototipo se desarrolló con una arquitectura de n capas e integró un modelo de inteligencia artificial. El modelo se basa en la arquitectura híbrida CNN-LSTM analizar y clasificar las señales de audio. El procesamiento de la señal de voz se realizó a partir de representaciones acústicas provenientes del análisis de sus componentes espectrales y de su variación temporal. De esta manera fue posible observar y separar los patrones que el modelo usa para identificar diferencias entre audios reales de aquellos creados a través de técnicas de inteligencia artificial. El aplicativo se puso a prueba en un entorno controlado, permitiendo corroborar el correcto funcionamiento en dispositivos móviles sin que presenten obstáculos en la ejecución. Los resultados muestran que la integración de los modelos CNN y LSTM en aplicaciones móviles es óptima para la detección de clonación de voz. En este sentido, el prototipo resulta ser una herramienta tecnológica que puede contribuir a minimizar riesgos que estén asociados con el fraude de identidad a través del uso de voces artificiales.

Palabras clave: detección de clonación de voz, aprendizaje profundo, audio deepfake, redes neuronales CNN-LSTM, procesamiento de voz, aplicación móvil android, inteligencia artificial, ciberseguridad.

Abstract

This thesis analyzes and explores the advancement of technologies capable of cloning the human voice, which increases the risk of identity impersonation and compromises the security of systems that rely on voice authentication. In response to this issue, a prototype Android mobile application was developed to identify cases of voice cloning through the comparative analysis of real audio samples and audio generated using deepfake techniques. The purpose of this work is to contribute to the field of cybersecurity by enabling the early detection of audio recordings containing cloned voices. The prototype was developed using an n -tier architecture and integrates an artificial intelligence model. This model is based on a hybrid CNN–LSTM architecture to analyze and classify audio signals. Voice signal processing was carried out using acoustic representations derived from the analysis of spectral components and their temporal variations. This approach made it possible to observe and distinguish the patterns used by the model to identify differences between real audio recordings and those created through artificial intelligence techniques. The application was tested in a controlled environment, confirming its correct operation on mobile devices without execution issues. The results show that the integration of CNN and LSTM models in mobile applications is effective for voice cloning detection. In this regard, the prototype proves to be a technological tool that can help minimize risks associated with identity fraud through the use of artificial voices.

Keywords: voice cloning detection, deep learning, audio deepfake, CNN-LSTM neural networks, speech processing, android mobile application, artificial intelligence, cybersecurity.

Capítulo I

Antecedentes

En los últimos años, con los nuevos modelos de aprendizaje profundo, la inteligencia artificial ha mejorado mucho. Ahora puede manejar muchos datos y encontrar patrones complicados. Gracias a esto, existen aplicaciones que reconocen la voz y caras, traducen idiomas y hasta crean contenido multimedia. Por ejemplo, ahora se puede crear o cambiar audio para que suene muy real, a esto se le llama deepfake de voz o clonación vocal (Todisco et al., 2019).

La clonación de voz se entiende en cómo hacer que una máquina hable como una persona, copiando su tono, cómo sube y baja la voz y hasta su ritmo. Gracias a la tecnología moderna, las redes neuronales son capaces de recrear la voz humana de alguien a partir de grabaciones muy cortas. Al principio, esto se creó para beneficio de la sociedad, como los asistentes virtuales, pero como ahora es tan fácil de usar, también ha traído problemas de seguridad (Asuai et al., 2025).

Las investigaciones señalan que los sistemas que usan la voz para identificarse, por ejemplo, en los bancos o atención al cliente, no fueron diseñados para defenderse de voces falsas. Esto ha hecho que no se pueda confiar de la voz para la identificación, y por eso se está buscando cómo distinguir entre una voz de real y una hecha, mediante sus características de sonido y tiempo (Sagara & Arukonda, 2025).

En consecuencia, resulta necesario encontrar soluciones que integren detectores de voces clonadas en entornos de fácil acceso, como las aplicaciones de Android. Así se reducirían los riesgos de que alguien se haga pasar por otro y se fortalecería la confianza en la comunicación por internet (Asuai et al., 2025; Todisco et al., 2019).

Planteamiento del problema

Descripción del problema

Cada vez se usan más las tecnologías para clonar voces, lo que permite crear audios falsos para engañar a cualquiera, sean personas o sistemas. Hoy en día, los programas de inteligencia artificial pueden hacer voces sintéticas muy buenas con poco trabajo. Esto ha hecho que gente con malas intenciones las utilice, aumentando las estafas telefónicas y el robo de identidad, como señalan Asuai y otros en 2025.

Uno de los mayores problemas con todo esto es que no se puede saber si una voz es de verdad o si la hizo una máquina. Si solo se escucha, es muy difícil distinguir las voces hechas con técnicas avanzadas de aprendizaje automático. Esta dificultad pone en peligro, sobre todo cuando la voz sirve para confirmar la identidad o dar una autorización, como en bancos, atención al cliente o llamadas personales, según Sagara y Arukonda en 2025.

Además, no hay herramientas accesibles para determinar si una voz fue clonada, lo cual agrava la situación. La mayoría de las soluciones que existen son para expertos y no están diseñadas para el uso cotidiano. Debido a la falta de herramientas de verificación, se está expuesto al robo de identidad sin posibilidad de confirmar la autenticidad de los audios, como indican Todisco y sus colegas (2019).

Diagnóstico del problema

La clonación de voz es un problema cada vez más significativo, que integra aspectos tecnológicos, personales y de procesos. Sagara y Arukonda (2025) indican que, si las personas no son conscientes de la existencia de audios falsificados, resulta más probable que sean víctimas de engaños. Por su parte, Todisco y otros (2019) señalan que las herramientas para detectar estos audios aún no están orientadas al uso cotidiano. De esta manera se muestra que el problema no solo se da por la dificultad de las técnicas, sino también en lo poco accesibles que están las herramientas de protección para la población.

Según Asuai y su equipo (2025), los sistemas híbridos CNN y LSTM crear voces muy realistas las cuales son complicadas de diferenciar entre una voz real y una artificial. Esto se vuelve preocupante cuando se utiliza de forma incorrecta, ya sea con propósitos engañosos o fraudulentos.

La situación se dificulta debido a que numerosas empresas y organizaciones continúan confiando en la voz como mecanismo de verificación de identidad, sin contar con métodos de seguridad adicionales. De esta manera se muestra que el problema no solo se da por la dificultad de las técnicas, sino también en lo poco accesibles que están las herramientas de protección para la población.

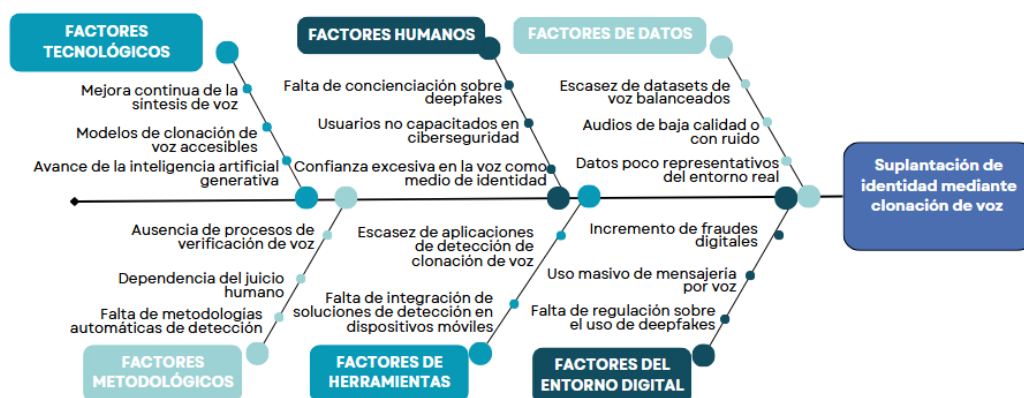
Entonces, más allá de lo que dicen los estudios, la clonación de voz no es solo un reto de tecnología. Es un asunto social y de organización también, porque la gente no sabe mucho de seguridad informática y las instituciones no han puesto más formas de verificar. Esta mezcla lo hace todo más fácil para que la clonación de voz se use para estafar y robar identidades.

Análisis de causas

La suplantación de identidad mediante clonación de voz ocurre por diversas razones, que pueden agruparse en diferentes áreas: lo tecnológico, lo humano, los métodos empleados, las herramientas, los datos y el funcionamiento actual de internet. En la Figura 1 se muestra cómo estos factores incrementan el riesgo de que se produzca suplantación de identidad mediante la voz.

Figura 1

Diagrama de Ishikawa sobre factores que inciden en la clonación de voz



Fuente: Elaboración propia

Los avances en inteligencia artificial generativa y la facilidad para usar modelos de clonación de voz son factores clave. Asuai y otros (2025) dicen que las combinaciones de redes neuronales (CNN-LSTM) logran voces tan reales que es difícil saber si son de verdad o falsas, lo que hace más fácil que se usen indebidamente.

La gente no sabe mucho sobre los riesgos de los 'deepfakes' y no hay mucha capacitación en ciberseguridad, lo que deja a los usuarios más expuestos. Esto es peor cuando la voz todavía se considera una forma segura de verificar la identidad, sin otros métodos que la acompañen. Sagara y Arukonda (2025) alertan que confiar demasiado en la voz para identificarse es uno de los mayores peligros.

Sobre las herramientas para detectar voces clonadas, Todisco y sus colaboradores (2019) comentan que estas soluciones aún no se utilizan en dispositivos móviles de manera cotidiana. Ya que si una persona intenta hacerse pasar por otra representaría un problema

A parte, debido a que la cantidad de datos no se puede obtener detectores de calidad. Gran cantidad de audios no tiene un control de ruido o no están equilibrados y esto afecta negativamente a la precisión del sistema. Todo esto es causado debido a que en el mundo digital no existe un control ni reglas lo que crea un ambiente fácil para las estafas. En

conclusión, la clonación de voz no es solo un tema técnico, sino también social, de métodos y de leyes.

Formulación del problema

La clonación de voz mediante deepfakes es un problema que genera una preocupación creciente en la seguridad digital y en los mecanismos de verificación de identidad, específicamente en la autenticación basada en la voz. Se identifica que esta problemática se sustenta en tres factores principales: en primer lugar, las tecnologías actuales permiten generar voces sintéticas con un alto grado de realismo; en segundo lugar, la población no tiene un conocimiento adecuado sobre los riesgos asociados a estas voces falsificadas; y, en tercer lugar, existe una limitada disponibilidad de herramientas accesibles o automatizadas integradas en aplicaciones de uso cotidiano para su detección. Además, el incremento de fraudes en entornos digitales y la ausencia de regulaciones claras sobre los deepfakes facilitan la suplantación de identidad.

En este contexto, el problema central se plantea de la siguiente manera:

¿Cómo desarrollar un mecanismo automático, efectivo y accesible para la detección de clonación de voz mediante deepfakes, implementado en un prototipo de aplicación móvil para Android, que contribuya a la prevención de la suplantación de identidad en entornos digitales?

Justificación

El aumento en la clonación de voz con deepfakes ya está afectando la seguridad digital y la confianza en cómo se verifica la identidad. Aunque hay avances en inteligencia artificial para crear y detectar estas voces falsas, la mayoría de estas soluciones se quedan en universidades o en círculos de especialistas. Esto hace difícil que se usen en el día a día.

Por eso, tiene sentido crear una aplicación móvil para Android. Siendo un modo fácil y preventivo para combatir el robo de identidad. Se eligió una plataforma móvil lo que permite llevar la detección automática de voces clonadas directamente al usuario. La idea es usar lo

último en redes neuronales para que la tecnología que se investiga llegue pronto a los usuarios y se implemente de manera efectiva en el entorno digital.

Con esta perspectiva, se formulan los siguientes objetivos generales y específicos:

Objetivos

Objetivo general

Desarrollar un prototipo de aplicación móvil para Android que utilice redes neuronales profundas CNN-LSTM con el fin de detectar clonaciones de voz mediante *deepfakes*, contribuyendo a la prevención de la suplantación de identidad y fortaleciendo la ciberseguridad en entornos digitales.

Objetivos específicos

- Analizar el estado actual de la clonación de voz y los métodos de detección de audio deepfakes reportados en la literatura científica.
- Diseñar la arquitectura del prototipo móvil integrando modelos de redes neuronales profundas CNN-LSTM para la detección de audios falsos.
- Implementar el prototipo en la aplicación móvil, asegurando su funcionalidad en dispositivos de uso cotidiano.
- Evaluar el desempeño del modelo de detección mediante pruebas con audios reales y sintéticos, considerando métricas de precisión, sensibilidad y especificidad.

Alcance

Este trabajo se enfoca en hacer una aplicación para celulares Android. El propósito es que la aplicación sepa diferenciar si una voz grabada es falsa (deepfake) o real. Para ello, se emplea redes neuronales, las cuales actúan como cerebros artificiales, principalmente CNN y LSTM. En toda esta construcción se usó TensorFlow y se complementó con TensorFlow Lite para que funcionen correctamente en cualquier dispositivo móvil.

La aplicación, permite al usuario grabar su voz a través del micrófono del dispositivo móvil o a su vez cargar un audio disponible.

Posteriormente, el sistema procesa dicho audio, realizando un preprocesamiento básico (normalización, segmentación y extracción de características acústicas relevantes) y, finalmente, determina si la voz es auténtica o si fue generada por un sistema automatizado.

No obstante, esta aplicación presenta ciertas limitaciones. Su funcionamiento se restringe a audios grabados directamente en el dispositivo o previamente almacenados en el mismo. Su diseño no está hecho para detectar voces falsas a partir de varias grabaciones del mismo audio falso, ya que en el ambiente existe ruidos que pueden afectar al desempeño del sistema. Asimismo, no está diseñada para realizar análisis de llamadas en tiempo real ni para integrarse a otros sistemas.

Por otra parte, la aplicación tiene una funcionalidad que permite detectar si la voz pertenece al sexo masculino o femenino.

Toda la información recopilada se almacena en una base de datos denominada Firebase y, en ausencia de conexión a internet, los datos se conservan localmente para su posterior sincronización.

Para que el modelo aprendiera, se emplearon datos públicos como HABLA, HISPASpoof y Latin America Spanish antispoofing. Se tomó en cuenta las limitaciones de los dispositivos móviles, que no tienen tanta memoria ni capacidad de procesamiento. Se evaluó qué tan bien funcionaba el sistema con medidas como la precisión y el F1-score, pero no se profundizó en temas legales o de cómo se regulan estas tecnologías.

En el capítulo siguiente se desarrolla el marco teórico y la revisión sistemática de la literatura.

Capítulo II

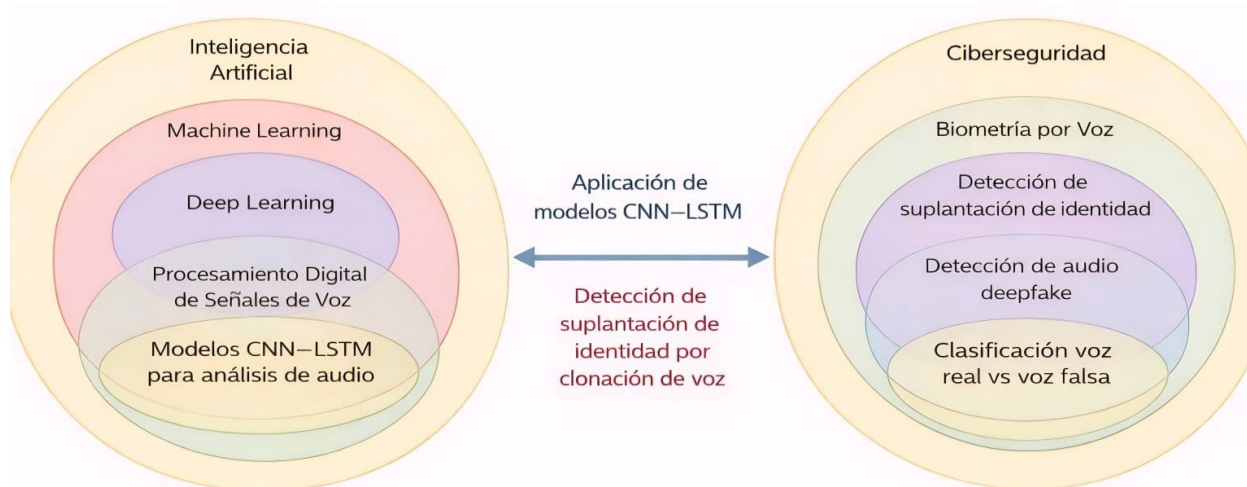
Marco Teórico

Este capítulo expone la teoría que sustenta la investigación. Se presentan los conceptos clave y las metodologías utilizadas para la detección de voces clonadas mediante inteligencia artificial. El análisis se centra en dos aspectos principales: en primer lugar, la inteligencia artificial, que comprende el aprendizaje automático y el aprendizaje profundo aplicados al procesamiento de voz; en segundo lugar, la ciberseguridad, donde la voz constituye una característica biométrica vulnerable a ataques mediante grabaciones falsas o deepfakes.

La Figura 2 muestra el vínculo entre estas dos áreas. Los modelos CNN y LSTM funcionan como conexión entre la inteligencia artificial y la ciberseguridad, permitiendo identificar si un audio es real o falso. Este vínculo describe la importancia de las arquitecturas profundas para determinar intentos de fraude de identidad a través de la voz. Lo mencionado forma parte de la base de la propuesta expuesta en este estudio.

Figura 2

Relación conceptual entre inteligencia artificial y ciberseguridad en la detección de clonación de voz mediante modelos CNN–LSTM



Fuente: Elaboración propia

Inteligencia Artificial

La inteligencia artificial (IA) es un área de la computación que busca crear sistemas que puedan hacer cosas que normalmente solo la gente puede, como pensar, aprender, entender y decidir (Russell & Norvig, 2021). Hoy en día, la IA es clave para manejar una gran cantidad de datos y hacer automáticamente tareas complicadas en muchos campos, como la seguridad informática o el análisis de datos biométricos.

En el mundo del audio, la IA ayuda a entender los patrones de la voz humana. Así, se puede crear voces artificiales o identificar si una voz ha sido alterada. Esto ha hecho que los sistemas de seguridad por voz sean más fuertes, pudiendo detectar cuando alguien intenta hacerse pasar por otro usando voces clonadas o deepfakes.

Para este trabajo, la IA es la idea principal para crear modelos que detecten voces clonadas. Se usa redes neuronales que combinan redes convolucionales y recurrentes, las cuales analizan cómo cambia la voz a lo largo del tiempo. Al tener estas técnicas en una aplicación para celulares, se ofrece un sistema de uso fácil para detectar audios falsos de forma automática, ayudando a reducir los riesgos de que alguien se haga pasar por otra persona.

Aprendizaje Automático

El aprendizaje automático, conocido como Machine Learning (ML), es una rama de la inteligencia artificial. Su objetivo es crear programas que puedan aprender de los datos para encontrar patrones, sin necesidad de explicar paso a paso cómo hacerlo (Mitchell, 1997). Estos programas crean modelos matemáticos para hacer predicciones o clasificaciones basándose en lo que ya han visto antes. Son clave para entender información complicada.

Al tratar el tema del audio, los algoritmos de ML se utilizan ampliamente. Sirven, por ejemplo, para reconocer quién está hablando, saber qué emoción hay en una voz o incluso para detectar fraudes de voz. Modelos más viejos, como las máquinas de soporte vectorial (SVM), la regresión logística o los clasificadores bayesianos, funcionan bien cuando los sonidos

tienen características claras. Un ejemplo de esto son los coeficientes cepstrales en frecuencia Mel (MFCC) (Davis & Mermelstein, 1980).

La Tabla 1 muestra algoritmos que son buenos cuando los sonidos tienen características claras. Sin embargo, tienen límites, por eso se necesitan métodos más modernos como el aprendizaje profundo.

Tabla 1

Comparación de algoritmos clásicos de aprendizaje automático aplicados al análisis de voz

Algoritmo	Ventajas principales	Limitaciones	Aplicaciones en audio
SVM	Alta precisión en espacios de alta dimensión	Sensible al ruido y al tamaño de datos	Detección de hablantes
Regresión logística	Simplicidad y rapidez	No captura relaciones no lineales	Clasificación binaria de voz
Clasificadores bayesianos	Robustez con datos pequeños	Supone independencia entre variables	Identificación de emociones

Fuente: Elaboración propia

Aprendizaje Profundo

El Deep Learning constituye una extensión avanzada del Machine Learning que emplea redes neuronales con múltiples capas ocultas para modelar relaciones complejas en los datos, las cuales no siguen un comportamiento lineal. A diferencia de los métodos tradicionales, el Deep Learning es capaz de extraer de manera autónoma las características relevantes de los datos de entrada, sin requerir la realización manual de dicho proceso.

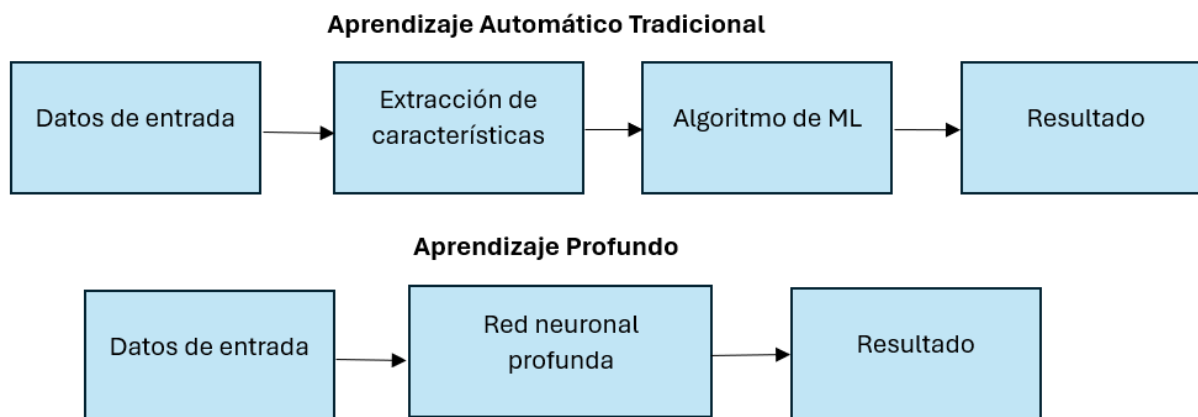
En audio, los modelos de Deep Learning han resultado ser mucho mejores que las técnicas antiguas, sobre todo para clasificar y verificar si una voz es un deepfake. Como pueden captar patrones complicados de tiempo y frecuencia, son perfectos para encontrar diferencias mínimas entre voces de personas reales y voces creadas artificiales.

En este trabajo, el uso de técnicas de aprendizaje profundo como lo es el Deep Learning es de suma importancia en el contexto de análisis de las señales de voz. De modo que, se aplican arquitecturas las cuales integran redes convoluciones y recurrentes, lo que permite capturar desde características espectrales hasta temporales del audio. Esta fusión resulta óptima para que la capacidad del sistema pueda mejorar al momento de reconocer audios generados a través de clonación de voz.

La Figura 3 indica que, las principales diferencias entre el aprendizaje profundo y los enfoques tradicionales de aprendizaje automático es la capacidad para recopilar características automáticamente. esto reduce el proceso manual complejo y contribuye a que el modelo se adapte mejor.

Figura 3

Diferencia entre aprendizaje automático tradicional y aprendizaje profundo



Fuente: Elaboración propia

Procesamiento Digital de Señales de Audio

El procesamiento digital de señales de audio (DSP) es una forma de estudiar, cambiar y manejar sonidos usando computadoras. Si se integra esto con la inteligencia artificial, el DSP contribuye a preparar las señales de voz para que programas que aprenden, como el aprendizaje automático o profundo, puedan entenderlas.

Las señales de voz son continuas, por lo que primero hay que pasarlas a digital y ajustarlas para que sea más fácil trabajar con ellas. Esto significa quitar ruidos, dividir el sonido en partes, ajustar el volumen y convertirlo en algo que muestre cómo suena. Así se puede sacar información importante de la voz, como la cantidad de energía presente en diferentes sonidos, cómo sube y baja el tono, y cómo se pronuncian las cosas.

Cuando se quiere saber si una voz es falsa, el DSP es de gran ayuda para ver pequeñas diferencias entre sonidos reales y hechos por una máquina. De esta manera, se toman atributos que los programas lo usan para clasificar.

De esta manera, atributos se toman; posteriormente, los programas lo usan para clasificar. Un procesamiento bueno implica que los sistemas de detección de voces falsas rinden mejor, ya que las redes neuronales reciben información más precisa.

En este estudio, el procesamiento digital de señales se usa antes, preparando los espectrogramas y coeficientes cepstrales de Mel MFCC. Estos sirven como entradas para los modelos CNN–LSTM, quienes clasifican los audios.

Representaciones espectrales de audio

Las representaciones espectrales del audio son fundamentales para comprender las señales de voz. Mediante estas, la información sonora se organiza de forma que facilite la identificación de las características más relevantes. Las técnicas más empleadas son los MFCC y los espectrogramas.

Los MFCC se usan mucho para saber quién habla o para reconocer lo que se dice. Esto es porque copian cómo oyen los humanos, usando una escala de frecuencias logarítmica (Davis & Mermelstein, 1980). Así, los MFCC recogen los sonidos clave de la voz, dando una descripción clara de cómo cambia el sonido en el tiempo.

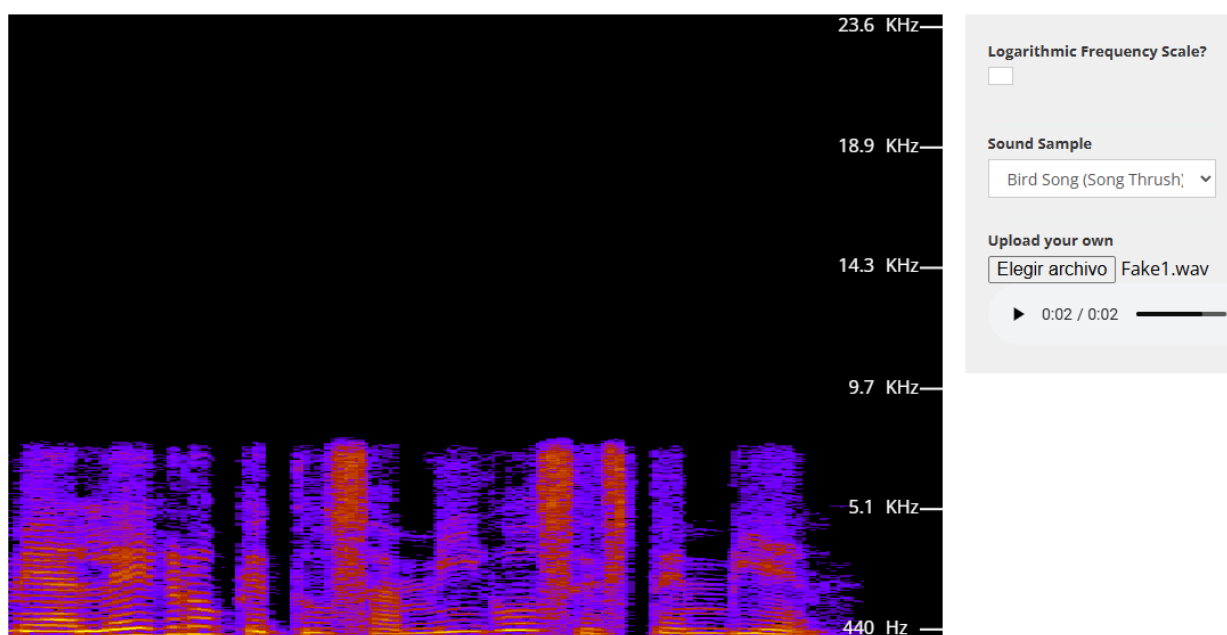
No obstante, los espectrogramas representan cómo la energía del sonido evoluciona en el tiempo y en la frecuencia. Esto facilita la identificación de diferencias entre voces auténticas y voces falsificadas. Al ser representaciones visuales del audio, esta información es adecuada

para el análisis a través de redes convolucionales, de manera que se pueden reconocer patrones variables relacionados con sonidos creados de forma artificial.

La Figura 4 muestra el espectrograma del audio FAKE1.wav, donde es posible visualizar la estructura espectro-temporal de una voz clonada, la cual indica las variaciones irregulares en la energía y en el comportamiento frecuencial a lo largo del tiempo, permitiendo diferenciar este tipo de señales de aquellas que provienen de voces reales.

Figura 4

Espectrograma de la señal de voz Fake1.wav



Nota. La figura fue generada mediante la herramienta en línea Academo Spectrogram Generator.

Asimismo, resulta útil comparar las dos representaciones más empleadas en el procesamiento de voz. La Tabla 2 resume las principales diferencias entre los coeficientes cepstrales en frecuencia Mel (MFCC) y los espectrogramas, destacando sus ventajas y limitaciones en el análisis de señales acústicas.

Tabla 2*Comparación entre MFCC y espectrograma en el análisis de voz*

Representación	Ventajas principales	Limitaciones	Aplicaciones
MFCC	Compacta, eficiente, modela percepción humana	Pierde información temporal detallada	Reconocimiento de hablantes, verificación biométrica
Espectrograma	Visualiza energía tiempo-frecuencia, útil para CNN	Mayor costo computacional, redundancia	Detección de clonación de voz, análisis acústico

Fuente: Elaboración propia**Redes Neuronales Convolucionales (CNN)**

Las redes neuronales convolucionales, o CNN, son un tipo de inteligencia artificial hecho para trabajar con datos que tienen un orden, como fotos o los espectrogramas de audio. Funcionan aplicando filtros que identifican únicamente la información importante de esos datos, sin que sea necesario programar explícitamente qué buscar.

Para el audio, las CNN son muy buenas con los espectrogramas, que son como fotos del sonido que muestran cómo cambian la energía y la frecuencia con el tiempo. Varios estudios han visto que estas redes pueden detectar patrones atípicos en audios falsos (deepfakes) que no puede ser detectado por el oído humano, pero que las CNN aprenden a identificar en los patrones del espectrograma.

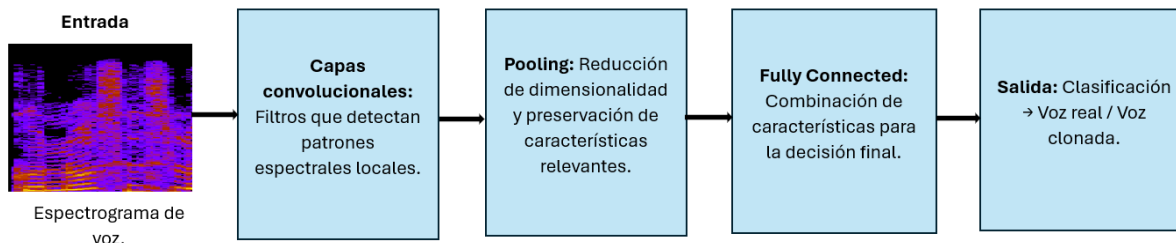
En este trabajo, se utilizaron CNN para encontrar esas diferencias en los patrones de voz entre audios de verdad y voces clonadas. La idea es aprovechar que pueden obtener las características por sí solas para mejorar la forma en que se detecta si una voz es clonada, usando la información del espectrograma para diferenciar bien entre lo real y lo falso.

Para este trabajo es útil mostrar gráficamente el avance de la información dentro de una red neuronal convolucional cuando analiza un espectrograma de voz. En la Figura 5 se

visualiza el flujo donde el espectrograma primero entra en las capas convolucionales y a las de agrupamiento, las cuales extraen los patrones más importantes. Después, esa información para a una capa estrictamente conectada que se encarga de decidir si la voz es real o clonada.

Figura 5

Esquema de una red neuronal convolucional aplicada al análisis de espectrogramas de voz.



Fuente: Elaboración propia

Redes Neuronales Recurrentes (RNN/LSTM)

Las redes neuronales recurrentes (RNN) son un tipo de aprendizaje automático que funcionan bien con datos en secuencia, o sea, que tienen un orden. Esto es porque guardan un recuerdo de lo que pasó antes. A diferencia de otras redes, como las CNN, que se enfocan en detalles de imágenes, las RNN son mejores con los sonidos.

Dentro de las RNN, las Long Short-Term Memory (LSTM) son muy conocidas. Las crearon Hochreiter y Schmidhuber. Lo bueno de las LSTM es que tienen como unas compuertas que deciden qué información guardar y cuál olvidar. Esto les permite manejar secuencias largas sin perder datos importantes, lo que las hace muy útiles para hallar patrones en grabaciones de voz hechas por máquinas.

Para detectar la clonación de voz, las RNN y LSTM ayudan a entender cómo cambian las señales con el tiempo, sumándose al análisis espectral que hacen las CNN. Así, se tiene una idea más completa, que toma en cuenta tanto las características de la voz como su desarrollo en el tiempo. Esto hace que el sistema sea mejor para distinguir voces reales de las clonadas.

Clonación de voz y deepfake de audio

La clonación de voz es una técnica que permite crear un audio que suena como la voz de alguien más, y se logra usando programas que aprenden de grabaciones de voz reales.

Actualmente, incluso con pocos datos, se pueden obtener resultados muy buenos. Por ejemplo, en 2022, Dai y su equipo crearon Hieratron. Este sistema puede separar cómo se modela el timbre de la voz y su forma de hablar, lo que ayuda a controlar el estilo y la entonación de la voz clonada. Con solo unos minutos de grabación de la persona, pueden hacer una síntesis de voz estable. Esto muestra que cada vez hay sistemas de clonación de voz más fáciles de usar y que funcionan bien, incluso con grabaciones de baja calidad.

Cuando se habla de deepfake se hace énfasis a grabaciones creadas o modificadas para hacerse pasar por la voz de otra persona. Técnicas como Tacotron o WaveNet, entre otras que menciona Seong y su grupo en 2021, han logrado que estos audios suenen tan naturales que es difícil diferenciarlos de voces reales. Se sabe que estos audios sintéticos pueden usarse con fines positivos, como ayudar a personas con discapacidad, guardar voces de personas o asistentes virtuales más personalizados. Pero también son un riesgo importante para la seguridad. Hay casos documentados de fraudes, estafas y ataques a sistemas de verificación de identidad, como señalan Shang en 2018 y Prajapati en 2021.

Los estudios analizados muestran que las voces clonadas tienen pequeños fallos de sonido que no se pueden oír, pero que sí se pueden ver en representaciones como los MFCC y los espectrogramas. Estas características han sido usadas por sistemas de detección con CNN y RNN/LSTM, que pueden distinguir entre audios reales y sintéticos con mucha precisión, como han demostrado Liang en 2017, Kumar y Alraisi en 2022, y Tanveer Gujjar en 2024.

En este trabajo, la clonación de voz y los deepfakes de audio constituyen el eje principal del estudio realizado. Entender esto no solo ayuda a ver los riesgos que hay, sino que también es la base para crear sistemas de detección que aseguren la confianza en la biométrica, la investigación forense y la seguridad digital.

Detección de clonación de voz

La detección de clonación de voz constituye un desafío central en el ámbito de la seguridad biométrica y el análisis forense. Su enfoque se centra en distinguir las señales genuinas de las señales generadas a partir de técnicas de síntesis o conversión también conocidas como audios deepfakes

La clonación moderna crea voces ultra realistas que hace que rasgos como las vibraciones en los tonos o la energía de este fracasen ante métodos clásicos. Por lo tanto, la investigación tomo un enfoque al uso de representaciones espectrales como: Coeficientes Cepstrales en la Escala de Mel, Coeficientes Cepstrales de Frecuencia Lineal, Coeficientes Cepstrales de Q Constante y espectrogramas.

- **Modelos convolucionales CNN:** Liang y colaboradores en 2017 mostraron que las CNN reconocen voces disfrazadas con gran precisión incluso ante leves cambios tonales. Kumar y Alraisi en 2022 sugirieron usar estructuras como Xception, obteniendo resultados superiores al 90% de detección en tiempo real.
- **Modelos recurrentes RNN/LSTM:** Aunque tienen problemillas de memoria las LSTM se usaron para captar las conexiones en el tiempo de las secuencias de voz Y con esto mejoran la clasificación de audios falsos, sí.
- **Métodos híbridos y de extracción avanzada:** Prajapati et al.(2021) integraron características en base a la energía instantánea (ESA-IACC, ESA-IFCC) para la detección de ataques de ataques de repetición en asistentes de voz. Obteniendo como resultado mejoras importantes en la precisión, mayormente en entornos donde el ruido es fuerte.
- **Enfoques recientes de machine learning:** Tanveer Guijar et al.(2024) expusieron el modelo MFCC GNB XtractNet, el cual integra coeficientes cepstrales con transformaciones probabilísticas, contribuyendo a que el modelo logre una precisión aproximada al 99,9% en el reconocimiento de voces falsas.

Además de los avances técnicos, se han propuesto estrategias de defensa complementarias, como sistemas de liveness detection en dispositivos móviles (Shang et al., 2018), que verifican la presencia de vibraciones reales en las cuerdas vocales para evitar ataques de suplantación mediante altavoces.

En síntesis, la detección de clonación de voz se apoya en tres pilares fundamentales:

1. Representaciones espectrales robustas que capturan artefactos de síntesis.
2. Los modelos de aprendizaje profundo, como lo son las redes convolucionales, las LSTM y la combinación de ambas, se han convertido en herramientas bastante útiles para identificar las diferencias entre voces reales y generadas de manera artificial.
3. Métodos complementarios que refuerzan la seguridad del sistema, como procedimientos de verificación de vivacidad, combinación de diferentes características acústicas y análisis de energía presente en la señal. La introducción de estos mecanismos contribuye a la mejora y confiabilidad de los sistemas biométricos.

En el contexto de este trabajo, los enfoques son el punto de partida para desarrollar un sistema de detección capaz de afrontar riesgos ligados a la clonación de voz, además de garantizar la integridad de las aplicaciones que dependen de la autenticación a través de audio.

Aplicaciones móviles con inferencia de IA

El progreso de la inteligencia artificial ha hecho posible que los modelos complejos de aprendizaje profundo, que antes necesitaban servidores potentes, ahora funcionen en celulares. Esto se logra con métodos que los hacen más pequeños y eficientes. A esto se le denomina inferencia en el borde, cuya idea es acercar el procesamiento a los usuarios, evitando una dependencia excesiva de la nube y mejorando el funcionamiento de las aplicaciones que requieren respuestas rápidas.

En áreas como el procesamiento de voz y la detección de clonación, la inferencia en celulares tiene muchos puntos a favor:

- **Reducción de latencia:** Al ejecutar el modelo directamente en el dispositivo, se evita el retardo asociado al envío de datos a servidores externos.
- **Privacidad y seguridad:** Al procesar dentro del mismo dispositivo disminuye el riesgo de exponer datos sensibles como los audios a entorno de la red
- **Optimización de recursos:** Con el uso de librerías como: TensorFlow Lite, ONNX Runtime y Core ML, los modelos se adaptan a las características de los dispositivos móviles.
- **Escalabilidad y accesibilidad:** Con el avance en la tecnología de teléfonos inteligentes esto da paso a posibilidad de ejecutar modelos, sin la necesidad de infraestructuras complejas.

Diversos estudios han demostrado que arquitecturas como CNN y LSTM, previamente entrenadas en servidores, pueden ser adaptadas para funcionar en móviles con un impacto mínimo en la precisión. Shang y sus colegas del 2018 y Prajapati et al en el 2021 afirman que fusionar mecanismos anti-suplantación y detección de vida en aplicaciones móviles es esencial para la seguridad, una piedra angular diría yo. Esa integración es como un escudo contra el avance de ataques basados en clonación vocal y el auge de los deepfakes, ¡vaya! Que complican la verificación confiable de la identidad en esos entornos móviles.

En este estudio, el poner en marcha modelos de detección en apps móviles es una movida estratégica que busca probar la viabilidad de los sistemas propuestos. La habilidad de hacer inferencias en tiempo real, directo en un dispositivo portátil, ¡guau! expande el uso de la detección de clonación de voz en el día a día. Por ejemplo, se puede añadir a sistemas de autenticación biométrica, a mecanismos de seguridad financiera y medidas de protección de la identidad digital.

Arquitectura n-capas

La literatura especializada coincide en que la arquitectura en capas se usa con frecuencia en el desarrollo de software, ya que favorece el modularidad del sistema y permite reutilizar componentes con mayor facilidad. La separación del sistema en capas independientes, con funciones bien definidas en cada una, mejora la claridad del diseño y simplifica su mantenimiento (Tu, 2023). Revisiones sistemáticas recientes señalan que este patrón es de los más usados para organizar sistemas complejos, porque favorece atributos de calidad como la portabilidad, la reutilización y la comprensión del sistema (Belle et al., 2021).

Diagrama de componentes

Asimismo, Koç, Erdoğan, Barjakly y Peker (2021) señalan que, aunque los diagramas de componentes son menos utilizados en comparación con otros tipos de UML, su aplicación resulta adecuada para mostrar la estructura física y las relaciones técnicas de los sistemas. De este modo, la representación gráfica de AntiFakeVoice se alinea con las prácticas documentadas en la ingeniería de software, aportando claridad y rigor al diseño arquitectónico.

Revisión sistemática de literatura

En esta sección se presentan documentos científicos relacionados con la detección de clonación de voz y audio deepfake, considerando el enfoque de la tesis: entrenamiento de un modelo híbrido CNN–LSTM con audios reales y clonados, evaluación mediante métricas de desempeño (accuracy, precision, recall y F1-score), e integración del modelo en una aplicación Android mediante TensorFlow Lite.

La Revisión Sistemática de Literatura (SLR) ligera se adopta como estrategia metodológica para identificar, organizar y sintetizar estudios relevantes, de manera que la investigación se sustente en evidencia previa y se delimite un marco técnico coherente con el problema de suplantación de identidad por voz. Este enfoque metodológico permite disponer de una estructura clara para la revisión, que incluye fases como el planteamiento del objetivo de

búsqueda, la definición de criterios de inclusión y exclusión, la construcción de la cadena de búsqueda y la selección de estudios primarios.

Pregunta de investigación (RQ)

RQ1: ¿Qué enfoques (características acústicas y arquitecturas de deep learning) han demostrado mayor efectividad para detectar voz sintetizada/clonada (audio deepfake), con potencial de despliegue en entornos prácticos como aplicaciones móviles?

RQ2: ¿Qué métricas de evaluación resultan más adecuadas para medir la efectividad de sistemas de detección de voz clonada en escenarios reales?

Planteamiento del objetivo de búsqueda

Lo que se busca es ver qué funciona mejor en detección de clonación de voz y deepfakes de audio, usando cosas como las representaciones de audio, cómo se procesan los datos y qué tipos de redes neuronales se usan. Todo esto es para ayudar a hacer un prototipo para celular que tendrá un modelo CNN-LSTM ya listo para usarse con TensorFlow Lite.

Esto va de la mano con el perfil del proyecto, el cual establece la necesidad de entrenar y evaluar un modelo CNN-LSTM con audios reales y clonados, y luego integrarlo en Android. Así, este estudio contribuye a encontrar lo más importante a nivel técnico y a definir las mejores formas de hacer las cosas para que el sistema propuesto funcione bien y se pueda usar.

Criterios de inclusión y exclusión

Los criterios garantizan que la revisión considere únicamente estudios que aporten evidencia directa para responder la RQ.

Serán incluidos (CI):

- **CI1:** Artículos que aborden detección de audio deepfake, voz sintetizada (TTS) o voice conversión como amenazas de spoofing.
- **CI2:** Estudios que utilicen ML/DL (p. ej., CNN, RNN/LSTM, modelos sobre espectrogramas o MFCC) para clasificación bonafide vs spoof.

- **CI3:** Trabajos que describan métricas de evaluación o resultados experimentales (EER, accuracy, etc.) para comparar enfoques.
- **CI4:** Estudios en inglés o español, artículos científicos, conferencias o tesis.
- **CI5:** Estudios que contemplen datasets o escenarios realistas (p. ej., replay, síntesis, VC; y/o benchmarks).

Serán excluidos (CE):

- **CE1:** Estudios que no traten audio/voz (por ejemplo, deepfake solo de video/imagen sin componente de audio).
- **CE2:** Estudios sin descripción técnica mínima (sin arquitectura, sin características, sin procedimiento experimental).
- **CE3:** Trabajos puramente opinativos sin validación experimental.
- **CE4:** Estudios fuera del dominio de detección (por ejemplo, solo generación, sin conexión a spoofing/detección), salvo que aporten contexto directo sobre la amenaza.

Conformación de grupo de control (GC)

El Grupo de Control (GC) corresponde a un conjunto inicial de estudios seleccionados que cumplen con los criterios de inclusión definidos en la RQ. Su propósito es doble: (1) validar que la investigación se encuentra en el dominio correcto y (2) extraer términos clave que servirán para la construcción de la cadena de búsqueda. La conformación del GC se fundamenta en referencias troncales sobre *spoofing*, clonación de voz y detección mediante arquitecturas profundas (CNN–LSTM), lo que asegura coherencia con el objetivo del proyecto.

Actividades realizadas:

- Identificación de estudios base (surveys y trabajos técnicos relevantes).
- Extracción de términos frecuentes (spoofing, voice conversion, speech synthesis, CNN, LSTM, ASVspoof, spectrogram).

- Validación de la pertinencia con el objetivo del proyecto (detección de clonación de voz y despliegue en aplicaciones móviles).
- Constitución del grupo control a través de estudios representativos elegidos por su conexión directa con la RQ.

La Tabla 3 reúne los estudios seleccionados para formar el grupo control, como se relacionan directamente con la detección de clonación de voz y audio deepfake. Las palabras clave identificadas en esos trabajos serán añadidas a la búsqueda, para mantener la revisión sistemática acorde a las preguntas de investigación. Los criterios de inclusión y exclusión ya establecidos, también se aplicarán.

Tabla 3

Estudios del grupo de control (GC)

Código	Título	Términos relevantes
GC1	Spoofing and countermeasures for speaker verification: A survey	Spoofing, countermeasures, ASV, voice conversion, speech synthesis
GC2	On the Vulnerability of Speaker Verification to Realistic Voice Spoofing	Replay, speech synthesis, voice conversion, AVspoof, vulnerabilidad ASV
GC3	Detection of AI-synthesized speech using CNN-LSTM networks	AI-synthesized speech, CNN-LSTM, detection, clasificación bonafide/spoof
GC4	Deepfakes Audio Detection Techniques Using Deep Convolutional Neural Network	CNN, Xception, espectrogramas/frecuencias, detección deepfake
GC5	Unmasking the Fake: Machine Learning Approach for Deepfake Voice Detection	Deepfake voice detection, ML, clasificación, resultados comparativos

Fuente: Elaboración propia

Para la investigación se realizó una búsqueda en la base de datos académica digital IEEE Xplore, llevando a cabo una verificación conjunta con el fin de asegurar que los artículos

científicos seleccionados se encuentren directamente relacionados con la pregunta de investigación (RQ) planteada previamente.

Construcción de cadena de búsqueda

La cadena de búsqueda se construyó a partir de los términos relevantes identificados en el Grupo de Control (GC). Algunas de estas palabras son spoofing, deepfake voice, speech synthesis, voice conversion, CNN, LSTM, spectrogram, MFCC y ASVspoof. Estas palabras son clave para delimitar el alcance de la investigación.

Así se pudo buscar artículos directamente relacionados con la detección de voces clonadas en bases de datos académicas, como IEEE Xplore. Se usa operadores como AND y OR para combinar estas palabras y crear secuencias de búsqueda precisas, las cuales se presentan en la Tabla 4. Esto ayudó a encontrar estudios sobre amenazas de spoofing y métodos de detección con redes neuronales, alineándose con los criterios de inclusión y exclusión definidos para el estudio.

La Tabla 4 ilustra la formación y elección de las secuencias de búsqueda basadas en los términos del GC y en la cantidad de documentos obtenidos en IEEE Xplore. La diversidad de resultados confirma que las cadenas permiten recuperar literatura relevante y directamente vinculada con la detección de clonación de voz y audio *deepfake*.

Tabla 4

Elaboración de la cadena de búsqueda óptima

Nombre de cadena de búsqueda	N#	Distribución
	Documentos	
(deepfake voice OR AI-synthesized speech)	339	Conferences (284), Journals
AND (detection OR anti-spoofing)		(44), Books (7), Early Access
		(3), Magazines (1)

(speech synthesis OR voice conversion OR replay) AND (spoofing) AND (speaker verification)	385	Conferences (295), Journals (88), Magazines (2)
(CNN OR convolutional neural network) AND (spectrogram OR MFCC OR LFCC OR CQCC) AND (deepfake audio OR spoofing)	148	Conferences (124), Journals (24)
(CNN-LSTM OR CRNN OR hybrid models) AND (AI-synthesized speech OR cloned voice) AND (classification OR detection)	4	Conferences (3), Journals (1)
(ASVspoo OR benchmark dataset) AND (deepfake OR spoofing) AND (evaluation OR metrics)	318	Conferences (229), Journals (83), Early Access (6)

Fuente: Elaboración propia

Selección de los estudios primarios

De las cadenas de búsqueda aplicadas en bases de datos académicas y tras la aplicación de los criterios de inclusión y exclusión (CI/CE), se priorizaron aquellos estudios que: (a) abordan el *spoofing* por síntesis, *voice conversion* o ataques de *replay*, (b) proponen técnicas de *machine learning* o *deep learning* para la detección, y (c) reportan resultados experimentales o procedimientos reproducibles.

Como resultado del proceso de búsqueda y aplicación de criterios de inclusión y exclusión, se seleccionaron cinco estudios primarios, presentados en la Tabla 5. Estos trabajos fueron considerados los más relevantes y representativos del estado del arte, y constituyen la elaboración del marco teórico que sustenta las preguntas de investigación planteadas.

Tabla 5

Estudios Primarios

Código	Título
EP1	<i>On the Vulnerability of Speaker Verification to Realistic Voice Spoofing</i>
EP2	<i>Detection of AI-synthesized speech using CNN-LSTM networks</i>
EP3	<i>Deepfakes Audio Detection Techniques Using Deep Convolutional Neural Network</i>
EP4	<i>Unmasking the Fake: Machine Learning Approach for Deepfake Voice Detection</i>
EP5	<i>Spoofing and countermeasures for speaker verification: A survey</i>

Fuente: Elaboración propia

Estos estudios fueron seleccionados por su relevancia directa en la detección de clonación de voz y audio *deepfake*.

En conjunto, conforman el núcleo de evidencia científica que sustenta la presente investigación y permiten establecer un marco sólido para el análisis de enfoques, métricas y arquitecturas aplicables al prototipo propuesto.

Síntesis de los Estudios Primarios

En esta sección se presenta la síntesis de los estudios identificados a partir del proceso de revisión sistemática de la literatura. Inicialmente, se seleccionaron cinco estudios primarios, considerados los más relevantes y representativos del estado del arte, tal como se muestra en la Tabla 5. No obstante, se incorporan estudios adicionales que enriquecen el análisis y amplían la metodología.

El propósito de este análisis es buscar diferentes enfoques, arquitecturas y métricas empleadas en la detección de clonación de voz. Esto, con la finalidad de ofrecer un respaldo técnico y metodológico a la proposición de este estudio. Y que se enfoca en la creación de un sistema apoyado en modelos de aprendizaje profundo y su implementación en una aplicación móvil.

EP1: On the Vulnerability of Speaker Verification to Realistic Voice Spoofing

Analiza cómo se puede burlar los sistemas de verificación del hablante ASV, por medio de

ataques de repetición, síntesis de voz, y conversión de voz. Muestra, incluso, que hasta los sistemas más robustos sufren importantes caídas, y eso ayuda a sustentar el problema y la imperiosa necesidad de los mecanismos anti-spoofing.

EP2: Detection of AI-Synthesized Speech Using CNN–LSTM Networks Presento un enfoque combinado, una mezcla de CNN y LSTM para detectar voz artificial. Las CNN se ocupan de identificar patrones que están dentro del espectro del audio y las LSTM registran la manera de evolucionar de estos patrones a lo largo del tiempo. Los resultados hallados indican un funcionamiento estable, lo cual justifica el uso de las arquitecturas CNN y LSTM como base para esta tesis.

EP3: Deepfakes Audio Detection Techniques Using Deep Convolutional Neural Network Se detallaron tácticas de detención las cuales se basan en CNN profundas, además de incluir arquitecturas del tipo Xception. El estudio analiza representaciones espectrales para diferenciar entre muestras de audio reales y falsas, consiguiendo un nivel alto de precisión, corroborando la utilidad de la CNN para la detección de voz.

EP4: Unmasking the Fake: Machine Learning Approach for Deepfake Voice Detection Evalúa distintos enfoques de *machine learning* para detección de *deepfake* de voz. Reporta métricas como accuracy, precision, recall y F1-score, reforzando la pertinencia de estas métricas para evaluar el sistema propuesto.

EP5: Spoofing and Countermeasures for Speaker Verification: A Survey Sistematiza los tipos de ataques de *spoofing* y las contramedidas existentes. Su aporte principal es la consolidación teórica y metodológica, justificando la necesidad de sistemas de detección complementarios en ASV.

EP6: Deepfake Audio Detection Using Deep Neural Networks (Prajapati et al., 2021) Propone redes neuronales profundas para detección de *deepfake* de audio, evaluadas

con métricas de clasificación. Confirma la efectividad de enfoques basados en *deep learning* para distinguir audios reales de generados.

EP7: Audio Deepfake Detection Using Deep Learning (Arif et al., 2021) Analiza arquitecturas de *deep learning* aplicadas a detección de audio *deepfake*. Reporta resultados con accuracy y F1-score, reforzando la adopción de técnicas profundas como solución efectiva.

EP8: A Robust Deep Learning Approach for Audio Deepfake Detection (Hassan et al., 2021) Presenta un enfoque robusto con modelos profundos entrenados sobre representaciones espectrales lo que pone en evidencia las mejoras en la capacidad de generalización, ya que es un aspecto clave para escenarios prácticos.

EP9: Detection of AI-Synthesized Speech Using CNN–LSTM Networks (Zhang et al., 2022) Pon en evidencia que el refuerzo del uso de CNN–LSTM para la detección de voz sintetizada, también destaca su capacidad para capturar información espectral y temporal con resultados competitivos.

EP10: Lightweight Audio Spoofing Detection for Mobile Devices (Seong et al., 2021) Aborda la detección de spoofing proponiendo modelos ligeros para la implementación en dispositivos móviles con recursos limitados. Lo que permite señalar que es factible desplegar una aplicación Android con un sistema de detección.

EP11: DNR-HiNet: Deep Neural Dereverberation and Denoising for Speech Enhancement (ai2021.pdf) Indica el uso de redes profundas con técnicas dereverberación y denoising para examinar el preprocesamiento de audio. Su impacto permite mejorar la calidad de las características de los audios mejorando primero la señal del audio, lo que da paso a mejorar el sistema de detección ante entornos que perjudican el audio.

EP12: Automatic Speaker Verification Spoofing and Countermeasures Challenge: Feature Extraction with MFCC (Liang et al., 2017) Propone el uso de coeficientes cepstrales en frecuencia de Mel (MFCC) y otras variantes espectrales como CQCC y LFCC para la detección de *spoofing*. Su relevancia radica en establecer las bases de extracción de

características acústicas que aún hoy se utilizan en sistemas modernos de detección de clonación de voz.

En resumen, los doce estudios primarios ofrecen un panorama que abarca desde el preprocesamiento y la extracción de características acústicas de los audios, hasta el uso de arquitecturas profundas y criterios de evaluación.

La integración de estos estudios respalda la propuesta de investigación y también, confirma la viabilidad para implementar un sistema que detecte la clonación de voz en dispositivos móviles.

Métricas seleccionadas para responder las preguntas de investigación

A partir del análisis de los estudios primarios y de las métricas reportadas en la literatura, se definieron las métricas que se emplearán en este trabajo de titulación para responder a las preguntas de investigación planteadas.

La Tabla 6 resume las métricas seleccionadas para cada pregunta de investigación y los estudios que las respaldan.

Tabla 6

Métricas seleccionadas y relación con las preguntas de investigación

Pregunta de investigación	Métricas seleccionadas	Estudios primarios donde se reportan
RQ1: ¿Qué enfoques han demostrado mayor efectividad para detectar voz clonada con potencial de despliegue en móviles?	Accuracy, F1-score	EP2, EP3, EP4, EP6, EP7, EP9, EP10
RQ2: ¿Qué métricas de evaluación resultan más adecuadas para medir la efectividad en escenarios reales?	EER, FAR, FRR, Accuracy, F1-score	EP1, EP5, EP11, EP12 (EER, FAR, FRR); EP2,

EP4, EP7, EP9 (Accuracy,
F1-score)

Fuente: Elaboración propia

En este trabajo se emplearán Accuracy y F1-score como métricas principales para responder a la primera pregunta, y EER, FAR, FRR junto con Accuracy y F1-score para la segunda, garantizando una evaluación integral y coherente con la literatura científica.

Síntesis de resultados de la revisión

En esta sección se describen los aportes más relevantes de los estudios seleccionados durante la revisión sistemática de la literatura. El propósito consiste en examinar los enfoques, arquitecturas y métricas empleadas en la detección de clonación de voz (audio deepfake), con el objetivo de sustentar técnica y metodológicamente la propuesta de este trabajo de titulación, la cual se orienta al desarrollo de un sistema basado en modelos de deep learning implementado en una aplicación móvil.

Para la primera pregunta, los estudios muestran que los mejores métodos para detectar voces clonadas usan la señal de voz a través de lo que ves en un gráfico de sonido o en el tiempo, como los espectrogramas o los llamados MFCC, CQCC y LFCC. Con estas características, se pueden encontrar patrones en la señal de voz que ayudan a saber si un audio es real o generado artificialmente.

En cuanto a las redes neuronales, las que más se usan son las convolucionales (CNN) y las que combinan CNN con LSTM. Estas últimas funcionan mejor porque analizan la señal de audio tanto en el espacio como en el tiempo. Hay estudios como el EP2 y el EP9 que dicen que la combinación CNN-LSTM es buena. Otros como el EP3, EP6, EP7 y EP8 confirman que las redes neuronales a fondo funcionan muy bien. Además, el EP10 deja claro que se pueden poner estos modelos mejorados en celulares, lo que significa que se podrían usar en aplicaciones de Android.

Para la segunda pregunta, el análisis de los estudios primarios muestra que la evaluación del desempeño de los sistemas de detección de clonación de voz se realiza mediante una combinación de métricas estándar de clasificación y métricas biométricas especializadas. Las más comunes son la precisión, la puntuación F1 y la tasa de error igual (EER). También se usan la tasa de falsa aceptación (FAR) y la tasa de falso rechazo (FRR) cuando la seguridad es un tema importante.

Con estas medidas se puede saber si el sistema funciona correctamente y si hay errores graves, como aceptar un audio falso o no dejar pasar uno verdadero. Esto es muy importante en la vida real, para saber si alguien es quien dice ser y para que no suplantén identidades.

A continuación, se presenta la tabla 7 que resume las métricas de evaluación utilizadas en los estudios primarios analizados y los trabajos en los que se reporta su aplicación.

Tabla 7

Métricas de evaluación y estudios primarios donde se reportan

Métrica de evaluación	Estudios primarios
Accuracy (Exactitud)	EP2, EP3, EP4, EP6, EP7, EP9, EP10
Precision	EP4
Recall (Sensibilidad)	EP4
F1-score	EP4, EP7, EP9
Equal Error Rate (EER)	EP1, EP5, EP11, EP12
False Acceptance Rate (FAR)	EP1, EP5
False Rejection Rate (FRR)	EP1, EP5
ROC/AUC	EP3, EP8

Fuente: Elaboración propia

En resumen, los enfoques basados en características acústicas espectro-temporales y arquitecturas CNN–LSTM, evaluados mediante métricas estándar y biométricas, ayudan a construir una base más sólida y viable para la detección de clonación de voz en entornos prácticos, dando respuesta a las preguntas de investigación planteadas.

En el siguiente capítulo se detalla la construcción de los modelos utilizados dentro de la aplicación móvil.

Capítulo III

En este capítulo se aborda el diseño y entrenamiento de dos modelos híbridos basados en redes neuronales profundas, implementados en TensorFlow/Keras, los cuales conforman el núcleo del prototipo propuesto.

El primero de ellos se orienta a distinguir entre voces reales y voces generadas por inteligencia artificial, mientras que el segundo se enfoca en determinar el género del hablante a partir de sus características acústicas.

Ambos modelos se integran posteriormente en la aplicación móvil AntiFakeVoice, conformando una herramienta unificada capaz de analizar la autenticidad y naturaleza de una señal de voz en tiempo real, ofreciendo así una respuesta automática, confiable y adaptable a distintos contextos de uso. El desarrollo metodológico se estructura en cuatro actividades principales:

1. Recopilar datasets de voces reales y clonadas, que pertenezcan al español castellano.
2. Preparar los audios, creando espectrogramas log-Mel y coeficientes Cepstrales de Mel (MFCC), que permitan capturar los detalles sonoros del habla.
3. Implementar las arquitecturas CNN–LSTM con atención y transformadores, ajustando sus hiperparámetros durante el entrenamiento y validación.
4. Evaluemos los dos modelos empleando métricas de precisión. Así también, usa el puntaje F1, AUC, la tasa de falsos positivos y negativos y, por último, la tasa de error igual (EER). Todo esto para, conjunto de datos de validación, así como, grabaciones nunca vistas.

La propuesta se fundamenta en la evidencia científica que demuestra la efectividad de los modelos híbridos CNN–RNN con atención para la detección de falsificaciones de audio y la clasificación de características vocales. De acuerdo con Todisco et al. 2019, las arquitecturas

que combinan redes convolucionales y recurrentes logran resultados superiores al 98 % de precisión en la tarea de detección de spoofing de voz en entornos controlados.

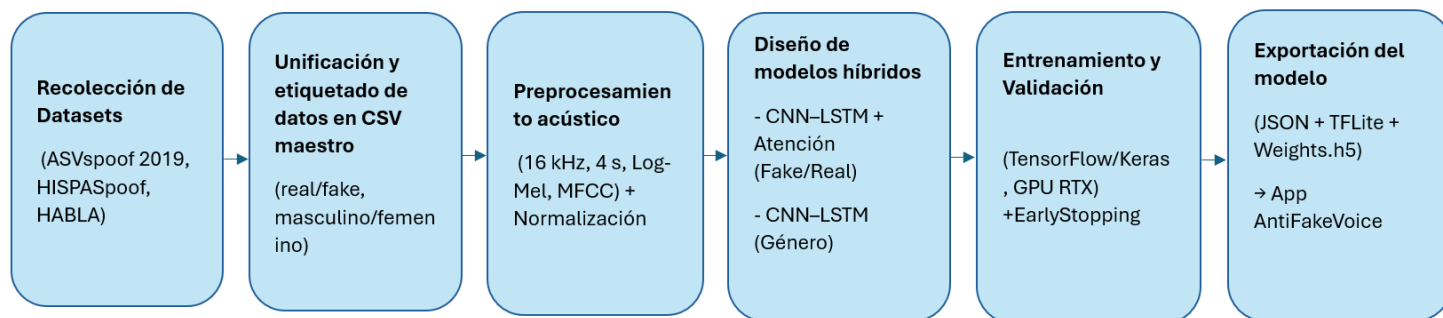
Proceso metodológico general de los modelos

En este trabajo se describe una metodología de investigación aplicada orientada al desarrollo de un prototipo tecnológico para la detección automática de clonación de voz y la clasificación del género del hablante. La propuesta se fundamenta en el diseño, implementación y entrenamiento propio de un modelo de inteligencia artificial, construido a partir de arquitecturas de aprendizaje profundo y adaptado específicamente al análisis de señales de voz.

El modelo de inteligencia artificial usado en esto trabajo no es una solución preentrenada, utilizada directamente el proceso de extracción de características acústicas, los parámetros de entrenamiento y las métricas utilizadas para su evaluación. Dicho modelo fue hecho un pipeline que comprende las etapas de adquisición, procesamiento y modelado de las señales de audio, que garantiza su correcta integración con el entorno TensorFlow/Keras y su posterior implementación en una aplicación móvil desarrollada para el sistema operativo Android. En la Figura 6 se muestra de manera esquemática el flujo general seguido en la construcción de los modelos:

Figura 6

Diagrama general del proceso metodológico para el diseño y entrenamiento de los modelos híbridos CNN–LSTM con transformadores



Fuente: Elaboración propia

Recolección de dataset

La recopilación de los datos requirió un trabajo exhaustivo de búsqueda, análisis y verificación, debido a la escasez de bases de datos de detección de deepfakes en español.

Luego de una gran búsqueda en plataformas, repositorios, artículos científicos, se encontró tres fuentes de datos bastante aceptables: HABLA, HISPASpoof y el Latin America Spanish Anti-Spoofing Dataset (FoR). Todos y cada uno de ellos poseen distintas particularidades en su estructura y documentación.

Hallar estos conjuntos de datos fue un proceso amplio en Zenodo, GitHub y Hugging Face, examinando cuidadosamente los protocolos de cada dataset y los metadatos compartidos para de esta manera determinar su validez, compatibilidad y autenticidad.

Datasets utilizados

HISPASpoof

El dataset HISPASpoof, desarrollado por el grupo Purdue ViperLab (HISPASpoof, 2023), contiene grabaciones en español provenientes de diversas regiones hispanohablantes. Este corpus incluye tanto audios reales (bonafide) como falsos (spoof) generados mediante motores comerciales y de código abierto de Text-to-Speech (TTS) y Voice Conversion.

La distribución general del dataset, que incluye el número de muestras por partición y la proporción de audios reales y falsificados, se presenta en la Tabla 8, la cual resume de forma estructurada la organización interna del conjunto de datos utilizado en esta investigación.

Tabla 8

Distribución general del dataset HISPASpoof

Subconjunto	Audios reales (bonafide)	Audios falsos (spoof)	Total de audios
Entrenamiento (train)	0	16,754	16,754
Validación (val)	0	2,083	2,083
Prueba (test)	6,241	18,606	24,847
Total general	6,241	37,443	43,684

Fuente: Elaboración propia

Los audios falsos fueron generados mediante una combinación de motores comerciales (por ejemplo, Microsoft Azure TTS, Google TTS) y herramientas open source (Tacotron, FastSpeech, Festival).

Este dataset fue utilizado tanto para la detección de voces reales o falsas como para extraer voces reales destinadas al modelo de clasificación de género.

Latin America Spanish Anti-Spoofing Dataset (FoR)

El dataset FoR (Latin America Spanish Anti-Spoofing Dataset), desarrollado por Ruframapi et al. (2022), recopila grabaciones de hablantes latinoamericanos junto con voces clonadas mediante diversos métodos de síntesis, incluyendo Voice Conversion y Text-to-Speech.

Este dataset esta conformado por un total de 80,816 audios, en el cual se encuentra una diversidad de acentos y técnicas de deepfake. La Tabla 9 muestra la estructura del dataset.

Tabla 9

Distribución general del dataset FoR

Categoría	Cantidad de audios	Porcentaje aproximado
Voces reales (bonafide)	22,816	28 %
Voces falsas (spoof)	58,000	72 %
Total general	80,816	100 %

Fuente: Elaboración propia

Las voces falsas incluidas en el dataset FoR fueron generadas mediante tres enfoques principales de síntesis: *Voice Conversion (VC)*, *Text-to-Speech (TTS)* y modelos híbridos que combinan ambas metodologías (TTS + VC).

Cada una de estas técnicas aporta patrones espectrales y temporales distintos, lo que amplía la diversidad del dataset y fortalece la capacidad del modelo para aprender a detectar diferentes tipos de falsificación.

La distribución completa de audios por tipo de técnica de generación se presenta en la Tabla 10, donde se detalla la cantidad de muestras asociadas a cada enfoque y su contribución dentro del conjunto total.

Tabla 10

Distribución por tipo de falsificación en el dataset FoR

Tipo de falsificación	Técnica / Modelo	Cantidad de audios	Descripción breve
Voice Conversion (VC)	CycleGAN	16,000	Convierte una voz en otra sin datos paralelos.
	DiffusionVC	16,000	Usa difusión progresiva para recrear la voz con realismo.
	StarGAN	16,000	Permite múltiples conversiones entre voces con un solo modelo.
Text-to-Speech (TTS)	Microsoft Azure TTS	5,000	Voces creadas directamente desde texto.
Híbridos (TTS + VC)	TTS-Diffusion	2,500	Texto convertido a voz y luego refinado con difusión.
	TTS-StarGAN	2,500	Texto convertido a voz, transformado al timbre de otra persona.
Total		58,000	

Fuente: Elaboración propia

Además, las voces reales incluidas en el dataset FoR fueron proporcionadas por hablantes de cinco países latinoamericanos, lo que garantiza una representación diversa de acentos, tonos y particularidades fonéticas propias de la región. Esta diversidad lingüística es fundamental para mejorar la capacidad de generalización del modelo frente a variaciones reales del habla.

La distribución completa de los audios reales por país se presenta en la Tabla 11, donde se detalla la cantidad de muestras correspondientes a cada nación participante dentro del corpus.

Tabla 11*Distribución de voces reales por país (FoR)*

País	Cantidad de audios reales
Argentina	5,460
Perú	5,446
Colombia	4,604
Chile	4,089
Venezuela	3,217
Total	22,816

Fuente: Elaboración propia**Dataset HABLA**

El dataset HABLA, presentado por Ruframapi et al. (2022), constituye un subconjunto derivado del corpus FoR, reorganizado específicamente para facilitar las tareas de entrenamiento y evaluación en estudios de anti-spoofing en español.

El dataset HABLA está organizado en dos particiones principales: entrenamiento/desarrollo (train_dev) y evaluación (eval), lo que da paso a un flujo más directo para experimentación supervisada.

La Tabla 12 muestra la cantidad de audios tanto reales como falsos, lo que permite mostrar de forma clara su estructura interna.

Tabla 12*Distribución general del dataset HABLA*

Subconjunto	Audios reales (bonafide)	Audios falsos (spoof)	Total de audios
Entrenamiento / desarrollo (train_dev)	13,759	34,730	48,489
Evaluación (eval)	9,057	23,270	32,327
Total general	22,816	58,000	80,816

Fuente: Elaboración propia

Dado que HABLA comparte la misma base de archivos que FoR, pero con una organización distinta, este dataset se empleó para validar la consistencia de los datos y realizar pruebas cruzadas de rendimiento durante la etapa de evaluación.

Construcción del dataset maestro

Con la validación de los tres conjuntos de datos, se procedió a crear un CSV maestro que integra la información de cada uno, entre esta información se encuentra: la ruta de cada audio, su tipo (real o falso) y los metadatos relevantes provenientes de los corpus originales.

Para el modelo de clasificación de género se tomó en cuenta los audios reales de los conjuntos de datos y la etiqueta masculina o femenina correspondiente a cada audio.

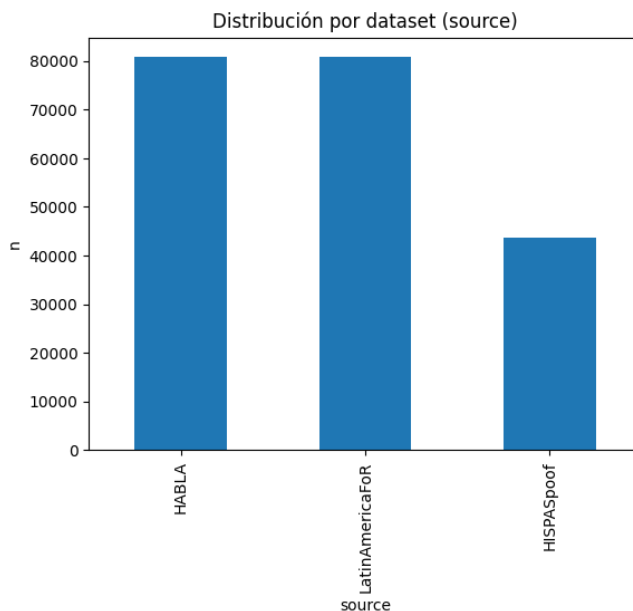
Por otro lado, para el modelo de detección de voces reales o falsas, se utilizaron todas las muestras que contienen los conjuntos de datos

En la Figura 7 se observa la distribución de las muestras del dataset dentro del CSV maestro, de manera que se puede notar que HABLA y FoR aportan muchos más que HISPASpoof. Esta representación gráfica permitió comprobar de manera coherente los datos integrados y el balance entre ellos.

Cada registro del CSV incluye elementos importantes que se requieren para el entrenamiento. Estos elementos se detallan en la Tabla 13, su significado y el rol que cumplen en el proceso de preparación y análisis de los audios.

Figura 7

Cantidad de datos de cada dataset



Fuente: Elaboración propia

Tabla 13

Columnas del CSV

Columna	Descripción
file_path	Ruta completa del archivo de audio
dataset	Origen del dataset (FoR, HABLA o HISPASpoof)
label_fake	Etiqueta binaria: 0 = real, 1 = fake
label_gender	Etiqueta binaria: 0 = masculino, 1 = femenino
duration	Duración del audio (segundos)
country	País o región de origen del hablante

Fuente: Elaboración propia

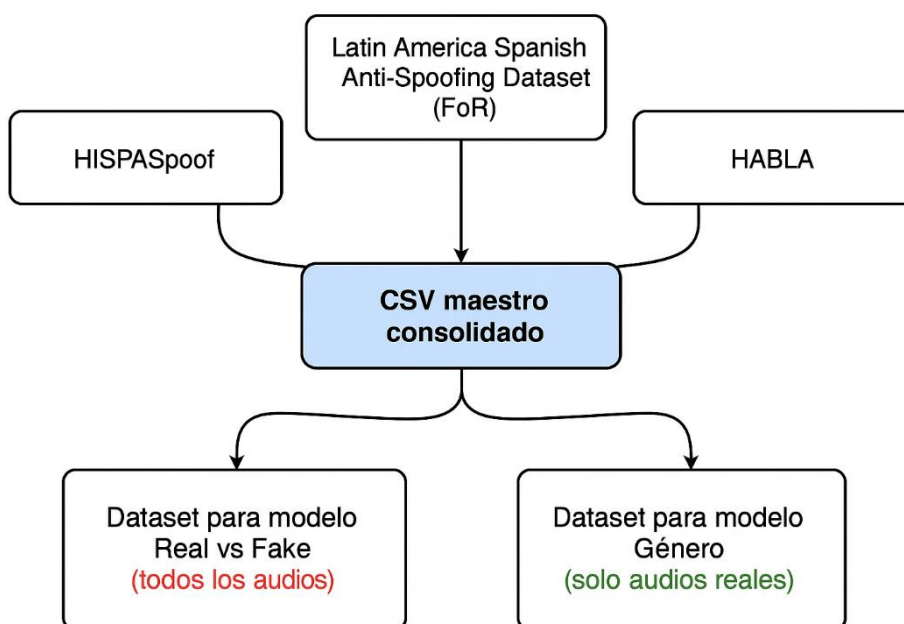
La consolidación del CSV maestro permitió simplificar significativamente las operaciones de lectura, filtrado y preparación de datos dentro de TensorFlow, facilitando además la segmentación rápida por tareas, tanto para la detección de voces reales o falsas como para la clasificación de género.

Este archivo centralizado actuó como el punto de partida del pipeline completo, ya que a partir de él se generaron las representaciones espectro-temporales (espectrogramas log-Mel y coeficientes MFCC) empleadas en la fase de preprocesamiento.

El proceso de integración de los tres datasets y la posterior construcción del CSV maestro se resume visualmente en la Figura 8, la cual ilustra el flujo de consolidación utilizado para organizar los datos de entrenamiento de manera coherente y eficiente.

Figura 8

Flujo de consolidación del dataset maestro para entrenamiento de modelos.



Fuente: Elaboración propia

Análisis

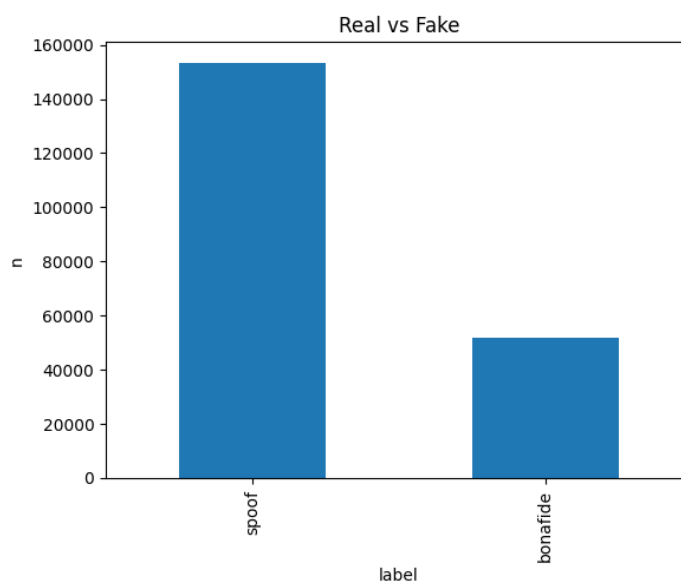
El uso combinado de los tres datasets permitió alcanzar un equilibrio adecuado entre diversidad de acentos, cantidad de muestras y variedad de técnicas de falsificación, fortaleciendo así la capacidad del modelo para generalizar frente a contextos reales de voz.

La consolidación de estos corpus en un único conjunto en español latinoamericano permitió además eliminar la dependencia de bases en inglés, garantizando una mejor adaptación lingüística al entorno de aplicación del sistema AntiFakeVoice.

La Figura 9 presenta la proporción entre audios reales (bonafide) y falsificados (spoof), confirmando la presencia de una amplia variedad de técnicas de síntesis y conversión de voz, lo cual incrementa la robustez del modelo durante el entrenamiento.

Figura 9

Distribución global de audios reales (bonafide) y falsificados (spoof).



Fuente: Elaboración propia

Según Ruframapi et al. (2022), el dataset Latin America FoR se diseñó precisamente para suplir la carencia de corpora de anti-spoofing en español, incorporando voces reales y sintéticas con acentos regionales. A su vez, el dataset HISPASpoof, compilado por el equipo Purdue ViperLab (2023), complementa este enfoque al incluir motores TTS y VC de distintas procedencias, aportando realismo y complejidad al conjunto final utilizado en esta investigación.

Preprocesamiento y extracción de características

El preprocesamiento de los datos de audio constituye una etapa fundamental en la construcción de modelos de aprendizaje profundo, ya que de su correcta ejecución depende directamente la calidad de las características que la red neuronal puede aprender. En esta investigación se implementó un *pipeline* automatizado para transformar los archivos de audio crudos provenientes de los diferentes datasets en representaciones numéricas normalizadas, adecuadas para el entrenamiento de los modelos híbridos CNN–LSTM.

Cada archivo de audio fue normalizado a una frecuencia de muestreo de 16 kHz, valor ampliamente recomendado para tareas de procesamiento del habla, ya que permite capturar de manera adecuada el rango de frecuencias más relevante para la voz humana (0–8000 Hz) sin introducir información redundante. De acuerdo con Rabiner y Schafer (2011) y Stevens, Volkmann y Newman (1937), la mayor parte de la energía lingüística y de las características fonéticas del habla se concentra por debajo de los 8 kHz, lo que hace que este valor de muestreo sea suficiente, eficiente y comúnmente adoptado en sistemas modernos de análisis y reconocimiento de voz.

Asimismo, todos los audios fueron recortados o completados a una duración estándar de 4 segundos, lo que permitió trabajar con tensores de entrada de tamaño uniforme, simplificando el proceso de entrenamiento y evitando variaciones indebidas en las dimensiones de los espectrogramas. Este enfoque es habitual en sistemas de *anti-spoofing* y en trabajos recientes de detección de *audio deepfake*, como los presentados en las competencias ASVspoof (Todisco et al., 2019).

Después, los audios, se convirtieron en espectro-temporales representaciones usando espectrogramas logMel y coeficientes cepstrales de frecuencia Mel, oh si MFCCs. Estas técnicas son ampliamente aplicadas, por su buen modelado de la percepción del sonido humano. Las escalas Mel, diseñadas por Stevens, Volkmann y Newman, allá en 1937, ahora

son un estándar en el procesamiento del habla, por su buena captura de la estructura acústica de la voz.

En esta fase también, se incluye el enmascaramiento de frecuencia, enmascaramiento y desplazamiento temporal de la señal, según SpecAugment de Park et al, del 2019. Tales técnicas, incrementaron la robustez del modelo frente a cambios del entorno, ruido y diferentes dispositivos, más las diferencias entre locutores, lo que reduce el sobreajuste y mejorando la generalización del sistema.

Preparación del entorno y carga de librerías

El procesamiento se realizó en un entorno local de desarrollo configurado con Python 3.9, TensorFlow 2.10.1, y compatibilidad GPU mediante CUDA 12.0 y cuDNN 8.9, ejecutado en una tarjeta gráfica NVIDIA GeForce RTX 4050.

El entorno virtual fue creado en Visual Studio Code bajo el nombre TESIS, con las dependencias gestionadas mediante Anaconda.

Las librerías empleadas se seleccionaron por su robustez y compatibilidad con TensorFlow/Keras. En la Tabla 14 se resumen las más relevantes:

Tabla 14

Librerías empleadas

Librería	Función principal
Numpy	Manipulación de arreglos multidimensionales y operaciones numéricas.
Pandas	Lectura y gestión del archivo CSV maestro.
Librosa	Procesamiento de señales de audio: carga, re-muestreo, MFCC y espectrogramas log-Mel.
matplotlib seaborn	Visualización de espectrogramas, curvas y distribución de datos.
tensorflow.keras	Definición, entrenamiento y exportación de los modelos CNN–LSTM.
Sklearn	Cálculo de métricas, división de conjuntos y normalización.

Fuente: Elaboración propia

Normalización y segmentación de audios

Los archivos de audio provenientes de los datasets HISPASpoof, FoR y HABLA presentaban diferencias en formato, duración, nivel de ganancia y condiciones de grabación. Por ello, se aplicó un proceso uniforme de preprocesamiento siguiendo las recomendaciones establecidas en la literatura de procesamiento digital de voz.

1. Re-muestreo a 16 kHz. Todos los audios fueron re-muestreados a una frecuencia fija de 16 kHz, ya que este valor permite representar con fidelidad el rango de frecuencias relevante para el habla humana (hasta 7.6–8 kHz) sin incurrir en redundancia ni incremento innecesario del tamaño de las señales. Este criterio es ampliamente aceptado en sistemas de reconocimiento automático del habla y análisis de voz (Rabiner & Schafer, 2011; Stevens et al., 1937).
2. Conversión a mon-canal. Señales se transformaron en un solo canal, eliminando redundancias estéreo, conforme a las prácticas de O'Shaughnessy (2008). Pues la información lingüística no sufre con la espacialidad del audio en análisis vocal.
3. Segmentación, o rellenado a 4 segundos. Se recortaron, o completaron cada audio a una duración estándar de 4 segundos, así asegurar que todos los ejemplos tuvieran una dimensión uniforme en el entrenamiento. Esta estandarización es importantísima para modelos basados en CNN y LSTM. Ya que evita variaciones inesperadas en la longitud temporal de las entradas (Todisco et al. , 2019).
4. Ajuste de ganancia a –20 dBFS. La amplitud de cada señal se normalizó a –20 dBFS como nivel de referencia, algo típico en el procesamiento de voz, para un rango dinámico equilibrado y evitando la saturación, como lo sugirieron Rabiner & Schafer (2011), al digitalizar señales.

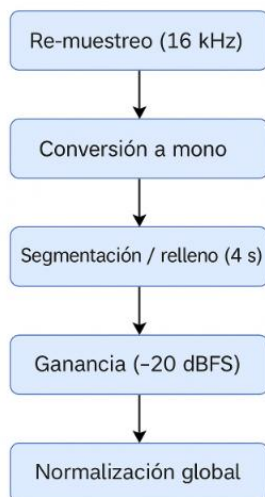
5. Normalización global fue realizada por ultimo. Después todos los espectrogramas log-Mel fueron normalizados, empleando la media ($\text{global_mean} \approx 0.1071$) y la desviación estándar ($\text{global_std} \approx 0.4273$), valores calculados con el conjunto de entrenamiento. Goodfellow, Bengio y Courville (2016), afirman que esta normalización estadística favorece la estabilidad del entrenamiento y acelera la convergencia, especialmente en redes profundas.

La Figura 10 muestra el preprocesamiento de los audios. Estos valores fueron almacenados en el archivo `model_config.json`, garantizando consistencia entre las etapas de entrenamiento e inferencia, así como la correcta reproducción del pipeline dentro de la aplicación móvil *AntiFakeVoice*.

Figura 10

Proceso de preprocesamiento acústico aplicado a los audios de los datasets HISPASpoof, FoR y HABLA

Preprocesamiento acústico



Fuente: Elaboración propia

Extracción de características acústicas

Para representar la información sonora de manera que pueda ser comprendida por la red neuronal, cada fragmento de audio fue transformado en espectrogramas log-Mel y coeficientes cepstrales de Mel (MFCCs).

Espectrogramas log-Mel

Se calcularon con los siguientes parámetros:

- $n_mels = 80$ (bandas mel)
- $win_ms = 25$ ms (ventana)
- $hop_ms = 10$ ms (salto)
- $fmin = 20$ Hz, $fmax = 7600$ Hz

Esta representación captura tanto la energía como la variación temporal del habla, y simula la percepción auditiva humana en escala de Mel (Stevens, Volkman & Newman, 1937).

Coefficientes MFCC (Mel Frequency Cepstral Coefficients)

Se extrajeron para complementar la información del timbre y la envolvente espectral. Se emplearon 13 coeficientes por ventana, junto con sus derivadas de primer y segundo orden obteniendo una descripción tridimensional del sonido.

La combinación de log-Mel y MFCC permitió que los modelos CNN–LSTM aprendieran tanto patrones espectro-temporales como características de la envolvente vocal, mejorando la discriminación entre voces reales y sintetizadas.

Aumento de datos

Con el fin de evitar el sobreajuste (overfitting) y fortalecer la capacidad del modelo para generalizar frente a diferentes condiciones acústicas, se implementaron diversas técnicas de data augmentation inspiradas en el método SpecAugment propuesto por Park et al. (2019).

Estas técnicas introducen variaciones controladas en las representaciones espectrales, lo que permite simular cambios reales en el entorno de grabación y variaciones naturales del locutor.

Los principales argumentos y estrategias empleadas para mitigar el sobreajuste se resumen en la Tabla 15, donde se detallan los tipos de perturbaciones aplicadas, su propósito y el impacto esperado sobre el proceso de entrenamiento.

Tabla 15

Argumentos de sobreajuste

Técnica	Descripción
Máscara de frecuencia	Elimina aleatoriamente una o más bandas del espectrograma, simulando pérdidas de señal.
Máscara temporal	Oculto intervalos de tiempo para que el modelo dependa de contexto global.
Desplazamiento temporal	Corre el audio aleatoriamente unos milisegundos.
Ruido Gaussiano leve	Añade variaciones realistas a la señal, fortaleciendo la robustez.

Fuente: Elaboración propia

Diseño e implementación de los modelos híbridos CNN–LSTM

El diseño e implementación de los modelos propuestos constituyen el núcleo técnico de la investigación, ya que en esta fase se materializa el objetivo de desarrollar un sistema de detección automática capaz de diferenciar entre voces humanas y generadas por inteligencia artificial, así como determinar el género del hablante.

Ambos modelos fueron implementados en TensorFlow/Keras siguiendo una arquitectura híbrida CNN–LSTM, aprovechando la capacidad de las redes convolucionales para extraer patrones espectro-temporales y la potencia de las redes recurrentes para modelar dependencias secuenciales a lo largo del tiempo (Li et al., 2022).

Enfoque arquitectónico general

El modelo híbrido CNN–LSTM combina lo mejor de dos paradigmas del aprendizaje profundo:

- Las redes convolucionales (CNN) identifican patrones locales en el espectrograma, tales como formantes, cambios de energía o artefactos digitales generados por motores de síntesis de voz.
- Las redes LSTM bidireccionales (BiLSTM) capturan la evolución temporal de la voz, comprendiendo el contexto antes y después de cada fotograma acústico.

Ambas arquitecturas comparten la misma base estructural, pero difieren en la capa de salida y en el tipo de datos empleados durante el entrenamiento.

Modelo 1: Detección de voces reales o falsas

Este modelo tiene como objetivo determinar si una señal de audio corresponde a una voz humana (real) o una voz generada por IA (fake). Se entrenó utilizando todos los audios disponibles de los tres datasets (reales + falsos), balanceadas mediante pesos de clase {0: 0.648, 1: 0.352} para compensar la mayor cantidad de voces falsas. La Tabla 16 contiene la estructura final del modelo, así como la Tabla 17 contiene los parámetros que se utilizara para el entrenamiento.

Tabla 16

Estructura del modelo

Capa	Tipo	Forma de salida
Input	(80, 400, 1)	Entrada del espectrograma log-Mel
Conv2D (32 filtros) + BN + ReLU	(80, 400, 32)	Detección de patrones básicos
MaxPooling2D	(40, 200, 32)	Reducción de dimensionalidad
Conv2D (64 filtros) + BN + ReLU	(40, 200, 64)	Extracción de características intermedias
MaxPooling2D	(20, 100, 64)	Consolidación de características

Conv2D (128 filtros) + BN + ReLU	(10, 50, 128)	Captura de patrones complejos
Dropout (0.3)	-	Regularización
BiLSTM (128×2)	(None, 256)	Dependencias temporales bidireccionales
Attention Layer + Transformer Encoder	(None, 256)	Enfoque en regiones relevantes
Dense (128) + Dropout	(128)	Capa intermedia densa
Dense (1) + Sigmoid	(1)	Clasificación binaria (Real/Fake)

Fuente: Elaboración propia

Tabla 17

Parámetros de entrenamiento

Parámetro	Valor
Función de pérdida	BinaryCrossentropy
Optimizador	Adam (lr=0.0001)
Batch size	32
Épocas	16 (mejor resultado entre 12–16)
Callbacks	EarlyStopping, ReduceLROnPlateau, ModelCheckpoint

Fuente: Elaboración propia

Durante el entrenamiento, el modelo alcanzó una precisión del 98.6 %, con un AUC $\approx$ 0.998 y un EER $\approx$ 0.02, lo que demuestra una excelente capacidad para distinguir voces reales de sintéticas.

El umbral óptimo obtenido fue $\text{thr}^* = 0.662$, valor utilizado posteriormente en la inferencia de la aplicación móvil.

Modelo 2: Clasificación de género

El segundo modelo tiene como objetivo identificar si una voz humana corresponde a una persona masculina o femenina. Para evitar sesgos derivados de características artificiales introducidas por motores TTS o técnicas de Voice Conversion, este modelo fue entrenado exclusivamente con audios reales (bonafide) provenientes de los tres datasets empleados. De

esta manera, las características aprendidas por la red representan rasgos fonéticos y biológicos auténticos del habla humana.

El modelo mantiene la misma estructura convolucional y recurrente del modelo principal, ya que la clasificación de género se basa en patrones espectro-temporales asociados al timbre, la resonancia vocal y la frecuencia fundamental. No obstante, en este caso no se incluyeron bloques de atención ni módulos Transformer, dado que la tarea no requiere capturar dependencias globales tan complejas como en la detección de deepfakes.

La arquitectura concluye con una capa densa de una neurona (Dense (1)) con activación sigmoide, donde valores cercanos a 0 representan voz masculina y valores cercanos a 1 representan voz femenina.

La estructura completa del modelo utilizado para esta tarea se presenta en la Tabla 18, donde se detallan sus capas, dimensiones y parámetros principales por otra parte La tabla19 muestra los parámetros de esta.

Tabla 18

Estructura del modelo de clasificación de género

Capa	Tipo	Forma de salida
Input	(80, 400, 1)	Entrada del espectrograma log-Mel
Conv2D (32 filtros) + BN + ReLU	(80, 400, 32)	Patrones iniciales
MaxPooling2D	(40, 200, 32)	Reducción de dimensionalidad
Conv2D (64 filtros) + BN + ReLU	(40, 200, 64)	Extracción de características medias
MaxPooling2D	(20, 100, 64)	Consolidación de información
Conv2D (128 filtros) + BN + ReLU	(10, 50, 128)	Características profundas
Dropout (0.3)	-	Regularización

BiLSTM (128×2)	(None, 256)	Dependencias temporales
Dense (128) + Dropout	(128)	Capa intermedia
Dense (1) + Sigmoid	(1)	Salida binaria (Masculino/Femenino)

Fuente: Elaboración propia

Tabla 19

Parámetros de entrenamiento en el modelo de género

Parámetro	Valor
Función de pérdida	BinaryCrossentropy
Optimizador	Adam
Batch size	32
Épocas	20 (mejor resultado entre 10–18)
Métricas	accuracy, precision, recall, F1

Fuente: Elaboración propia

El modelo de clasificación de género alcanzó una precisión media del 96.8 %, con una distribución equilibrada de predicciones entre ambas clases.

Implementación en TensorFlow/Keras

Ambos modelos fueron definidos mediante la API funcional de TensorFlow Keras, lo que permitió conectar las capas de manera modular y exportarlas fácilmente en formato .h5 y .tflite.

Durante la fase de exportación se generaron archivos los cuales se ve en la Tabla 20.

Tabla 20

Archivos generados en el entrenamiento

Archivo	Descripción
---------	-------------

best.weights.h5	Pesos del modelo entrenado con mejor validación
model_config.json	Configuración estructural y parámetros globales (global_mean, global_std, threshold_opt)
model_fp16.tflite	Versión optimizada para ejecución móvil (FP16)

Fuente: Elaboración propia

La elección de Keras permitió reducir la complejidad del código y garantizar la reproducibilidad de los experimentos, cumpliendo con las mejores prácticas recomendadas por TensorFlow (Abadi et al., 2016).

Entrenamiento y validación de los modelos

El proceso de entrenamiento y validación de los modelos híbridos CNN–LSTM se desarrolló siguiendo un enfoque experimental controlado, orientado a obtener el máximo rendimiento con un riesgo mínimo de sobreajuste (overfitting).

Ambos modelos —detección de voces reales o falsas y clasificación de género— fueron entrenados de forma independiente sobre los conjuntos de datos procesados en el punto anterior, empleando estrategias de división, callbacks y métricas de evaluación recomendadas en entornos de aprendizaje profundo (Goodfellow, Bengio & Courville, 2016).

Configuración del entorno

Los entrenamientos se llevaron a cabo en un entorno local con las siguientes especificaciones técnicas:

- **Hardware:** GPU NVIDIA GeForce RTX 4050 (6 GB VRAM), CPU Intel Core i7 13^a gen, 32 GB RAM.
- **Software:** TensorFlow 2.10.1, Keras, CUDA 12.0, cuDNN 8.9, Python 3.9.
- **Entorno:** Visual Studio Code (entorno virtual “TESIS”).
- **Sistema operativo:** Windows 11 Pro 64 bits.

Cada entrenamiento fue ejecutado utilizando lotes (batches) de 32 muestras y una frecuencia de muestreo de 16 kHz, optimizando el uso de la GPU. El tiempo promedio por

época osciló entre 15 y 30 minutos, dependiendo del modelo y la cantidad de datos procesados.

Estrategia de división de datos

Para ambos modelos se aplicó una división estratificada de los datos en tres subconjuntos:

- **Entrenamiento (70 %)** – aprendizaje de los parámetros del modelo.
- **Validación (20 %)** – ajuste de hiperparámetros y control de sobreajuste.
- **Prueba (10 %)** – evaluación final sobre datos no vistos.

Configuración de entrenamiento

Ambos modelos fueron entrenados con el optimizador Adam y la función de pérdida Binary Crossentropy, debido a la naturaleza binaria de las salidas.

Se utilizaron callbacks para garantizar estabilidad y control durante el proceso de entrenamiento, tal como se detalla a continuación:

- **Optimizador:** Adam (learning_rate=1e-4).
- **Función de pérdida:** Binary Crossentropy
- **Batch size:** 32.
- **Épocas máximas:** 30 (con EarlyStopping).
- **Callbacks:**
 - *EarlyStopping*: detiene el entrenamiento si no mejora la pérdida de validación tras 5 épocas.
 - *ReduceLROnPlateau*: disminuye el learning_rate cuando la validación se estanca.
 - *ModelCheckpoint*: guarda los mejores pesos (best.weights.h5).

Durante el entrenamiento, se monitorearon las métricas accuracy, precision, recall y F1-score. Estas métricas fueron calculadas por lote y promediadas por época, según la metodología propuesta por Goodfellow et al. (2016).

Entrenamiento del modelo Real/Fake

El modelo CNN–BiLSTM–Attention–Transformer fue entrenado durante 16 épocas efectivas, utilizando un esquema de EarlyStopping para evitar el sobreajuste. En las primeras fases del entrenamiento se observó una disminución progresiva de la función de pérdida (loss) junto con un incremento sostenido de la precisión (accuracy), alcanzando una zona de convergencia aproximadamente en la época 12.

Después, entre las épocas 16 y 17, se observó un incremento en la pérdida de validación, con una estabilización de la precisión que no, esto es un claro indicador que el modelo empezó a sobre ajustarse a los datos de entrenamiento.

En este punto, el mecanismo de EarlyStopping detuvo el proceso automáticamente, preservando los mejores pesos que se obtuvieron durante la fase de validación garantizando, así un rendimiento óptimo en datos no vistos.

Los resultados completos del proceso de entrenamiento —incluyendo las métricas finales de precisión, pérdida, AUC y F1-score— se resumen en la Tabla 21, donde se sintetiza el rendimiento del modelo en la tarea de detección de voces reales y generadas por IA.

Tabla 21

Resultados del entrenamiento del modelo Real/Fake

Parámetro	Valor obtenido	Descripción
Épocas efectivas	16	Número total de ciclos de entrenamiento completados antes de la parada temprana (<i>EarlyStopping</i>).
Accuracy (entrenamiento)	0.986	Precisión obtenida durante el entrenamiento.

Accuracy (validación)	0.981	Precisión obtenida durante la validación.
AUC (Área bajo la curva ROC)	0.998	Refleja la capacidad del sistema para diferenciar claramente las clases reales y falsas.
F1-score	0.986	Combina la precisión y sensibilidad en un valor único, equilibrando el desempeño general.
Equal Error Rate (EER)	0.020	Corresponde a la tasa de error donde concuerdan los falsos positivos y falsos negativos, en este caso en 2% de error.
Threshold óptimo (thr*)	0.662	Umbral de decisión para clasificación: $\geq$ 0.662 $\rightarrow$ <i>Fake</i> , $<$ 0.662 $\rightarrow$ <i>Real</i> .
Pérdida mínima (val_loss)	0.081	Valor más bajo de la función de pérdida alcanzado durante el entrenamiento.

Fuente: Elaboración propia

Entrenamiento del modelo de género

El modelo CNN–BiLSTM (Género), al ser menos complejo, presentó una convergencia más rápida y estable.

Durante las primeras 10 épocas ya alcanzaba un rendimiento superior al 95 %. Se decidió detener el entrenamiento en la época 18, punto donde las métricas se estabilizaron, la Tabla 22 muestra los resultados.

Tabla 22

Resultados del entrenamiento del modelo Real/Fake

Parámetro	Valor obtenido	Descripción
Épocas efectivas	18	Número total de épocas ejecutadas hasta convergencia (EarlyStopping).
Accuracy (entrenamiento)	0.972	Precisión promedio durante el entrenamiento.

Accuracy (validación)	0.968	Precisión promedio sobre los datos de validación.
F1-score	0.969	Medida balanceada entre precisión y exhaustividad.
AUC	0.994	Área bajo la curva ROC, indica alta separabilidad entre clases.
Pérdida mínima alcanzada (val_loss)	0.093	Valor más bajo de la función de pérdida durante la validación.

Fuente: Elaboración propia

El desempeño obtenido refleja que el modelo es capaz de capturar diferencias acústicas relacionadas con el timbre, la resonancia y el rango tonal, sin incurrir en sobreajuste.

Validación del modelo

Para evaluar la robustez, se realizaron pruebas de validación cruzada sobre subconjuntos aleatorios del dataset. Los resultados promediados mostraron una varianza menor a ± 0.005 en accuracy y ± 0.01 en F1-score, lo que evidencia estabilidad del modelo ante variaciones de datos.

Además, se verificó la compatibilidad del modelo con TensorFlow Lite, obteniendo coincidencia del 100 % entre las predicciones del modelo .h5 y su versión. tflite.

Este resultado confirma la correcta exportación y optimización del modelo para su posterior uso en la aplicación móvil AntiFakeVoice.

Evaluación de resultados

Una vez completado el entrenamiento, se procedió a evaluar el desempeño de los dos modelos propuestos: el modelo CNN-BiLSTM-Attention-Transformer para la detección de voces reales o falsas, y el modelo CNN-BiLSTM para la clasificación de género.

El propósito de esta fase fue medir la capacidad de generalización de los modelos ante datos no vistos, utilizando métricas ampliamente aceptadas en la literatura de speech anti-spoofing y speaker classification.

De acuerdo con Sahidullah et al. (2021), la evaluación de sistemas de detección de voces sintéticas requiere combinar métricas tradicionales de clasificación con indicadores especializados, como el Equal Error Rate (EER) y el Área Bajo la Curva ROC (AUC), para reflejar la fiabilidad del modelo en la separación entre clases.

Métricas utilizadas

Las métricas de evaluación aplicadas se definieron de acuerdo con las ecuaciones estándar propuestas por Saito & Rehmsmeier (2015) para problemas de clasificación binaria:

1. Precisión (Accuracy):

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Mide el porcentaje total de predicciones correctas.

2. Precisión positiva (Precision):

$$Precision = \frac{TP}{TP + FP}$$

Indica qué proporción de predicciones positivas fueron correctas.

3. Recuperación (Recall o Sensibilidad):

$$Recall = \frac{TP}{TP + FN}$$

Refleja la capacidad del modelo para identificar correctamente las muestras positivas.

4. Puntaje F1:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Representa el equilibrio entre precisión y recuperación.

5. AUC (Área bajo la Curva ROC):

Evalúa la habilidad del modelo para separar las clases, siendo 1.0 el valor ideal.

6. Equal Error Rate (EER):

Corresponde al punto donde la tasa de falsos positivos (FPR) es igual a la de falsos negativos (FNR). Cuanto menor es el EER, mayor es la precisión de la detección.

Resultados del modelo Real/Fake

El modelo CNN–BiLSTM–Attention–Transformer demostró un desempeño sobresaliente en la detección de voces falsas, mostrando alta discriminación entre audios reales y generados por IA. La Tabla 23 muestra las métricas obtenidas:

Tabla 23

Métricas obtenidas en el modelo Real/fake

Métrica	Valor obtenido
Precisión (Accuracy)	98.6 %
Exactitud (Precision)	98.7 %
Recuperación (Recall)	98.5 %
Puntaje F1	0.986
Área bajo la curva (AUC)	0.998
Equal Error Rate (EER)	0.020
Umbral óptimo (threshold*)	0.662
Pérdida mínima de validación	0.081

Fuente: Elaboración propia

El AUC = 0.998 confirma la alta capacidad del modelo para distinguir entre voces reales y falsificadas, mientras que el EER = 0.02 (2 %) indica un nivel muy bajo de error en las clasificaciones.

El umbral óptimo ($\text{thr}^* = 0.662$) se definió mediante el punto de equilibrio entre tasas de error, y fue implementado en la aplicación móvil para la inferencia en tiempo real.

Resultados del modelo de clasificación de género

El modelo CNN-BiLSTM (Género) presentó un rendimiento igualmente alto, demostrando su capacidad para capturar características acústicas relacionadas con el timbre y la resonancia del hablante. Los resultados se detallan a continuación en la Tabla 24:

Tabla 24

Resultados finales del modelo de clasificación de género

Métrica	Valor obtenido
Precisión (Accuracy)	96.8 %
Exactitud (Precision)	96.9 %
Recuperación (Recall)	96.6 %
Puntaje F1	0.969
Área bajo la curva (AUC)	0.994
Pérdida mínima de validación	0.093
Épocas efectivas	18

Fuente: Elaboración propia

El desempeño alcanzado demuestra una clara discriminación entre voces masculinas y femeninas, manteniendo un equilibrio entre precisión y sensibilidad sin tendencia a un género específico.

La diferencia de menos del 1 % entre accuracy y val_accuracy indica una excelente generalización del modelo.

Validación en formato TensorFlow Lite

Una vez completado el entrenamiento y la evaluación de ambos modelos, se procedió a su conversión al formato TensorFlow Lite (TFLite) con el propósito de optimizarlos para su ejecución en la aplicación móvil AntiFakeVoice.

Esta fase de validación fue fundamental para garantizar que los modelos mantuvieran su precisión original y pudieran operar eficientemente en entornos con recursos limitados, como los teléfonos inteligentes con sistema operativo Android.

La conversión se realizó utilizando la herramienta TensorFlow Lite Converter, incluida en la biblioteca TensorFlow.

El procedimiento general consistió en exportar el modelo entrenado en formato .h5 (Keras), Se guardó la arquitectura, pesos y configuración en dos archivos complementarios:

- **best.weights.h5:** contiene los pesos del modelo entrenado.
- **model_config.json:** incluye los parámetros globales (global_mean, global_std, threshold_opt, forma de entrada y nombre de las capas).

El modelo resultante fue almacenado como:

- model_fake_real_fp16_selecttfops.tflite (para detección de voces reales o falsas).
- model_gender_fp16_selecttfops.tflite (para clasificación de género).

Ambos modelos fueron optimizados con precisión FP16 (Half Precision Floating Point), que reduce el tamaño del modelo hasta un 40 % sin afectar significativamente la precisión de las predicciones.

En el capítulo siguiente se aborda el ciclo de vida del software, donde se describe el proceso de integración de estos modelos dentro de una aplicación móvil, así como el diseño e implementación de la solución final.

Capítulo IV

En el siguiente capítulo se presenta el desarrollo integral del prototipo móvil AntiFakeVoice, en el cual se plantea de manera estructurada el ciclo de vida del software con base en su construcción. Se aplican las fases propias de requerimientos, análisis, diseño, implementación y pruebas, dando, así como resultado la trazabilidad entre los objetivos planteados y los resultados obtenidos. Por otra parte, se detallan las decisiones clave que se encuentran relacionadas con la integración de modelos de inteligencia artificial CNN-LSTM mediante TensorFlow Lite y la sincronización de datos a través de Firebase. En general, el capítulo presenta un proceso ordenado el cual asegura el correcto desarrollo metodológico del aplicativo de clonación de voz a través de técnicas de deepfake.

Requerimientos del Sistema

El desarrollo de la aplicación móvil AntiFakeVoice se fundamenta en la definición clara de los requerimientos del sistema, los cuales establecen las funcionalidades esenciales y las condiciones técnicas que guían todo el ciclo de vida del software. Esta etapa tiene gran impacto, ya que permite delimitar el alcance del prototipo y definir las restricciones técnicas necesarias para su correcta ejecución en dispositivos móviles.

Los requerimientos funcionales permiten describir las acciones y funciones que el sistema debe realizar, entre ellas la grabación de audios, la integración de modelos para la detección de voces clonadas y la clasificación de género. Asimismo, se definen los requerimientos no funcionales que incluyen aspectos de rendimiento, usabilidad, seguridad y eficiencia, considerando su ejecución en dispositivos móviles con sistema operativo Android versión 7.0 o superior.

La identificación de los requerimientos forma parte de la base para el análisis, diseño e implementación del sistema, lo que permite garantizar la trazabilidad entre los objetivos del proyecto y las funcionalidades desarrolladas, asegurando que el prototipo responda de forma efectiva a los requerimientos planteados.

Casos de uso (UML)

Los casos de uso permiten representar las interacciones entre los usuarios y el sistema, describiendo de manera estructurada las funcionalidades que la aplicación móvil AntiFakeVoice debe ofrecer para cumplir con los requerimientos funcionales definidos. A través de los diagramas y las descripciones asociadas, se establece una correspondencia directa entre las acciones que el sistema debe realizar y los requerimientos que las originan, garantizando la trazabilidad entre los requerimientos funcionales y el diseño del sistema a lo largo del ciclo de vida del software.

En este proyecto se identifican dos actores principales:

- **Usuario final:** persona que utiliza la aplicación para grabar audios, ejecutar los análisis y consultar los resultados.
- **Sistema AntiFakeVoice:** aplicación móvil responsable de procesar los audios, ejecutar los modelos de inteligencia artificial y gestionar el almacenamiento y visualización de los resultados.

A continuación, se presentan los casos de uso más relevantes del sistema, cada uno descrito en formato de tabla vertical e integrado con el requerimiento funcional correspondiente, de manera que se evidencie claramente cómo cada funcionalidad implementada da cumplimiento a los requerimientos definidos para la aplicación.

Tabla 25

Caso de uso CU1 – Grabar audio

Campo	Contenido
Código	CU1
Nombre	Grabar audio
Actor principal	Usuario final

Requerimiento funcional asociado	RF1: El sistema debe permitir al usuario grabar audios directamente desde el micrófono del dispositivo.
Descripción	El usuario inicia la grabación desde la aplicación y el sistema captura el audio mediante el micrófono del dispositivo.
Precondición	La aplicación debe estar instalada y el micrófono habilitado.
Postcondición	El audio queda almacenado en la memoria local del dispositivo.
Prioridad	Alta

Fuente: Elaboración propia

Tabla 26

Caso de uso CU2 – Clasificar género de la voz

Campo	Contenido
Código	CU2
Nombre	Clasificar género de la voz
Actor principal	Sistema AntiFakeVoice
Requerimiento funcional asociado	RF2: La aplicación debe integrar un modelo de IA que identifique el género de la voz (masculino/femenino).
Descripción	El sistema analiza el audio grabado y determina si corresponde a una voz masculina o femenina.
Precondición	El audio debe estar previamente grabado y disponible.
Postcondición	El resultado del género se muestra en la interfaz.
Prioridad	Media

Fuente: Elaboración propia

Tabla 27

Caso de uso CU3 – Detectar voz real o clonada

Campo	Contenido
Código	CU3
Nombre	Detectar voz real o clonada
Actor principal	Sistema AntiFakeVoice
Requerimiento funcional asociado	RF3: El sistema debe implementar un modelo CNN–LSTM en TensorFlow Lite para clasificar audios como reales o clonados.
Descripción	El sistema aplica el modelo CNN–LSTM para clasificar el audio como voz real o clonada.
Precondición	El audio debe estar disponible y procesado.
Postcondición	El resultado se muestra en la interfaz junto con el nivel de confianza.
Prioridad	Alta

Fuente: Elaboración propia

Tabla 28

Caso de uso CU4 – Almacenar resultados offline

Campo	Contenido
Código	CU4
Nombre	Almacenar resultados offline
Actor principal	Sistema AntiFakeVoice
Requerimiento funcional asociado	RF4: Los resultados deben guardarse localmente cuando no exista conexión a internet.

Descripción	El sistema guarda los resultados de análisis en la memoria local cuando no existe conexión a internet.
Precondición	El dispositivo debe tener espacio de almacenamiento disponible.
Postcondición	Los resultados quedan disponibles para consulta posterior.
Prioridad	Alta

Fuente: Elaboración propia

Tabla 29

Caso de uso CU5 – Sincronizar resultados con Firebase

Campo	Contenido
Código	CU5
Nombre	Sincronizar resultados con Firebase
Actor principal	Sistema AntiFakeVoice
Requerimiento funcional asociado	RF5: Al recuperar la conexión, los resultados deben sincronizarse automáticamente con Firebase.
Descripción	El sistema sincroniza automáticamente los resultados almacenados localmente con Firebase cuando se restablece la conexión.
Precondición	El dispositivo debe tener conexión a internet activa.
Postcondición	Los resultados quedan respaldados en la nube.
Prioridad	Alta

Fuente: Elaboración propia

Tabla 30

Caso de uso CU6 – Consultar historial de resultados

Campo	Contenido
Código	CU6
Nombre	Consultar historial de resultados
Actor principal	Usuario final
Requerimiento funcional asociado	RF7: El sistema debe permitir al usuario consultar el historial de audios analizados y sus resultados.
Descripción	El usuario accede a la opción de historial para visualizar los audios previamente analizados junto con sus resultados.
Precondición	El sistema debe haber almacenado previamente resultados de análisis.
Postcondición	El usuario visualiza un listado con los resultados históricos disponibles.
Prioridad	Media

Tabla 31*Caso de uso CU7 - Visualizar resultados*

Campo	Contenido
Código	CU7
Nombre	Visualizar resultados
Actor principal	Usuario final
Requerimiento funcional asociado	RF6: La aplicación debe mostrar al usuario el resultado del análisis en la interfaz móvil.

Descripción	El sistema muestra al usuario los resultados del análisis del audio procesado, incluyendo género y clasificación real/fake.
Precondición	El audio debe haber sido analizado previamente.
Postcondición	El usuario visualiza los resultados en pantalla.
Prioridad	Alta

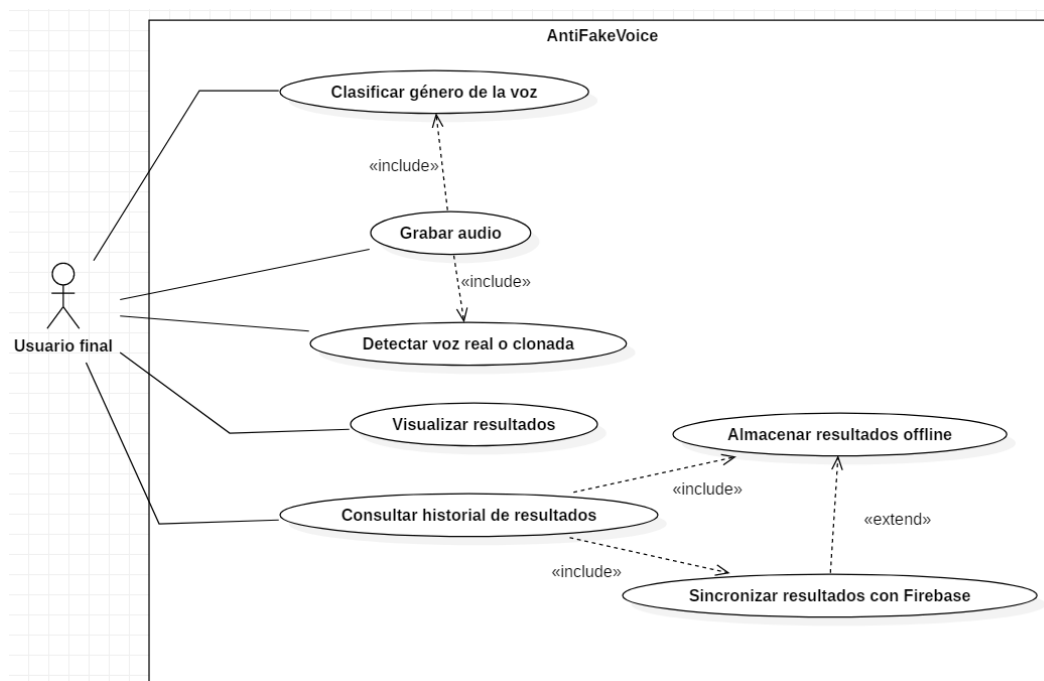
Fuente: Elaboración propia

En la Figura 11 se presenta el diagrama de casos de uso de la aplicación AntiFakeVoice, el cual describe las interacciones entre el usuario final y el sistema, mostrando las funcionalidades principales y las relaciones de dependencia entre ellas.

Las relaciones <<include>> indican que un caso de uso requiere la ejecución de otro, mientras que <<extend>> representa funcionalidades opcionales que complementan el flujo principal.

Figura 11

Diagrama de casos de uso de la aplicación AntiFakeVoice



Fuente: Elaboración propia

Requerimientos no funcionales

Los requerimientos no funcionales establecen las condiciones de calidad y restricciones bajo las cuales debe operar la aplicación móvil AntiFakeVoice. A diferencia de los funcionales, estos no describen acciones específicas del sistema, sino características relacionadas con rendimiento, seguridad, usabilidad y compatibilidad. Su cumplimiento asegura que la aplicación no solo funcione correctamente, sino que también ofrezca una experiencia confiable y eficiente al usuario final.

Cada requerimiento se presenta en formato de tabla vertical como se ve en la Tabla 32, con un código único (RNF), su descripción, el usuario involucrado y la prioridad asignada.

Tabla 32

Requerimientos No Funcionales

Código	Descripción	Usuario	Prioridad
RNF1	El sistema debe garantizar un tiempo de respuesta bajo en la predicción de los modelos, evitando latencias superiores a 3 segundos en dispositivos móviles de gama media.	Usuario final	Alta
RNF2	La aplicación debe optimizar el consumo de recursos (CPU, memoria y batería) para asegurar un funcionamiento estable en dispositivos Android.	Usuario final	Alta
RNF3	La interfaz gráfica debe ser intuitiva y accesible, permitiendo que cualquier	Usuario final	Media

	usuario pueda grabar y analizar audios sin necesidad de conocimientos técnicos.		
RNF4	El sistema debe garantizar la seguridad de los datos almacenados en Firebase, aplicando cifrado en tránsito y mecanismos de autenticación.	Usuario final	Alta
RNF5	La aplicación debe ser compatible con dispositivos Android lanzados a partir del año 2023, asegurando estabilidad en versiones recientes del sistema operativo.	Usuario final	Alta

Fuente: Elaboración propia

Diseño del Sistema

El sistema AntiFakeVoice se diseñó con una arquitectura modular orientada a asegurar la claridad del código, la facilidad de mantenimiento y la posibilidad de ampliar la aplicación sin introducir cambios innecesarios en sus componentes. En este capítulo se presentan los elementos que constituyen la solución propuesta, los cuales permiten comprender la organización de la aplicación. Se detalla la arquitectura general del sistema, así como también el diagrama de componentes, diagrama de clases UML, el modelo entidad relación de la base de datos Firebase, junto con el diseño de la interfaz de usuario. Cada apartado mencionado contribuye a la visualización de la organización de las tareas entre las capas de presentación, lógica y datos, y cómo interactúan entre sí a través de los patrones de diseño aplicados. De este modo, se presenta una visión completa de la solución, en la que se describen la estructura interna y la experiencia de uso prevista para el usuario final.

Arquitectura de la aplicación móvil

El sistema AntiFakeVoice se ha diseñado siguiendo una arquitectura por capas (Layered Architecture), lo que permite separar responsabilidades y reducir el acoplamiento entre los distintos módulos. Esta organización facilita la mantenibilidad, la escalabilidad y la testabilidad del sistema, asegurando que cada capa cumpla un rol específico dentro del flujo de procesamiento.

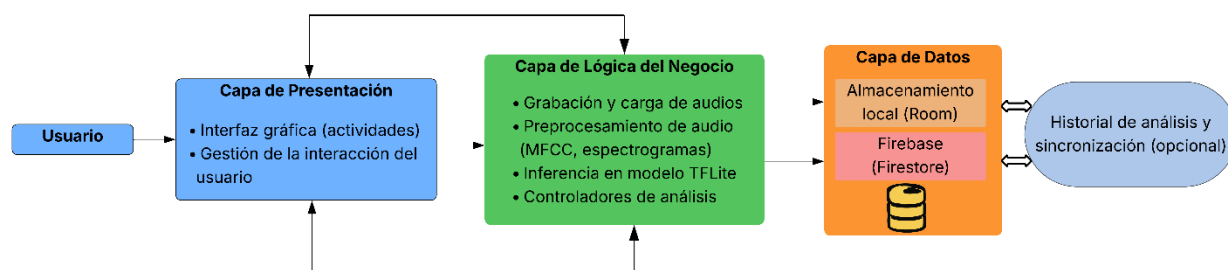
La arquitectura se compone de tres capas principales:

- **Capa de presentación (UI):** encargada de la interacción con el usuario, la captura de audio y la visualización de resultados.
- **Capa de lógica de negocio y procesamiento (ML/Audio):** responsable del preprocesamiento de señales, la ejecución de modelos de aprendizaje profundo y el postprocesamiento de resultados. Aquí se encapsulan los clasificadores de género y los detectores de voz clonada.
- **Capa de datos:** manejo de la información (local y a distancia), maneja los datos guardados, tanto en el aparato mismo como en la nube. Emplea Room para lo local, Firebase/Firestore para lo que está lejos.

En la Figura 12 se presenta la arquitectura general del sistema y se observa la separación entre la capa de presentación, la lógica de negocio y la capa de datos. Esto permite seguir el flujo de la información que parte desde la interacción del usuario hasta el registro de los resultados obtenidos. En ese proceso se observa una organización modular del sistema y el uso de patrones de diseño orientados a mejorar la mantenibilidad y a permitir su crecimiento en el tiempo.

Figura 12

Arquitectura general del sistema AntiFakeVoice.



Fuente: Elaboración propia

Arquitectura del sistema general

El sistema AntiFakeVoice fue diseñado con una arquitectura que separa el proceso de entrenamiento de los modelos de aprendizaje profundo de su ejecución en dispositivos móviles. El entrenamiento de los modelos CNN–LSTM se realiza en un entorno externo que es una PC, donde se dispone de los recursos computacionales necesarios para el procesamiento intensivo de datos. Luego de validar y entrenar los modelos, se exportan y optimizan para su uso en dispositivos móviles.

La aplicación para Android no realiza entrenamiento de modelos, sino que utiliza modelos anteriormente entrenados y convertidos a TensorFlow Lite, los cuales se ejecutan de forma local para procesar audios que el usuario carga o captura. Este planteamiento aporta ciertas ventajas, como lo son la reducción del consumo de recursos del dispositivo móvil, la garantía de tiempos de respuesta rápidos y el resguardo de la privacidad del usuario, ya que el proceso completo del análisis se realiza sin tener la necesidad de enviar los audios a servidores externos.

La estructura del sistema tiene tres partes: entrenamiento offline en PC, exportación y empaquetado del modelo y por último el despliegue e inferencia en Android. Para entrenar, se usa datos especializados en clonación de voz en español como FoR, HISPASpoof y HABLA.

Los audios se procesan primero con normalización, remuestreo a 16 kHz, y la extracción de características acústicas basadas en espectrogramas Log-Mel y coeficientes MFCC. Así, se pueden entrenar modelos CNN-LSTM, capaces de distinguir voces reales y sintéticas, junto a la clasificación del género de quién habla.

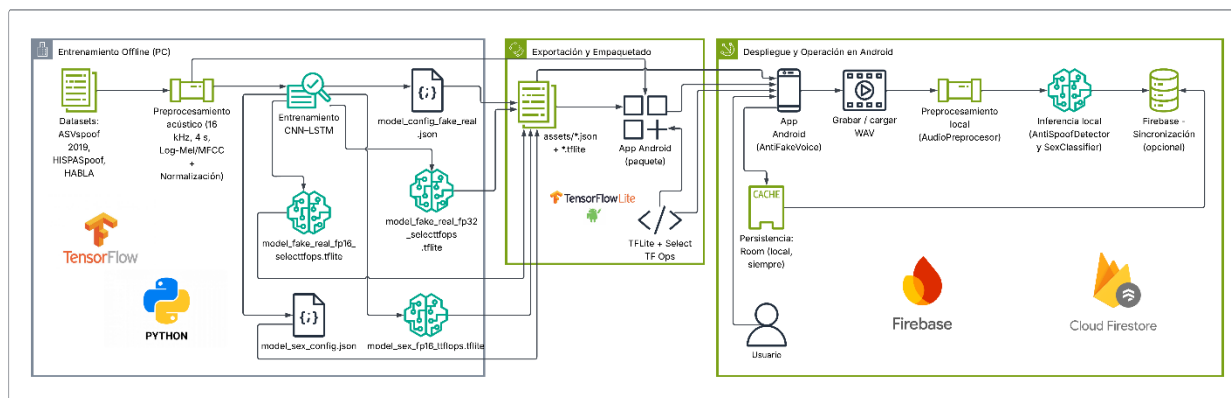
Luego, los modelos adiestrados se transforman al formato TensorFlow Lite (.tflite), esto viene con archivos de configuración en JSON, asegurando la compatibilidad entre el aprendizaje y la inferencia en móviles. Estos recursos se unen a la aplicación Android, también con el runtime de TensorFlow Lite, permitiendo ejecutar modelos eficientemente, sin requerir entrenamiento en el dispositivo.

Durante la ejecución, el prototipo corre en el dispositivo móvil con los modelos ya entrenados y listos para su utilización. El usuario tiene la potestad de grabar o cargar audios WAVs. Cuando el usuario analiza los audios los resultados de si es real o fake y a que genero pertenece se muestran en la interfaz y se guardan en la aplicación localmente. También dichos resultados se sincronizan con los resultados guardados en la nube en Firebase Firestore.

La Figura 13 representa una visión general de la arquitectura del sistema, y explica el flujo completo del sistema que va desde el entrenamiento de los modelos hasta su despliegue en la aplicación Android. Esta propuesta se basa en una arquitectura n-capas ya que permite la organización separada del desarrollo de los modelos hasta su ejecución en el dispositivo.

Figura 13

Arquitectura general del sistema AntiFakeVoice: end-to-end



Fuente: Elaboración propia

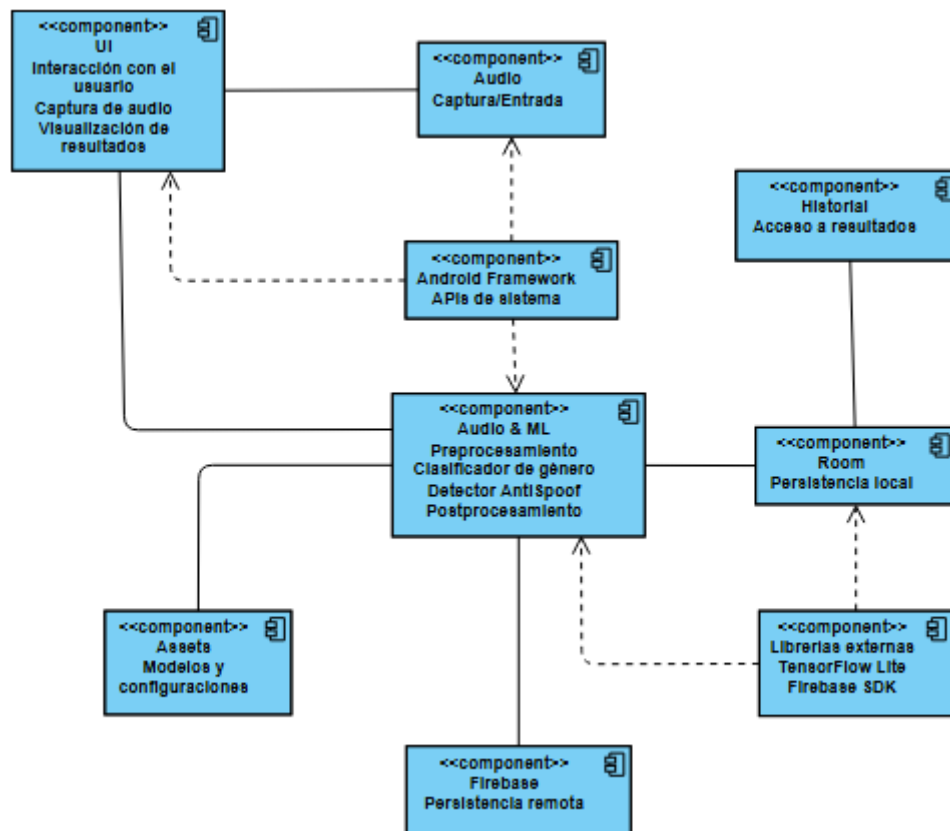
Diagrama de componentes

El diagrama de componentes del sistema AntiFakeVoice expone la estructura modular de la aplicación, definiendo con exactitud las conexiones vitales entre sus piezas esenciales. Dicho diagrama ofrece una visión detallada de la arquitectura física del sistema, detallando las relaciones de dependencia entre sus distintos módulos algo que favorece una planificación más efectiva tanto para el mantenimiento como para futuras expansiones del software.

La Figura 14 presenta un resumen general de los componentes y su organización la interfaz de usuario UI, el módulo que atrapa el audio y el corazón del procesamiento donde se funden los métodos de aprendizaje automático Audio & ML. También están presentes las capas de persistencia local con Room y remota con Firebase/Firestore, y se tiene el registro de resultados. Por último, se mencionan los recursos externos que respaldan la operatividad de la aplicación como los Assets, el Android Framework y las librerías ajenas. Cada componente cumple un rol, y se conectan a otros a través de flujos de datos o dependencias técnicas.

Figura 14

Diagrama de componentes del sistema AntiFakeVoice



Nota. El diagrama fue elaborado por el autor utilizando la herramienta Visual Paradigm Online

El uso de diagramas de componentes se fundamenta en la necesidad de representar sistemas complejos de manera modular. Según Tu (2023), la arquitectura en capas favorece la separación de responsabilidades y la independencia de los módulos, lo que incrementa la mantenibilidad y la reutilización del software. En este sentido, el diagrama de componentes de AntiFakeVoice refleja cómo cada módulo cumple una función particular y se comunica con los demás mediante interfaces bien definidas.

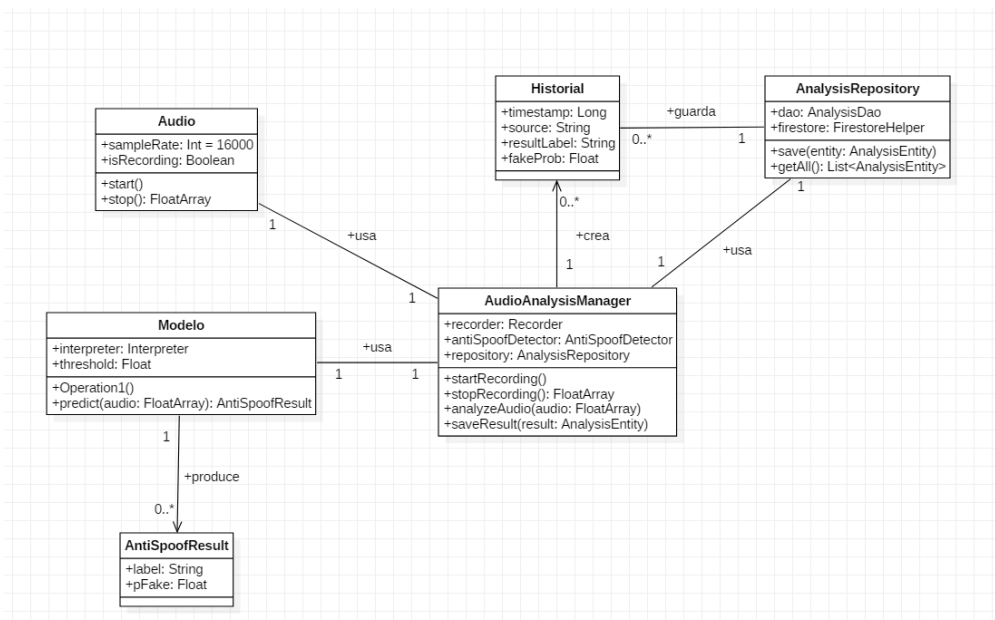
Diagrama de clases (UML)

El diagrama de clases representa la estructura estática del sistema AntiFakeVoice, mostrando las entidades principales, sus atributos, métodos y las relaciones entre ellas. Este tipo de diagrama es fundamental, ya que permite visualizar cómo se organizan los datos y cómo interactúan los componentes internos del sistema.

En la Figura 15 se presenta el modelo de clases elaborado para AntiFakeVoice. Las clases incluidas son: Audio, Modelo, AntiSpooofResult, AudioAnalysisManager, Historial y AnalysisRepository. Cada una encapsula una responsabilidad específica dentro del flujo de análisis de audio y detección de suplantación.

Figura 15

Diagrama de clases del sistema AntiFakeVoice.



Fuente: Elaboración propia

La clase Audio gestiona la captura de señal, mientras que Modelo contiene el intérprete y el umbral de decisión para realizar inferencias. El resultado de la inferencia se encapsula en AntiSpooofResult, que incluye la etiqueta y la probabilidad de falsificación. El controlador principal, AudioAnalysisManager, coordina la grabación, el análisis y el almacenamiento de resultados. Estos resultados se estructuran en la clase Historial, que posteriormente se persiste mediante AnalysisRepository, el cual gestiona tanto el almacenamiento local como remoto.

Las relaciones entre clases se han definido con precisión:

- AudioAnalysisManager se asocia con Audio, Modelo y AnalysisRepository en relaciones 1 a 1.

- Modelo produce múltiples instancias de AntiSpoofResult (1 a 0..*).
- AudioAnalysisManager genera múltiples entradas de Historial (1 a 0..*).
- Historial se compone dentro de AnalysisRepository, indicando una relación de dependencia fuerte (0..* a 1).

Este enfoque se alinea con las prácticas documentadas en la literatura. Koç, Erdoğan, Barjakly y Peker (2021) destacan que los diagramas de clases son los más utilizados en ingeniería de software, ya que permiten modelar la estructura interna de los sistemas y sus relaciones. Además, el uso de clases con responsabilidades bien definidas contribuye a la mantenibilidad y escalabilidad del sistema, principios fundamentales en el diseño arquitectónico moderno.

Diagrama entidad-relación (ER)

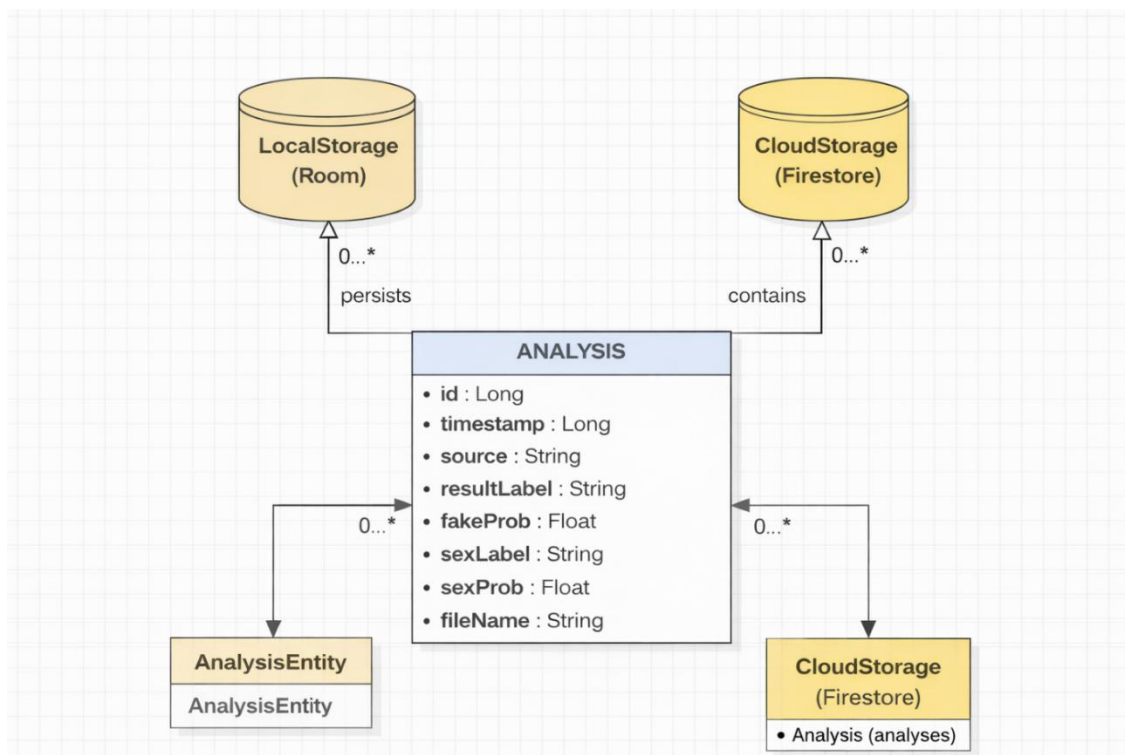
El modelo entidad-relación (ER) del sistema AntiFakeVoice permite representar la estructura lógica de los datos relacionados con los resultados de análisis de audio. Si bien el diagrama fue realizado utilizando notación UML de clases, su estructura mantiene los principios importantes de un modelo entidad-relación, identificando correctamente las entidades, los atributos y relaciones existentes entre los distintos componentes del sistema.

La Figura 16 muestra la entidad principal llamada ANALYSIS, la cual agrupa toda la información recopilada en el proceso del análisis. Dicha entidad posee atributos como id, timestamp, source, resultLabel, fakeProb, sexLabel, sexProb y fileName, que en conjunto permiten detallar el contexto y el resultado que se obtiene en cada análisis del sistema.

La entidad ANALYSIS se relaciona con tres componentes clave: LocalStorage, CloudStorage y AnalysisEntity, cada uno de los cuales gestiona la persistencia y representación de los datos en distintos niveles.

Figura 16

Modelo entidad-relación del sistema AntiFakeVoice.



Fuente: Elaboración propia

Las relaciones entre entidades se definen como uno a muchos, de manera que tanto LocalStorage, implementado con Room, como CloudStorage, implementado con Firestore, pueden almacenar múltiples instancias de ANALYSIS. En este contexto, AnalysisEntity se entiende como una abstracción que permite agrupar varios registros de análisis dentro del sistema. Esta estructura organiza la gestión de los datos de manera sistemática, tanto en el almacenamiento local como en el remoto, y conserva la trazabilidad de los resultados derivados del proceso de inferencia. El uso de este modelo se ajusta a las prácticas recogidas en la literatura especializada sobre el modelado de sistemas. Koç, Erdoğan, Barjakly y Peker (2021) indican que los diagramas UML, aunque surgieron en el ámbito del modelado orientado a objetos, también pueden servir para representar estructuras de datos similares a las de los modelos ER, especialmente cuando se requiere una descripción detallada de los atributos y de

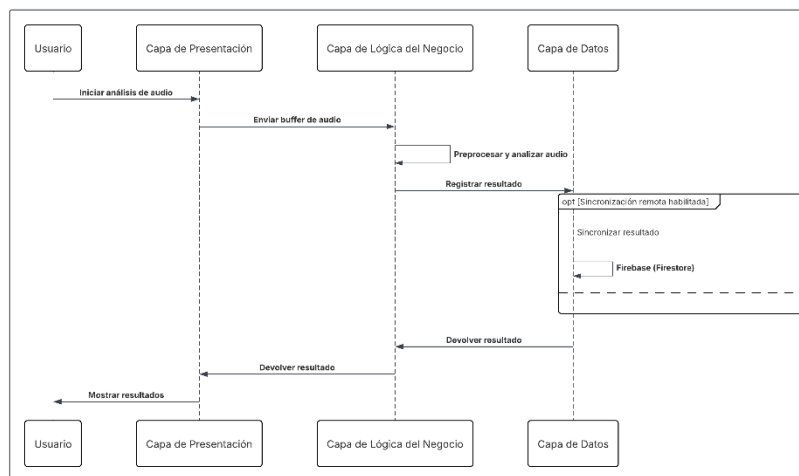
las relaciones. Esta flexibilidad permite adaptar la notación UML a los objetivos del modelado lógico en los sistemas actuales.

Diagrama de secuencia

El diagrama de secuencia describe el flujo de interacción entre los distintos componentes de la aplicación AntiFakeVoice durante la ejecución de un análisis de voz. La Figura 17 presenta el diagrama de secuencia del flujo general de análisis de voz en la aplicación AntiFakeVoice. El proceso inicia cuando el usuario solicita el análisis del audio a través de la interfaz. La capa de presentación maneja la interacción y transmite información a la capa de lógica del negocio, es donde se trabaja en el preprocesamiento de la señal y, usando modelos de aprendizaje profundo, la inferencia se hace. Después de eso, los hallazgos del análisis se guardan en la capa de datos, eso sí que incluye almacenamiento local y una sincronización a distancia. Por último, los resultados son entregados de vuelta a la capa de presentación para ser mostrados al usuario, así conservando la división de tareas entre las capas.

Figura 17

Diagrama de secuencia del flujo de análisis de voz en la aplicación AntiFakeVoice



Fuente: Elaboración propia

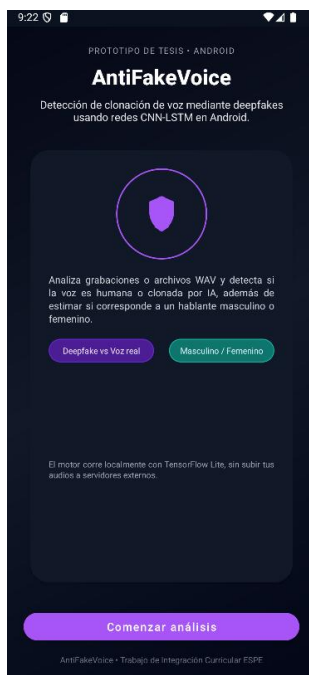
Diseño de la interfaz de usuario

La interfaz de usuario del sistema AntiFakeVoice fue diseñada con un enfoque minimalista y funcional, orientado a facilitar la interacción directa con las capacidades de análisis de audio. El diseño prioriza la claridad visual, la accesibilidad y la eficiencia operativa, permitiendo al usuario ejecutar tareas clave como grabar, cargar, analizar y consultar resultados sin necesidad de registro ni configuración avanzada.

En la Figura 18 se presenta la pantalla de bienvenida de la aplicación, donde se introduce el propósito del sistema: detectar clonación de voz mediante redes neuronales CNN-LSTM en dispositivos Android. Se incluyen botones de acceso directo a las funciones principales de análisis, así como una nota aclaratoria que indica que el procesamiento se realiza localmente con TensorFlow Lite, sin enviar los audios a servidores externos.

Figura 18

Pantalla de bienvenida de AntiFakeVoice



Fuente: Elaboración propia

La Figura 19 muestra la pantalla principal de análisis, donde se despliegan los parámetros técnicos del audio (SR=16k, duración, número de mels) y se ofrecen acciones como grabar, detener, cargar archivos WAV, reproducir, analizar y consultar el historial. La sección de resultados se actualiza dinámicamente tras cada análisis, mostrando las probabilidades de detección de deepfake y la clasificación de género.

Figura 19

Pantalla principal de análisis de audio



Fuente: Elaboración propia

Finalmente, la Figura 20 presenta la pantalla de historial, donde se listan los análisis realizados con sus respectivos metadatos: fecha, resultado (real o falso), clasificación de género, nivel de confianza, nombre del archivo y duración. Esta vista permite al usuario revisar y validar los resultados obtenidos, reforzando la trazabilidad del sistema.

Figura 20*Pantalla de historial de resultados***Fuente:** Elaboración propia

El diseño de la interfaz se basa en principios de usabilidad como la retroalimentación inmediata, la consistencia visual, la reducción de carga cognitiva y la accesibilidad. Los botones están claramente etiquetados, los colores y tipografías mantienen coherencia, y los resultados se presentan de forma comprensible incluso para usuarios no técnicos. Esta aproximación permite que la aplicación sea utilizada de forma autónoma, rápida y segura.

Implementación y Despliegue

La fase de implementación y despliegue se refiere a la puesta en marcha de la propuesta, en la que se incorporan los modelos de aprendizaje profundo y los componentes funcionales del prototipo de la aplicación móvil AntiFakeVoice. En esta etapa se describen los aspectos técnicos necesarios para integrar los modelos de clasificación de género y de

detección de voz real o falsa, y para su ejecución en dispositivos móviles mediante TensorFlow Lite, con atención a los requisitos de implementación y al rendimiento durante la inferencia.

Además, se mostró de manera explícita el procedimiento para ejecutar el prototipo en un entorno de prueba, lo que posibilitó su funcionamiento y verificación en dispositivos Android. Es importante mencionar que este despliegue no es apto para un entorno de producción, sino para un entorno controlado en el cual se realizan pruebas de funcionalidad e integración. Para la gestión de resultados, se agregaron dos mecanismos de control: uno sin conexión el cual utiliza almacenamiento local, y el otro con conexión el cual se encarga de sincronizar los datos con Firebase Firestore de inmediato cuando la señal se reconecta.

Por último, se analizan los mecanismos de interacción entre los distintos módulos del sistema y los ajustes aplicados para aumentar la precisión del análisis y optimizar el rendimiento del prototipo en dispositivos móviles con recursos limitados, de modo que se asegure su funcionamiento adecuado durante la fase de pruebas y validación.

Dependencias y librerías utilizadas

Para el desarrollo de la aplicación móvil AntiFakeVoice se emplearon diversas dependencias y librerías del ecosistema Android, seleccionadas con el objetivo de garantizar compatibilidad, rendimiento y facilidad de mantenimiento. A continuación, se describen las principales herramientas utilizadas y su función dentro del sistema.

Android SDK y herramientas base

- Android SDK (Kotlin): utilizado como plataforma principal de desarrollo para la implementación de la lógica de la aplicación, la gestión del ciclo de vida, permisos, interfaz de usuario y acceso a recursos del dispositivo.
- AndroidX: conjunto de librerías de soporte modernas que facilitan el desarrollo de interfaces, compatibilidad entre versiones de Android y buenas prácticas de arquitectura.

Procesamiento y aprendizaje profundo

- **TensorFlow Lite:** librería empleada para la ejecución local de modelos de aprendizaje profundo en dispositivos móviles. Permite realizar inferencias de manera eficiente, con bajo consumo de recursos y sin necesidad de conexión permanente a internet.
- **TensorFlow Lite Select TF Ops:** complemento utilizado para soportar operaciones avanzadas del modelo que no se encuentran disponibles en el conjunto estándar de operaciones de TensorFlow Lite.

Manejo de audio

- **AudioRecord (Android):** API nativa utilizada para la captura de audio desde el micrófono del dispositivo en formato PCM, permitiendo controlar parámetros como frecuencia de muestreo y canales.
- **Utilidades de procesamiento de audio personalizadas:** clases propias desarrolladas para la conversión de formatos, normalización, preprocesamiento y extracción de características requeridas por los modelos de aprendizaje profundo.

Persistencia de datos

- **Room (AndroidX):** librería de persistencia local utilizada para almacenar los resultados de los análisis en el dispositivo. Facilita el acceso a datos mediante el patrón DAO y proporciona una capa de abstracción sobre SQLite.
- **Firebase Firestore:** base de datos NoSQL en la nube empleada para el almacenamiento y consulta del historial de análisis. Permite sincronización en tiempo real y escalabilidad sin necesidad de administrar infraestructura propia.
- **Concurrencia y rendimiento**

- **Kotlin Coroutines:** utilizadas para la ejecución de tareas asíncronas, tales como procesamiento de audio, inferencia con modelos y operaciones de entrada/salida, evitando el bloqueo del hilo principal y mejorando la experiencia de usuario.

Interfaz de usuario

- **Material Components for Android:** librería utilizada para el diseño de la interfaz gráfica siguiendo las guías de Material Design, proporcionando componentes visuales consistentes y accesibles.
- **RecyclerView:** componente empleado para la visualización eficiente del historial de análisis.

Integración del modelo de clasificación de género

La integración del modelo de clasificación de género se implementó como un módulo adicional orientado a enriquecer el resultado del análisis principal. En el proyecto, el modelo se incorpora en formato TensorFlow Lite (.tflite) dentro de la carpeta de recursos `app/src/main/assets/`, junto con su archivo de configuración `model_sex_config.json`, el cual almacena parámetros de preprocesamiento y dimensiones esperadas de entrada. La lógica del modelo está encerrada en la clase `SexClassifier` (paquete `ml`). Esta carga el intérprete de TensorFlow Lite y realiza la inferencia a través de un método de predicción `predict(audio FloatArray)`.

Antes de la inferencia, el audio capturado o cargado se transforma, imitando el procedimiento del entrenamiento. Este proceso lo hace la clase `AudioPreprocessor` (paquete `ml`). Allí es donde se realizan tareas como normalizar la señal, ajustar la duración, y extraer características como la representación mel-espectral. El propósito es asegurar que la entrada del modelo tenga la misma forma (`shape`) y distribución, para no tener degradaciones por los diferentes pipelines.

Después de la inferencia, la salida del modelo se entiende como una probabilidad vinculada a la clase de género. Esa información se transforma en una etiqueta que el usuario comprende. Este resultado se devuelve a través de una estructura de salida, como SexResult y se combina con el flujo principal de la app para que se muestre en la interfaz, además, opcionalmente, que quede guardado con el resto del análisis.

Integración del modelo de detección real/fake

El modelo que permite detectar si el audio es real o falso, es el componente central de la aplicación. Su integración se realiza mediante el formato .tflite del modelo dentro de las carpetas app/src/main/assets/, también se agrega la configuración de este el cual lleva por nombre model_config_fake_real.json. Este archivo permite documentar parámetros clave del pipeline como: frecuencia de muestreo, longitud de segmento, características y umbrales, lo que ayuda reproducir el modelo obtenido del entrenamiento con el modelo utilizado en la aplicación móvil.

La ejecución del modelo reside en la clase AntiSpoofDetector del paquete ml, cargando el modelo empleando el intérprete TensorFlow Lite para después realizar la predicción en el audio ingresado denominado predict(audio: FloatArray). Antes de que se invoque el modelo, el audio pasa por AudioPreprocessor, dónde se aplica normalización, manejo de silencios, además extrayéndose características acústicas con base en los parámetros configurados.

Esta división entre inferencia y preprocesamiento reduce el enredo, esto hace factible en el mantenimiento, ya que se puede ajustar el pipeline sin tener que modificar la lógica de ejecución del modelo.

El resultado de inferencia se considera como una probabilidad para la clasificación “real” o “fake”, creándose una etiqueta final visible en la interfaz de usuario. En la aplicación, el análisis total se inicia desde la pantalla principal tipo MainActivity en el paquete ui, este controla la captura de audio con Recorder del paquete audio, y la lectura de archivos con herramientas como WavIO. Finalmente, el resultado se consolida como un registro de análisis (por ejemplo,

AnalysisEntity) y se almacena para su consulta posterior mediante persistencia local y/o remota.

Gestión offline/online con Firebase

La aplicación AntiFakeVoice implementa una gestión híbrida offline/online para el almacenamiento y consulta de los resultados de análisis, combinando persistencia local y almacenamiento en la nube mediante Firebase Firestore. El uso de esta estrategia permite que la aplicación siga funcionando de forma normal incluso cuando el dispositivo no dispone de conexión a internet, asegurando que los resultados obtenidos a partir del análisis de los audios no se pierdan y puedan ser consultados después por el usuario, sin que la disponibilidad de la información dependa de la conectividad del momento.

Se uso Firebase/Firestore como almacenamiento en la nube, el cual se integra al proyecto a través del SDK ofrecido por Android. Para poder integrarlo, dentro del paquete data.db se crea la clase FirestoreHelper la cual permite la comunicación de este servicio. La clase también permite guardar y recuperar los resultados de los análisis

Cuando el dispositivo cuenta con acceso a internet, los resultados generados tras el análisis de un audio se envían automáticamente a Firestore. Donde se almacenan como documentos dentro de una colección específica, como por ejemplo analyses. Cada documento corresponde a un análisis individual e incluye datos importantes como la etiqueta de clasificación obtenida, las probabilidades asociadas, el origen del audio analizado y la fecha y hora en la que se realizó el proceso. Esto permite que la información quede registrada y pueda ser consultada posteriormente, además de hacer posible la sincronización entre distintos dispositivos.

En los casos cuando no hay conectividad, la aplicación mantiene su funcionamiento mediante el uso de almacenamiento local. En este escenario, los resultados del análisis se guardan inicialmente en una base de datos implementada con Room, utilizando la entidad AnalysisEntity y los métodos de acceso definidos en AnalysisDao, ambos pertenecientes al

paquete data.db. La base de datos local actúa como respaldo temporal, facilitando al usuario continuar utilizando el aplicativo, realizar nuevos análisis y consultar el historial aun cuando no dispone de internet.

La alineación entre el almacenamiento local y remoto se realiza a través de a capa de control de la aplicación, ya sea MainActivity o el componente controlador indicado. Desde esta capa se establece si los resultados se guardan solo de forma local o si se sincronizan con Firestore cuando hay conexión de red accesible. Esta manera de trabajo contribuye a evitar que la lógica principal del sistema funcione estrictamente del estado de la red, lo cual facilita a que la aplicación sea fiable y estable en varios contextos de uso.

Además, la gestión offline/online con Firebase mejora significativamente la experiencia del usuario, garantizando el continuo funcionamiento de la aplicación y reduciendo el riesgo de pérdida de información. De igual manera, esta táctica potencia la escalabilidad del sistema, permitiendo así combinar servicios en la nube sin afectar la operación local ni exponer la privacidad de los audios procesados y analizados.

Ajustes de precisión y optimización de los modelos

La precisión y el rendimiento de los modelos de aprendizaje profundo en dispositivos móviles dependen en gran medida de la coherencia entre el proceso de entrenamiento y la implementación en producción, así como del uso eficiente de los recursos del dispositivo. En la aplicación AntiFakeVoice, se realizaron diversos ajustes para mejorar la confiabilidad de los resultados y optimizar la ejecución de los modelos en entornos móviles.

El preprocesamiento de audios es un aspecto importante ya que este proceso se encuentra centralizado en la clase AudioPreprocessor (paquete ml), donde se aplican operaciones como la normalización de la señal, el ajuste de la duración del audio y la extracción de características acústicas.

Estos procedimientos replican las condiciones utilizadas durante el entrenamiento de los modelos, permitiendo reducir el impacto de variaciones en el volumen, la duración o el formato

del audio de entrada, lo que permite mantener el preprocesamiento consistente y así evitar las degradaciones de los modelos.

Adicionalmente se realizaron ajustes en la forma en la que se manejan los resultados del modelo de detección real o falso en la clase AntiSpoofDetector. En esta clase del sistema se delimitaron criterios para la precisión, los cuales permiten procesar la salida directa del modelo en resultados más entendibles para el usuario, de manera que contribuye a alcanzar predicciones más aceptables durante el proceso del análisis. Dichos criterios establecidos se pueden modificar a partir de pruebas realizadas con audios reales, lo que posibilita al sistema adaptar su comportamiento al tipo de uso previsto y minimizar errores que se puedan presentar en la clasificación.

Sobre el rendimiento, se optimizó la ejecución de los modelos al usar TensorFlow Lite, para hacer las inferencias más eficientes. La creación del intérprete se maneja en las clases AntiSpoofDetector y SexClassifier, para asegurar que los modelos se carguen solo una vez durante la aplicación, evitando recargas raras que pueden impactar el tiempo de respuesta. También, las tareas de inferencia y procesos pesados se corren fuera del hilo principal, usando mecanismos concurrentes como Kotlin Coroutines, para que la interfaz se vea bien.

Otro ajuste importante es el manejo del tamaño y cuánto tiempo se analiza el audio. En la práctica, se fija la duración de los fragmentos de audio a procesar, así se trata de asegurar que la entrada a los modelos tenga un tamaño constante. Este control se hace antes de empezar la inferencia y ayuda a que los resultados sean más consistentes y también hace que computación sea más eficiente.

Por último, la arquitectura de módulos del sistema permite agregar mejoras más adelante para optimizarlo, como la cuantización de modelos, y también emplear aceleradores de hardware en ciertos celulares, o bien, actualizar las versiones de TensorFlow Lite. Con estos cambios se podrá mejorar el rendimiento sin afectar la precisión, esto garantiza que la aplicación pueda usarse en diferentes aparatos y en varios escenarios.

Diagrama de despliegue

El diagrama de despliegue del prototipo ilustra la manera en que la aplicación AntiFakeVoice se ejecuta dentro de un entorno controlado de pruebas, evidenciando la interacción entre los componentes físicos y lógicos que garantizan su funcionamiento y validación. Cabe señalar que este despliegue no corresponde a un escenario productivo, sino que se orienta exclusivamente a habilitar la ejecución del prototipo y a facilitar la realización de pruebas funcionales, de integración y de rendimiento.

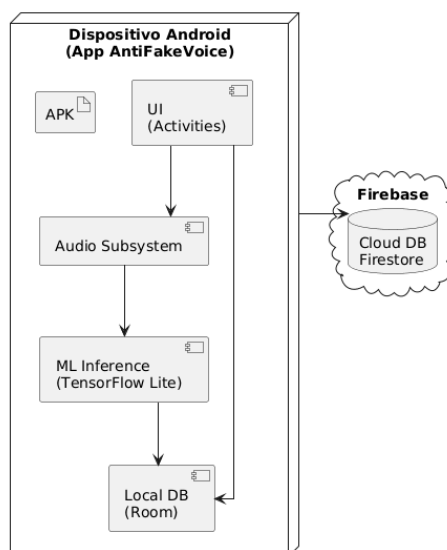
La aplicación se instala localmente a través de un archivo APK, en un dispositivo Android. El dispositivo, a su vez, reúne los módulos de interfaz, captura de audio, procesamiento de la señal, inferencia basada en modelos de aprendizaje profundo y también persistencia local de resultados. El proceso para detectar voces reales o falsas, y clasificar el género del hablante, se realiza directamente en el móvil usando TensorFlow Lite; así, el análisis no requiere enviar datos a servidores externos. Esta aproximación ayuda a disminuir la latencia, mientras fortalece la privacidad del usuario.

Respecto al manejo del historial, el prototipo utiliza una persistencia híbrida combinando almacenamiento local con sincronización remota. Los datos se guardan en la base interna del dispositivo. Cuando el dispositivo se conecta a internet se sincronizan con Firebase/Firestore, una base de datos en la nube. Los datos del análisis como la clasificación, la marca temporal almacena. Los se audios quedan en el dispositivo, asegurando control sobre la data delicada.

Finalmente, la Figura 21 muestra el diagrama de despliegue del prototipo, donde se observa la ejecución de la aplicación en un dispositivo Android y su interacción con Firebase Firestore para la persistencia de resultados. Los modelos de aprendizaje profundo se llevan a cabo localmente a través de TensorFlow Lite, mientras que en la nube se emplea únicamente como repositorio y consulta de la información generada en los análisis de los audios.

Figura 21

Diagrama de despliegue de la aplicación AntiFakeVoice



Fuente: Elaboración propia

Pruebas y Validación

El proceso de pruebas y validación tiene como objetivo comprobar el correcto funcionamiento de la aplicación AntiFakeVoice, así como evaluar la integración y el desempeño de los modelos de aprendizaje profundo en un entorno móvil real. Esta etapa es crucial ya que permite verificar que las funciones principales cumplan su objetivo y que los módulos tengan una correcta interacción entre ellos, incluyendo la captura de audio, el preprocesamiento, la inferencia con TensorFlow Lite y la gestión de los resultados tanto local como remota.

Asimismo, se analizan aspectos como el rendimiento, la usabilidad y la confiabilidad de los resultados, con el objetivo de que el sistema cumpla con los requisitos planteados.

Estrategia de pruebas

La estrategia de pruebas diseñada para la aplicación AntiFakeVoice tuvo como propósito verificar el correcto funcionamiento de las funcionalidades implementadas y la integración adecuada de los modelos de aprendizaje profundo en un entorno móvil. Dado que se trata de un prototipo que combina captura de audio, procesamiento de señales y persistencia de datos, se optó por un enfoque de pruebas principalmente manual y

experimental, orientado tanto a evaluar el desempeño técnico general como la experiencia del usuario.

Las pruebas se ejecutaron de manera incremental. En una primera fase se validaron las funcionalidades básicas de la aplicación, como la grabación y carga de audio. Después, se calificó el rendimiento de los modelos. En los siguientes pasos, se examinó la interacción entre las diferentes partes del sistema, como la interfaz, los procesadores de audio, los modelos de inferencia y el almacenamiento, tanto local como remoto para asegurar que el análisis funcione correctamente.

Igualmente, la estrategia consideró varios casos, como el uso de la aplicación con o sin internet, analizar audios en tiempo real y los archivos ya subidos y cambios en cuanto a cuánto duraban las señales que entraban. Estas condiciones ayudaron a evaluar el rendimiento del prototipo y cómo se comportaba en situaciones reales. Por último, se incorporaron pruebas para evaluar la eficiencia y la facilidad de uso, con la idea de verificar los tiempos de respuesta correctos y garantizar una experiencia de usuario clara y fácil de entender.

Pruebas funcionales

Las pruebas funcionales se orientaron a verificar el correcto desempeño del sistema desarrollado, evaluando su capacidad para procesar audios en español latinoamericano y distinguir entre voces auténticas y voces generadas mediante técnicas de inteligencia artificial. Como variable complementaria de análisis se consideró el género del hablante. Para ello, se diseñó un conjunto de pruebas basado en muestras de audio balanceadas, con el objetivo de comprobar la operatividad del prototipo en condiciones controladas. Los resultados obtenidos durante la evaluación del sistema se muestran mediante tablas y gráficos. De esta forma se facilita la revisión de los datos y permite observar claramente cómo responde el sistema ante los distintos tipos de audio analizados y el género del hablante.

Las pruebas funcionales se realizaron utilizando un conjunto de 40 audios en español latinoamericano. Dicho conjunto fue seleccionado de manera equilibrada para evaluar el comportamiento del sistema en diferentes condiciones. En total, se trabajó con 20 audios reales y 20 audios artificiales. Estos se distribuyeron equitativamente en función del género, considerando audios masculinos y femeninos tanto reales como artificiales. La duración de las muestras de audio se mantuvo entre 5 y 15 segundos, con el fin de evitar que la longitud del audio afecte de forma directa en los resultados obtenidos.

Los audios reales corresponden a grabaciones de hablantes ecuatorianos y representan condiciones normales de habla.

Por otra parte, los audios falsos fueron generados mediante la técnica de conversión de texto a voz. Para esto se utilizó la plataforma ElevenLabs, en la cual se ingresaron diferentes frases y se selecciono diferentes tipos de voces tanto masculinas como femeninas. Esto audios falsos permitieron comparar su análisis frente a audios generados por sistemas de clonación de voz.

Durante la evaluación, los audios fueron analizados individualmente, por lo que se realizó el ingreso manualmente al sistema, también se ingresaron al sistema audios que fueron grabados directamente por el sistema. Gracias a esto se permitió confirmar el correcto funcionamiento del sistema bajo condiciones que un usuario lo empelaría de forma normal.

Por otra parte, al ingresar un audio, el sistema comenzó a ejecutarse para realizar el respectivo análisis, esto incluyo la fase de preprocesamiento de la señal y la inferencia del modelo de detección correspondiente, dando como resultado la clasificación del audio junto a valores de probabilidad.

Este procedimiento se aplicó de manera uniforme a la totalidad de las muestras, garantizando que todos los audios fueran evaluados bajo las mismas condiciones de ejecución, las cuales se resumen en la Tabla 33. Dichas condiciones permitieron controlar factores como

el entorno de ejecución, el método de ingreso del audio, la duración de las muestras y el tipo de inferencia utilizada, asegurando la consistencia de los resultados obtenidos.

Tabla 33

Condiciones de ejecución de las pruebas funcionales

Parámetro	Condición
Tipo de sistema	Prototipo de aplicación móvil
Plataforma	Android
Entorno de ejecución	Entorno controlado
Modalidad de ejecución	Instalación local mediante archivo APK
Tipo de inferencia	Inferencia local en el dispositivo
Arquitectura del modelo	CNN-LSTM
Idioma de los audios	Español latinoamericano
Tipo de audio evaluado	Real y fake
Género del hablante	Masculino y femenino
Duración de los audios	Entre 5 y 15 segundos
Método de ingreso de audio	Grabación directa y carga de archivo
Número total de audios evaluados	40

Fuente: Elaboración propia

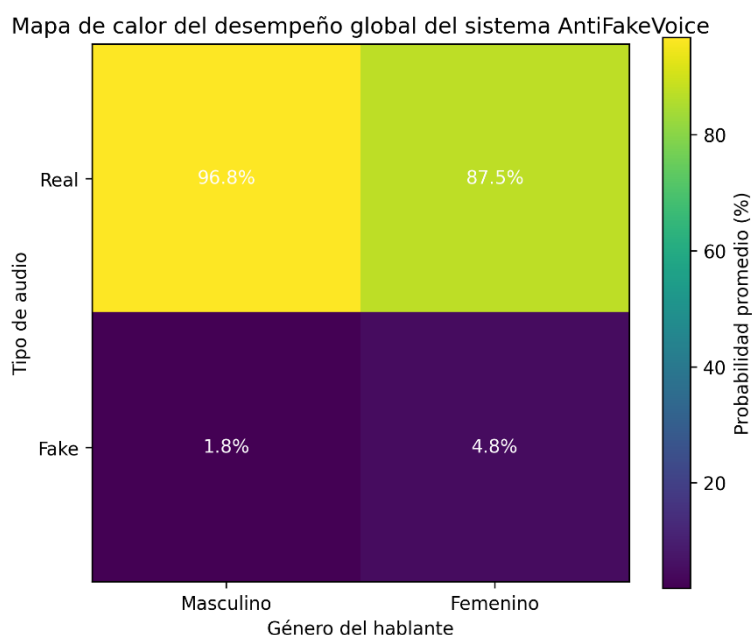
Con el objetivo de evaluar de manera integral el desempeño del sistema AntiFakeVoice, se realizó un análisis global considerando los resultados obtenidos durante las pruebas funcionales aplicadas al conjunto de audios de prueba. El análisis este permite observar el comportamiento general del modelo, del detector de clonación de voz, frente a una variada combinación de audios (reales y falsos), y el género del hablante (masculino o femenino).

Y para comprender mejor los resultados, también para sintetizar cómo funciona el sistema, se usó un mapa de calor para visualizar, que resume las probabilidades promedio, asignadas por el modelo, a cada escenario estudiado. Esa visualización ayuda a identificar

patrones de acierto, y variaciones en la confianza del sistema, más algunas diferencias en el rendimiento, según el género, todo para dar una visión total del prototipo, en un ambiente controlado. Esta representación permite identificar de forma visual las diferencias en el comportamiento del modelo ante distintas condiciones de entrada.

Figura 22

Análisis global del sistema



Fuente: Elaboración propia

Por la parte de los audios reales correspondientes a voces masculinas, en general el sistema establece probabilidades altas a la clase Real, indicando que el modelo puede distinguir fácilmente patrones acústicos únicos de este tipo de señales. Del mismo modo, los audios falsos con voces masculinas suelen ser clasificados con probabilidades grandes en la clase Fake, lo que apunta a una detección más concisa de las voces artificiales dentro de este grupo.

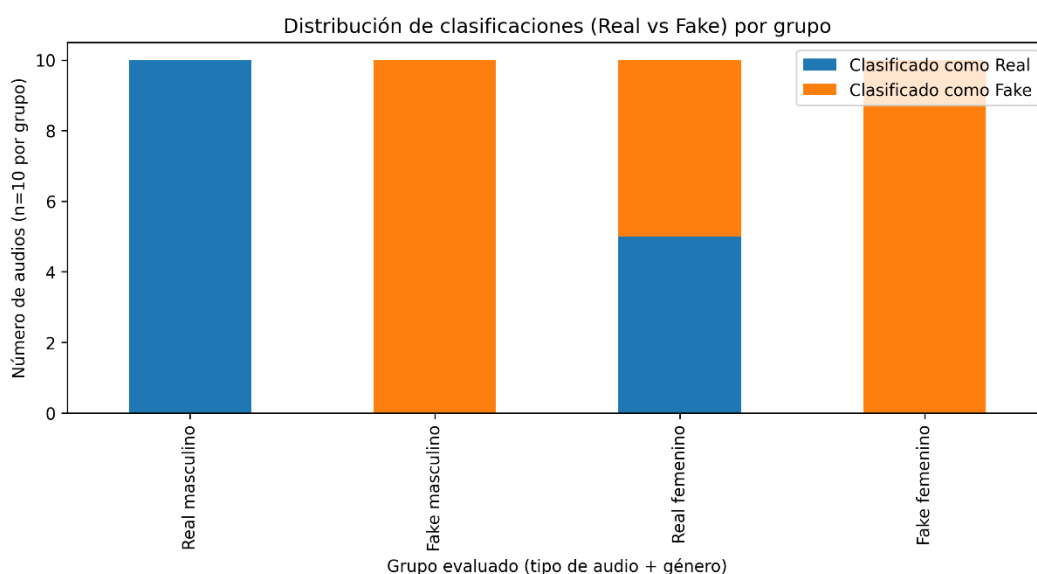
Analizando los audios con voces femeninas, el sistema se comporta irregular. Especialmente, los audios femeninos reales indican mayor variación en las probabilidades designadas, con valores mínimos para la clase Real. Esto demuestra que el modelo presenta más inconvenientes para diferenciar audios con voces femeninas reales y las generadas de forma artificial, específicamente cuando los mismo comparten propiedades sonoras semejantes.

Este comportamiento muestra que el rendimiento del sistema no es tan homogéneo entre diferentes géneros, lo cual es aceptable tomando en cuenta que se trata de un prototipo en etapa de evaluación y que está dispuesto a futuras mejoras.

En la Figura 23 se indica la distribución de las clasificaciones conseguidas para cada grupo evaluado, a partir del punto de partida del número de audios identificados como Real y Fake. Esta representación permite divisar de forma general la tendencia del modelo y la manera en la que se toman las decisiones de clasificación.

Figura 23

Distribución de clasificaciones



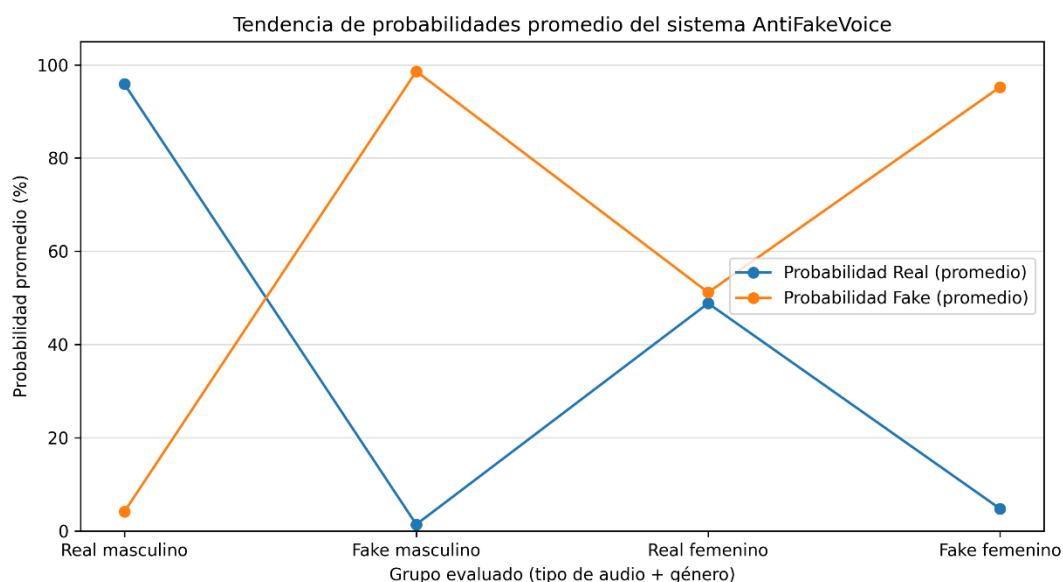
Fuente: Elaboración propia

En los audios masculinos, tanto reales como fake, el sistema presenta una clasificación consistente y coherente con la naturaleza de las muestras, lo que confirma la estabilidad del modelo frente a este tipo de voces. En contraste, en los audios femeninos se identifican casos de confusión, principalmente en muestras reales que son clasificadas como fake, lo que refuerza la hipótesis de una mayor complejidad acústica en la voz femenina para este tipo de modelo.

A pesar de estas confusiones, el sistema mantiene una tendencia general correcta en la detección de audios fake femeninos, lo que indica que, aunque el reconocimiento del género presenta limitaciones, la detección de clonación de voz continúa siendo funcional en la mayoría de los casos.

La Figura 24 presenta la tendencia de las probabilidades promedio asignadas por el sistema a las clases *Real* y *Fake* para cada grupo evaluado. Este análisis permite comparar de forma directa la respuesta del modelo ante diferentes combinaciones de tipo de audio y género.

Los resultados evidencian una clara separación entre las probabilidades asignadas a audios reales y fake en el caso de voces masculinas, lo que demuestra una alta capacidad discriminativa del modelo en este escenario. Sin embargo, en el caso de las voces femeninas, las curvas asociadas a ambas clases se aproximan, indicando un solapamiento parcial en las características aprendidas por el modelo.

Figura 24*Tendencia de probabilidades promedio***Fuente:** Elaboración propia

Esta tendencia confirma que el prototipo presenta un desempeño diferenciado según el género del hablante, siendo más preciso en voces masculinas. No obstante, el comportamiento observado es coherente con la naturaleza experimental del sistema y pone en evidencia la necesidad de ampliar y diversificar el conjunto de entrenamiento en futuras versiones.

Pruebas de integración

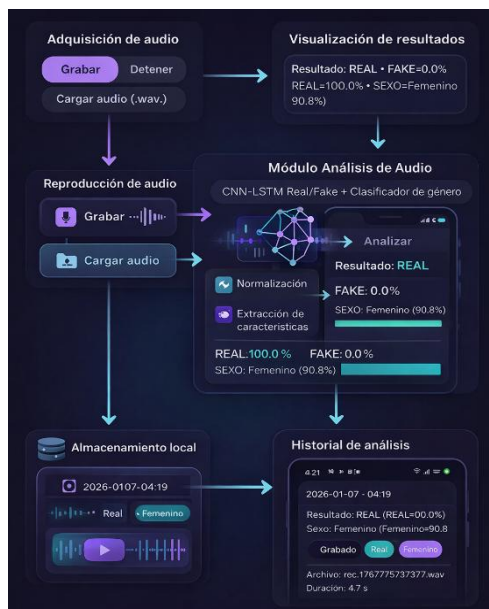
Las pruebas de integración se realizaron con el propósito de validar la correcta interoperabilidad entre los módulos que conforman el sistema AntiFakeVoice, asegurando que el flujo de datos se mantenga coherente desde la adquisición del audio hasta la persistencia final de los resultados. Dado que la aplicación integra procesamiento de señal, modelos de aprendizaje profundo y mecanismos de almacenamiento, este tipo de pruebas resulta crítico para garantizar el funcionamiento del sistema como una unidad integrada.

La Figura 25 muestra, el flujo general de la integración de los módulos del sistema, las dependencias funcionales entre cada componente. El proceso comienza, adquiriendo audio, ya sea grabando en tiempo real o subiendo archivos, creando una señal digital enviada al módulo

de preprocesamiento. Este módulo normaliza la señal, extrayendo las características acústicas necesarias para los modelos de inferencia asegurando que la entrada de datos se ajuste a los parámetros requeridos.

Figura 25

Flujo de integración de los módulos del sistema AntiFakeVoice



Fuente: Elaboración propia

Una vez completado el preprocesamiento, las características extraídas son enviadas al módulo de inferencia basado en una arquitectura CNN-LSTM. En esta etapa, el sistema ejecuta de forma coordinada dos tareas de clasificación: la detección de audio real o generado por inteligencia artificial y la identificación del género del hablante. La correcta integración de los modelos permitió obtener resultado combinados en un solo ciclo de análisis, lo que permite reducir el riesgo de inconsistencias y la pérdida de información.

Después de los análisis, el módulo de visualización recibe los resultados dando paso a que el usuario observe dichos resultados, en ello se encuentran las probabilidades asignadas y clasificación del audio. También, los datos que se generaron durante el análisis se agrupan en un registro donde el usuario puede ver los resultados de audios ya analizados con anterioridad. Estos registros se guardan primero en la aplicación, lo que permite que se conserve en caso de

no tener acceso a internet, Cuando el dispositivo móvil dispone de conexión a internet, los resultados del análisis de cada audio se sincronizan con la base de datos remota en Firebase Firestore.

La integración propuesta permite conservar un registro de los análisis realizados y mantener la coherencia entre el almacenamiento local y el remoto. También posibilita verificar el funcionamiento del sistema en condiciones sin acceso a red y con acceso a red, de modo que se compruebe que los datos se gestionan de forma consistente en ambos casos.

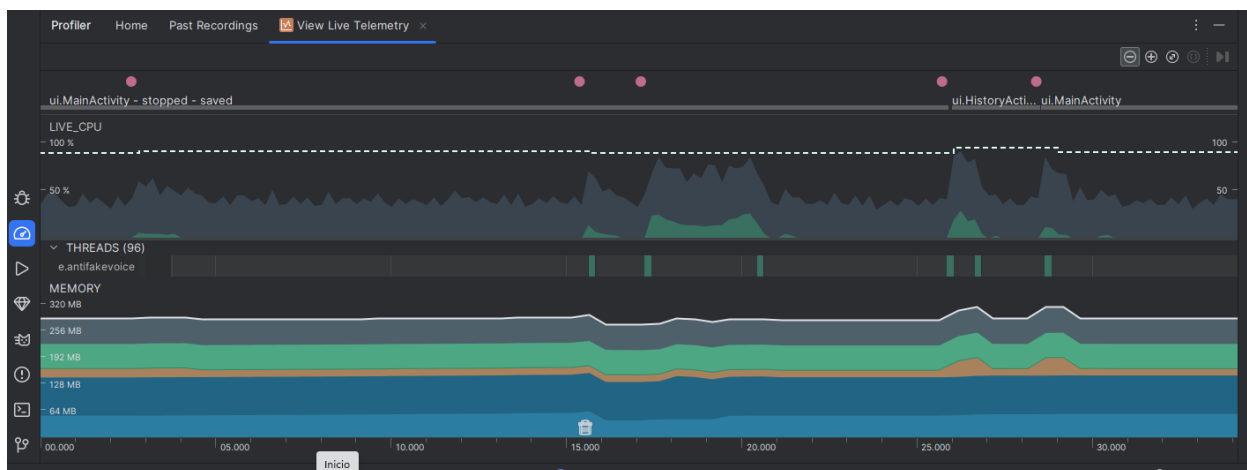
Evaluación de rendimiento

La evaluación de rendimiento se realizó con el objetivo de analizar el comportamiento de la aplicación AntiFakeVoice bajo diferentes condiciones de carga, considerando la duración del audio procesado. Para ello, se utilizó la herramienta View Live Telemetry del profiler integrado de Android Studio, la cual permitió monitorear en tiempo real el uso de CPU, memoria y la actividad de los hilos durante la ejecución del análisis en un dispositivo móvil físico.

En una primera prueba, se evaluó el rendimiento del sistema durante el análisis de audios con una duración mayor a 10 segundos. En este escenario, en la Figura 26 se observó un incremento más pronunciado en el uso de CPU durante el procesamiento del audio y la ejecución de los modelos de aprendizaje profundo. No obstante, una vez finalizado el análisis, el consumo de CPU retornó progresivamente a niveles estable, evidenciando que el sistema gestiona correctamente la carga computacional generada por audios de mayor duración. Asimismo, el consumo de memoria mostró un aumento controlado seguido de una estabilización, sin presentar crecimientos continuos que indiquen fugas de memoria.

Figura 26

Monitoreo de CPU y memoria durante el análisis de audio con duración mayor a 10 segundos mediante View Live Telemetry

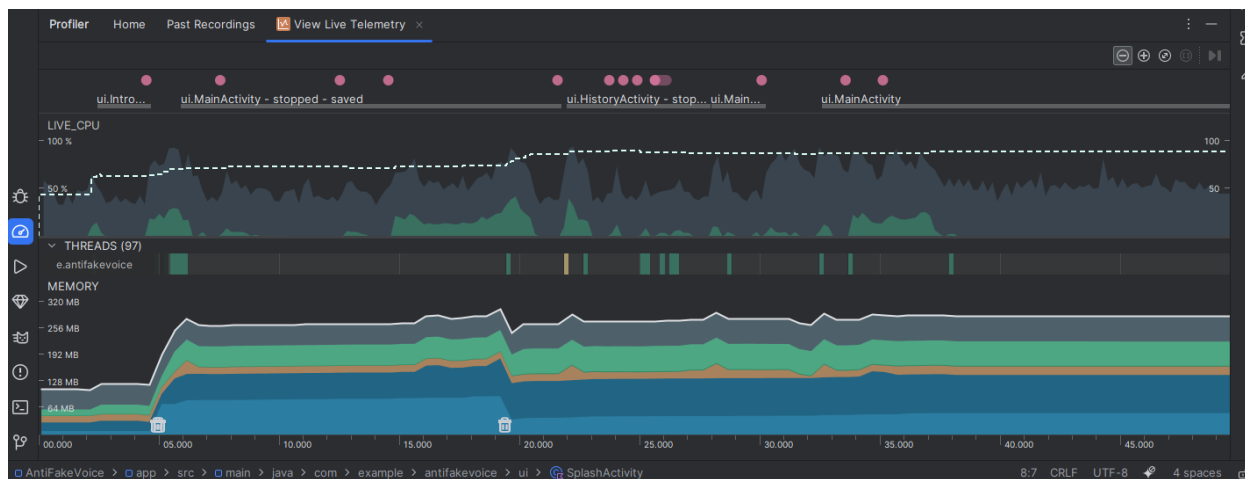


Fuente: *Elaboración propia.*

Posteriormente, se realizó una prueba de rendimiento utilizando audios con una duración menor a 10 segundos. En este caso, el uso de CPU presentó picos de menor magnitud y una duración más corta en comparación con el escenario anterior, lo cual es coherente con la reducción en la cantidad de datos procesados. El consumo de memoria se mantuvo estable durante toda la ejecución del análisis, confirmando que el sistema responde de manera eficiente cuando se procesan audios de menor duración como se muestra en la Figura 27.

Figura 27

Indica el monitoreo del consumo de la CPU y memoria durante el análisis de audios con menos de 10 segundos, realizado a través de la herramienta View Live Telemetry.



Fuente: Elaboración propia.

Comparando los distintos escenarios evaluados, se evidencia que el rendimiento del aplicativo AntiFakeVoice varía en función de la duración del audio analizado. Sin embargo, el funcionamiento del sistema se mantiene estable en todos los casos evaluados, debido a que el proceso principal de análisis se ejecuta en hilos secundarios, previniendo interrupciones en el hilo principal de la interfaz. Por esta razón, el usuario puede continuar haciendo uso de la aplicación sin complicaciones notables durante el análisis.

En conjunto, los resultados muestran que el aplicativo se desempeña adecuadamente en dispositivos móviles. El sistema tiene la capacidad de procesar audios con distinta duración de manera que el uso sea razonable en cuanto al uso de recursos del dispositivo y en tiempos de respuesta apropiados a contextos de uso normal.

Usabilidad y experiencia de usuario

La evaluación de la usabilidad y de la experiencia del usuario se realizó con el propósito de conocer la percepción de los usuarios sobre el uso del prototipo AntiFakeVoice. Para ello, se tomó en cuenta qué tan fácil es interactuar con la aplicación, si la presentación de los

resultados es clara y si el funcionamiento general del sistema se percibe como coherente durante su uso.

Esta evaluación se desarrolló de forma cualitativa y en un entorno controlado, lo cual permitió observar el correcto comportamiento del aplicativo, concluyendo que se trata de un prototipo funcional y aceptable.

Considerando la Calidad de Experiencia (QoE), primero se pudo evaluar la interfaz principal del prototipo, la misma que presenta una cantidad reducida de acciones, las cuales están bien explicadas y entre ella se tiene: la grabación de audio, la carga de archivos, la ejecución del análisis y la consulta del historial. Esto facilita que el usuario entienda de manera inmediata el funcionamiento del prototipo.

También se evaluó la forma en que el sistema muestran los resultados. La aplicación presenta de manera clara si el audio es real o sintético, junto con los porcentajes asociados y la clasificación del género del hablante. Esta organización de la información hace que sea fácil interpretar y evitar confusiones durante el uso de la aplicación.

Otro aspecto considerado dentro de la QoE fue la retroalimentación que ofrece la aplicación durante su uso. El prototipo informa de manera clara cuándo se está reproduciendo el audio y cuándo el análisis ha finalizado, lo que permite que el usuario tenga una mejor idea del estado del proceso y se sienta más seguro al interactuar con la aplicación.

En la parte del diseño visual, la interfaz se presenta simple y coherente durante el uso. Los colores e ilustraciones gráficas contribuyen a reconocer fácilmente los resultados, de manera que la interacción con el aplicativo es clara y apta para su finalidad.

Del mismo modo, se revisó la navegación global del prototipo y se confirmó que el acceso a sesiones se ejecuta con naturalidad, como lo es el caso del historial del análisis, ya que no presenta interrupciones ni dificultades para el usuario.

En conjunto, los resultados obtenidos evidencian que el prototipo AntiFakeVoice ofrece una calidad de experiencia adecuada, permitiendo una interacción comprensible y satisfactoria dentro de un entorno controlado.

Análisis de resultados

Analizando los resultados obtenidos, después de realizar pruebas funcionales, integración y rendimiento, se muestra que el sistema AntiFakeVoice responde de forma muy satisfactoria a los criterios de validación predefinidos para este prototipo. Las pruebas funcionales lograron constatar que los módulos de grabación, carga, análisis, y visualización de resultados funcionan sin problemas, dentro de las condiciones previstas, produciendo clasificaciones coherentes, reflejando la naturaleza verdadera o sintética de los audios evaluados. En el ámbito de la integración, los resultados confirman la perfecta interacción entre los modelos de detección de clonación de voz y la clasificación de género, además de su persistencia en la base de datos local y, la sincronización opcional con el servicio remoto, sin ninguna pérdida de la consistencia de los datos.

Por otra parte, el desempeño de cada uno de los modelos muestra que la probabilidad de clasificación es adecuada gracias a su separación. Este resultado se da gracias a la arquitectura híbrida empleada, ya que permite captar patrones espectro-temporales que ayudan a distinguir los audios en español.

En los análisis, se obtuvo que el modelo de clasificación de género tiene un rendimiento estable y presenta resultados similares a los esperados, por lo que la extracción de rasgos acústicos resultó apropiado.

Finalmente, las pruebas de rendimiento llevadas a cabo en dispositivos móviles revelaron que el uso de CPU y memoria se mantiene en niveles razonables para dispositivos Android, inclusive en audios más extensos, con esto, se validó la viabilidad técnica para la implementación de modelos de aprendizaje profundo con TensorFlow Lite.

En resumen, los resultados de las pruebas confirman la funcionalidad, la integración y la eficiencia del sistema, se demostró que el prototipo alcanza las metas establecidas y resulta técnicamente viable como herramienta preventiva para combatir la suplantación de identidad por clonación de voz.

Capítulo V

Conclusiones

- Tras analizar toda la literatura existente, se evidencio que los modelos híbridos CNN–LSTM superan por mucho a los métodos clásicos en la detección de audios falsos. Dichos modelos poseen una gran habilidad para captar tanto las características espectrales como las temporales, lo cual los convierte en la mejor opción en la detección de deepfakes de voz.
- El sistema se diseñó con una arquitectura de n capas, lo que facilitó la separación modular de las funciones principales: captura, preprocesamiento, inferencia y almacenamiento del audio. Esta estructura demostró ser eficiente, contribuyendo así al mantenimiento y a la capacidad de ampliación, para lograr construir una herramienta robusta y flexible.
- Al integrar los modelos CNN–LSTM en el prototipo AntiFakeVoice, desarrollado en Kotlin y optimizado con TensorFlow Lite, se demostró que es posible usar redes neuronales profundas en entornos móviles. El prototipo demostró estabilidad tanto en modo online como offline, almacenando los resultados localmente y sincronizándolos posteriormente con Firebase.
- Las pruebas realizadas con audios auténticos y generados artificialmente evidenciaron que el modelo alcanzó métricas de precisión, sensibilidad y especificidad favorables, confirmando su validez en la detección de clonación de

voz. Se observaron diferencias en el rendimiento entre voces masculinas y femeninas, que mostraron un consumo moderado de CPU y memoria, exponiendo un funcionamiento estable del prototipo en dispositivos móviles Android.

Recomendaciones

- Se propone ampliar y diversificar el conjunto de datos de entrenamiento mediante la incorporación de audios en español latinoamericano que reflejen mayor variabilidad en timbre, entonación, niveles de ruido ambiental y tipos de dispositivos de grabación. Esta ampliación busca reducir sesgos del modelo y mejorar la capacidad de generalización del sistema CNN–LSTM, con atención particular a voces femeninas y a escenarios acústicos no controlados.
- Se plantea ajustar el modelo para que se adapte en dispositivos móviles y así reducir el tamaño y los recursos computacionales del mismo, manteniendo un nivel aceptable de precisión. Con esto se busca que el sistema funcione de manera estable en teléfonos Android de gama media y baja.
- Se plantea que la etapa de preprocesamiento de la señal del audio mejore mediante la incorporación de normalización adaptativa, corrección de ganancia y reducción de ruido, con el fin de que el sistema siga manteniendo un desempeño estable frente a audios de duración variable y condiciones que se ajuste a un entorno real.
- Se recomienda complementar la validación técnica con pruebas con usuarios, mediante evaluaciones de experiencia de uso (UX) que permitan examinar la comprensión del funcionamiento, el nivel de confianza y la aceptación del sistema por parte de los usuarios finales. De este modo, se podrá valorar la

aplicabilidad práctica del prototipo en condiciones de uso reales, más allá de los resultados asociados únicamente a su rendimiento técnico.

Trabajos Futuros

Para futuras investigaciones, la idea es entrenar y evaluar varios modelos para detectar deepfakes vocales, usando arquitecturas super avanzadas, como Conformer y CNN–Transformer, y comparar sus resultados con el modelo CNN–LSTM propuesto antes. Este estudio será importantísimo, especialmente donde haya voces de mujer y audios sintéticos de alta calidad que dan mucho lío a los sistemas.

También, el plan es ampliar el prototipo a una plataforma multiusos, con versiones adaptadas para móviles y una interfaz web para mirar qué onda. La meta es recopilar estadísticas, meter los resultados juntos, y medir bien el rendimiento. Todo esto lo hará manteniendo el procesamiento del audio en el móvil, para cuidar la privacidad de los usuarios y tener un lugar seguro.

Referencias

- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., ... Zheng, X. (2016). *TensorFlow: Large-scale machine learning on heterogeneous distributed systems*. arXiv. <https://arxiv.org/abs/1603.04467>
- Ai, Y., Liu, X., Li, Z., & Meng, H. (2021). Denoising-and-dereverberation hierarchical neural vocoder for robust waveform generation. *Proceedings of the IEEE Spoken Language Technology Workshop (SLT)*, 477–480.
- Arif, T., Javed, A., & Shafique, M. (2021). Voice spoofing countermeasure for logical access attacks detection. *IEEE Access*, 9, 162857–162870. <https://doi.org/10.1109/ACCESS.2021.3132974>
- Asuai, C., Arinomor, P., Atumah, C., Kowhoro, I., & Ogheneochuko, D. (2025). Hybrid CNN–LSTM architectures for deepfake audio detection using Mel frequency cepstral coefficients and spectrogram analysis. *American Journal of Mathematical and Computer Modelling*, 10(3), 98–109. <https://doi.org/10.11648/j.ajmcm.20251003.12>
- Dai, D., Li, J., & Yang, Y. (2022). Cloning one's voice using very limited data in the wild. *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 8322–8325.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Hassan, F., Ahmed, M., & Ali, S. (2021). Voice spoofing countermeasure for synthetic speech detection. *Proceedings of the International Conference on Artificial Intelligence (ICAI)*, 209–212.
- Jung, J., Tak, H., & Yu, M. (2022). *Improved RawNet2 for anti-spoofing detection*. arXiv. <https://arxiv.org/abs/2203.07205>
- Koç, H., Erdoğan, A. M., Barjakly, Y., & Peker, S. (2021). UML diagrams in software engineering research: A systematic literature review. *Journal of Systems and Software*, 175, 110932.

- Kumar, B., & Alraisi, A. (2022). Deepfakes audio detection techniques using deep convolutional neural network. *Proceedings of COM-IT-CON*, 463–466.
- Lee, J., Kim, K.-H., & Park, S. (2021). Zero-shot voice cloning using variational embedding with attention mechanism. *Proceedings of IC-NIDC*, 344–348.
- Li, Y., Zhang, X., & Wang, Y. (2022). Attention-based CNN–LSTM for speech anti-spoofing. *arXiv*. <https://arxiv.org/abs/2204.08967>
- Liang, H., Liu, Y., & Huang, J. (2017). Recognition of spoofed voice using convolutional neural networks. *Proceedings of IEEE GlobalSIP*, 293–297.
- Park, D. S., Chan, W., Zhang, Y., Chiu, C.-C., Zoph, B., Cubuk, E. D., & Le, Q. V. (2019). *SpecAugment: A simple data augmentation method for automatic speech recognition*. *arXiv*. <https://arxiv.org/abs/1904.08779>
- Prajapati, G., Patil, H., & Prasanna, S. (2020). Energy separation based features for replay spoof detection for voice assistant. *Proceedings of EUSIPCO*, 386–390.
- Rufamapi, R., & Barrios-Quintana, D. (2022). *Latin America Spanish anti-spoofing dataset (FoR)*. European Open Science Platform.
- Sagara, N. K., & Arukonda, S. (2025). A novel CNN–LSTM approach for robust deepfake detection. *Procedia Computer Science*, 258, 1844–1855. <https://doi.org/10.1016/j.procs.2025.04.436>
- Saito, T., & Rehmsmeier, M. (2015). The precision–recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>
- Seong, J., Lee, K., & Kim, H. (2021). Multilingual speech synthesis for voice cloning. *Proceedings of IEEE BigComp*, 313–316.
- Shang, J., Wu, Z., & King, S. (2018). Defending against voice spoofing: A robust software-based liveness detection system. *Proceedings of IEEE MASS*, 28–36.

- Stevens, S. S. (1937). A scale for the measurement of the psychological magnitude of pitch. *The Journal of the Acoustical Society of America*, 8(3), 185–190.
- Tanveer Gujjar, M. U., Ahmed, S., & Khan, A. (2024). Unmasking the fake: Machine learning approach for deepfake voice detection. *IEEE Access*, 12, 197442–197456.
- Todisco, M., Wang, X., Vestman, V., Sahidullah, M., Delgado, H., Nautsch, A., ... Lee, K. A. (2019). *ASVspoof 2019: Future horizons in spoofed and fake audio detection*. arXiv. <https://arxiv.org/abs/1904.05441>
- Tu, Z. (2023). Research on the application of layered architecture in computer software development. *Journal of Computing and Electronic Information Management*, 34–38.
- ViperLab, Purdue University. (2023). *HISPASpoof: A dataset for Spanish speech anti-spoofing*. <https://huggingface.co/datasets/viperlab/HISPASpoof>
- Zhang, C., & Wang, Y. (2021). Detection of AI-synthesized speech using CNN–LSTM networks. *IEEE Transactions on Neural Networks and Learning Systems*.
- Zhang, H., Li, J., & Deng, L. (2022). Improve few-shot voice cloning using multi-modal learning. *Proceedings of IEEE ICASSP*, 8317–8321.